

Омский государственный технический университет

И. В. Червенчук, А. С. Грицай

ОСНОВЫ КОМПЬЮТЕРНОЙ АРИФМЕТИКИ

Учебное пособие

Учебное текстовое локальное электронное издание

*Рекомендовано редакционно-издательским советом
Омского государственного технического университета*

Омск
Издательство ОмГТУ
2024

Сведения об издании: [1](#), [2](#), [3](#)

© ОмГТУ, 2024
ISBN 978-5-8149-3789-6

УДК 004.9(075)
ББК 32.973.202+22.12я73
Ч-45

Рецензенты:

О. Н. Лучко, к.п.н., профессор, зав. каф. информатики,
математики и естественнонаучных дисциплин
ЧУ ОО ВО «Омская гуманитарная академия»;

В. А. Маренко, к.т.н., доцент, ст. науч. сотр. Омского филиала
ФГБУН «Институт математики им. С. Л. Соболева СО РАН»

Червенчук, И. В. Основы компьютерной арифметики : учеб. пособие / И. В. Червенчук, А. С. Грицай ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2024. – 1 CD-ROM (1,10 Мб). – Минимальные систем. требования: процессор с частотой 800 МГц и выше ; 128 Мб RAM и более ; свободное место на жестком диске 300 Мб и более ; Linux / Windows XP и выше ; MacOS X 10.4 и выше ; CD/DVD-ROM-дисковод ; ПО для просмотра pdf-файлов. – Загл. с титул. экрана. – ISBN 978-5-8149-3789-6.

Изложены основы двоичной компьютерной арифметики. Приведены примеры алгоритмов выполнения арифметических операций в ЭВМ для двоичных чисел в формате с плавающей запятой и в двоично-десятичных кодах. Рассмотрены методы реализации основных арифметических операций посредством цифровых устройств. Описаны алгоритмические модели выполнения арифметических операций и методы их проектирования.

Издание предназначено для обучающихся по направлению 09.03.01 «Информатика и вычислительная техника».

Учебное текстовое локальное электронное издание

Червенчук Игорь Владимирович

Грицай Александр Сергеевич

ОСНОВЫ КОМПЬЮТЕРНОЙ АРИФМЕТИКИ

Учебное пособие

Издание поставляется на одном CD-ROM-диске

Воспроизведение издания автоматическое –
без установки на жесткий диск компьютера

Для корректной работы с изданием на компьютере должны быть установлены
CD/DVD-ROM-дисковод и программное обеспечение
для просмотра pdf-файлов

Редактор *Т. А. Москвитина*

Компьютерная верстка *Ю. П. Шелехиной*

Для дизайна обложки использованы материалы
из открытых интернет-источников

Иллюстрации для издания предоставлены авторами

Сводный темплан 2024 г.

Подписано к использованию 13.05.2024.

Объем 1,10 Мб. Тираж 9 эл. опт. дисков.

Издательство ОмГТУ. 644050, г. Омск, пр. Мира, 11; т. 8(3812)23-02-12.

Эл. почта: publisher@omgtu.ru, izdomgtu@mail.ru

ОГЛАВЛЕНИЕ

ПРЕДИСЛОВИЕ	6
1. ОСНОВЫ ДВОИЧНОЙ КОМПЬЮТЕРНОЙ АРИФМЕТИКИ.....	7
1.1. Позиционные системы счисления и выполнение арифметических операций	7
1.2. Представление чисел с плавающей запятой (точкой).....	10
1.3. Представление отрицательных чисел	11
1.4. Переполнения при выполнении операций	13
2. ОСНОВЫ РАЗРАБОТКИ УСТРОЙСТВ ДВОИЧНОЙ АРИФМЕТИКИ	15
2.1. Базовые цифровые узлы	15
2.2. Реализация операции умножения.....	18
2.2.1. Реализация операции умножения методом младшими разрядами вперед	18
2.2.2. Реализация операции умножения методом старшими разрядами вперед	21
2.2.3. Реализация операции умножения в дополнительных кодах	24
2.2.4. Реализация операции умножения в обратных кодах.....	26
2.2.5. Ускоренные методы умножения.....	28
2.3. Реализация операции деления	36
2.3.1. Реализация операции деления на устройстве.....	38
с подвижным сумматором в прямых кодах.....	38
2.3.2. Реализация операции деления на устройстве с неподвижным сумматором в прямых кодах.....	44
2.3.3. Реализация операции деления в дополнительных кодах	48
2.3.4. Ускоренные методы деления	54

2.4. Реализация операции извлечения квадратного корня.....	65
3. ОСНОВЫ РАЗРАБОТКИ УСТРОЙСТВ	
ДВОИЧНО-ДЕСЯТИЧНОЙ АРИФМЕТИКИ	73
3.1. Двоично-десятичные коды	73
3.2. Умножение в двоично-десятичном коде	80
3.2.1. Умножение в двоично-десятичном коде младшей	
секцией вперед.....	80
3.2.2. Умножение в двоично-десятичном коде старшей секцией	
вперед	84
3.3. Деление в двоично-десятичном коде	88
3.3.1. Деление в двоично-десятичном коде со сдвигом	
сумматора	88
3.3.2. Деление в двоично-десятичном коде с неподвижным	
сумматором	98
4. РАЗРАБОТКА СТРУКТУРЫ И АЛГОРИТМА	
ФУНКЦИОНИРОВАНИЯ АРИФМЕТИЧЕСКОГО УСТРОЙСТВА	104
4.1. Разработка структуры устройства	104
4.2. Разработка алгоритма функционирования устройства	106
4.3. Построение таблиц условий и сигналов	114
ЗАКЛЮЧЕНИЕ	117
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	118

ПРЕДИСЛОВИЕ

В учебных планах подготовки бакалавров по направлению «Информатика и вычислительная техника» дисциплина «Арифметические и логические основы вычислительной техники» входит в состав фундаментального цикла дисциплин, формирующих у студентов представление о базовых понятиях информатики и вычислительной техники как о предмете их дальнейшей профессиональной деятельности. Изучение указанной дисциплины и овладение навыками разработки и моделирования алгоритмов выполнения арифметических и логических операций, синтеза логических схем, разработки устройств на структурном уровне, создает теоретическую базу для изложения таких дисциплин, как «Прикладная теория цифровых автоматов», «Схемотехнические основы вычислительных систем», «Архитектура вычислительных систем», и других специальных курсов.

Учебное пособие содержит теоретический материал и задания к выполнению расчетно-графических работ по дисциплине «Арифметические и логические основы вычислительной техники» и первого этапа курсового проекта по дисциплине «Прикладная теория цифровых автоматов», призванных углубить и закрепить полученные теоретические знания в области разработки цифровых операционных устройств.

Итогом выполнения расчетно-графической работы по дисциплине «Арифметические и логические основы вычислительной техники» является разработка устройства, выполняющего заданную арифметическую операцию, предполагающего разработку структуры устройства на базе типовых цифровых узлов и разработку управляющего алгоритма данного устройства.

В данном пособии приводятся примеры разработки подобных арифметических устройств.

1. ОСНОВЫ ДВОИЧНОЙ КОМПЬЮТЕРНОЙ АРИФМЕТИКИ

1.1. ПОЗИЦИОННЫЕ СИСТЕМЫ СЧИСЛЕНИЯ И ВЫПОЛНЕНИЕ АРИФМЕТИЧЕСКИХ ОПЕРАЦИЙ

В современных вычислительных системах для представления обрабатываемой числовой информации используются, как правило, позиционные системы счисления.

Системой счисления называется совокупность принципов и правил, позволяющих представить число.

Позиционной системой счисления называют такую систему, в которой значение (вес) каждой цифры, образующей число, зависит от ее позиции в числе. То есть одна и та же цифра в зависимости от ее местоположения в числе умножается на различный весовой коэффициент для определения ее значения.

Основанием системы счисления называется количество различных цифр, применяемых в соответствующей позиционной системе счисления, необходимых для изображения любого числа в этой системе.

Так, например, основанием двоичной позиционной системы является число 2, восьмеричной – число 8, десятичной – 10, шестнадцатеричной – 16 и т. д.

Пусть p ($p = 2, 3, \dots$) является основанием p -ичной позиционной системы счисления. В такой системе используется p различных цифр, представленное множеством $\{0, 1, 2, \dots, p-1\}$. Любые числа записываются в виде последовательности цифр, в которых целая часть отделена от дробной части запятой (точкой) и каждая последующая (справа налево) цифра имеет весовой коэффициент в p раз больше, чем предыдущая.

Если буквами $\alpha_n, \alpha_{n-1}, \dots, \alpha_1, \alpha_0, \alpha_{-1}, \alpha_{-2}, \dots, \alpha_{-m}$ обозначить цифры p -ичной позиционной системы счисления, то последовательность цифр обозначает число

$$\alpha_n \alpha_{n-1} \dots \alpha_1 \alpha_0 \alpha_{-1} \alpha_{-2} \dots \alpha_{-m},$$

$$A_{(p)} = \alpha_n p^n + \alpha_{n-1} p^{n-1} + \dots + \alpha_1 p^1 + \alpha_0 p^0 + \alpha_{-1} p^{-1} + \alpha_{-2} p^{-2} + \dots + \alpha_{-m} p^{-m}.$$

Преимуществом позиционных систем счисления является приспособленность их к выполнению арифметических операций, арифметические операции можно выполнять последовательно, начиная с младших или старших цифр числа, а не над всем числом в целом неразрывно.

В двоичной системе счисления для записи произвольных чисел используются цифры из множества $\{0,1\}$. Основание этой системы счисления число два ($p = 2$) (вопросы перевода из двоичной системы в десятичную здесь рассматриваться не будут, подробнее см. учебное пособие [3]).

Например, десятичное число 173,75 в двоичной системе счисления будет выглядеть так:

$$10101101,11 = 1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + \\ + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}.$$

Легко проверить теперь правильность двоичной записи данного числа, переписав правую часть последнего равенства в десятичной системе счисления:

$$1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} = \\ = 128 + 0 + 32 + 0 + 8 + 4 + 0 + 1 + 0,5 + 0,25 = 173,75.$$

Таким образом, весовые коэффициенты цифр двоичного числа являются числами, кратными целой степени (положительной или отрицательной) цифры 2. При этом для целой части двоичного числа, начиная от младшего разряда к старшему, они равны 1, 2, 4, 8, 16, 32, 64, 128 и т. д., а для дробной части двоичного числа от старшего разряда к младшему весовые коэффициенты цифр имеют значения $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{32}, \frac{1}{64}, \frac{1}{128}$ и т. д.

Таблицы сложения и умножения двоичных цифр, лежащие в основе двоичной арифметики, содержат всего по четыре строки (табл. 1.1; 1.2), что существенно облегчает техническую реализацию машинных операций.

Таблица 1.1

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

Двоичная таблица
сложения

Таблица 1.2

$$0 \cdot 0 = 0$$

$$0 \cdot 1 = 0$$

$$1 \cdot 0 = 0$$

$$1 \cdot 1 = 1$$

Двоичная таблица
умножения

На базе приведенных таблиц арифметических операций выполняются арифметические операции сложения, умножения, деления.

Пример 1.

Сложение

$$\begin{array}{r} 11001011,101 \\ + 1010101,111 \\ \hline 100100001,100 \end{array}$$

Умножение

$$\begin{array}{r} 10110101,011 \\ \times 10,101 \\ \hline 10110101,011 \\ + 10110101,011 \\ \hline 10110101,011 \\ \hline 111011100,000111 \end{array}$$

Деление

$$\begin{array}{r|l} 111011100000111 & 10110101011 \\ - 10110101011 & 10101 \\ \hline 0011100010101 \\ - 10110101011 & \\ \hline 0010110101011 \\ - 10110101011 & \\ \hline 0 \end{array}$$

Реализация указанных операций в цифровых вычислительных устройствах имеет свои особенности, которые будут рассмотрены в дальнейшем.

Вопросы и задания

1. Что такое система счисления?
2. Какие системы счисления называют позиционными?
3. К какой системе счисления (позиционной или непозиционной) относятся римские цифры?
4. Какие преимущества имеет позиционная система счисления?

1.2. ПРЕДСТАВЛЕНИЕ ЧИСЕЛ С ПЛАВАЮЩЕЙ ЗАПЯТОЙ (ТОЧКОЙ)

Формат чисел с плавающей запятой позволяет представить вещественные числа в большом диапазоне значений и применяется для них наиболее часто. Любое число в позиционной системе счисления можно представить в следующем виде:

$$N = \pm m \cdot 10^{\pm p},$$

где m – мантисса числа N , причем $0,5 \leq |m| < 1,0$ (или в двоичной системе $0,1 \leq |m| < 1,0$); p – нуль или произвольное целое число, называемое порядком числа N ; 10 (2_{10}) – основание системы счисления.

Например, при двоичной записи мантиссы m и порядка p имеем:

$$N_1 = + 0,100101 = + 0,100101 \cdot 10^0; (m_1 = + 0,100101; p_1 = 0);$$

$$N_2 = - 1001,01 = - 0,100101 \cdot 10^{+100}; (m_2 = - 0,010101; p_2 = +100 = 4);$$

$$N_3 = + 0,000101 = + 0,101000 \cdot 10^{-11}; (m_3 = + 0,101000; p_3 = -11 = -3).$$

Из приведенных примеров легко видеть, что мантисса m представляет собой цифровую часть числа, выраженную правильной дробью, а порядок

p указывает на место положения запятой в числе N_i и является целым числом. То есть порядок p указывает номер разряда правильной дроби m , после которого при $p > 0$ или перед которым при $p < 0$ необходимо поставить запятую, начиная счет с первого (старшего) разряда дроби.

Подобный вид записи числа называется нормализованным. Будем считать, что в качестве входных операндов разрабатываемых устройств будут подаваться числа в нормализованной форме, при этом и на само устройство накладывается требование выдавать данные в нормализованном виде.

Вопросы и задания

1. Какие форматы представления чисел в ЭВМ вы знаете?
2. Назовите недостатки форматов с фиксированной запятой.
3. Назовите преимущества формата с плавающей запятой.
4. Что такое мантисса числа?
5. Предоставьте в формате с плавающей запятой двоичные числа:
а) 101.1001; б) 0.1010101; в) 0.0011001101.

1.3. ПРЕДСТАВЛЕНИЕ ОТРИЦАТЕЛЬНЫХ ЧИСЕЛ

Для представления отрицательных чисел используется прямой, обратный и дополнительный коды.

В прямом коде первый разряд кодирует знак числа: 0 – число положительное, 1 – число отрицательное, остальные разряды числа остаются без изменения. Обратный код получается путем инверсии всех разрядов числа, таким образом, старший разряд будет содержать 1. Дополнительный код дополняет исходное число до переполнения разрядной сетки таким образом, что при сложении такого числа с исходным получим число 0,0000...0 и перенос из старшего разряда. Получить дополнительный код можно путем инверсии всех разрядов исходного числа с прибавлением

единицы к младшему разряду, или инвертируя все разряды исходного числа, начиная со старших, до последней единицы, последнюю единицу оставляем. Если применить процедуру вычисления дополнительного кода дважды, вернемся к исходному числу. В следующем примере приводится прямой, обратный и дополнительный коды числа.

Пример 2.

$$\begin{array}{rcl}
 X = & -0,10110100 & \\
 [X]_{\text{пр}} = & -1,10110100 & \\
 [X]_{\text{обр}} = & -1,01001011 & \\
 [X]_{\text{доп}} = & -1,01001100 &
 \end{array}
 \qquad
 \begin{array}{r}
 0,10110100 \\
 + \quad 1,01001100 \\
 \hline
 0,00000000
 \end{array}$$

Для положительного числа прямой, обратный и дополнительный коды совпадают.

Для непосредственной реализации операций наиболее удобен дополнительный код, при выполнении операций с которым не требуется каких-либо дополнительных действий, поэтому в большинстве систем используется именно дополнительный код для представления отрицательных чисел. При этом для реализации вычитания второй операнд переводится в дополнительный код, то есть

$$A - B = A + [B]_{\text{доп.}}$$

При этом не важно, какой знак имеет операнд B, если он положительный, то будет представлен отрицательным числом в дополнительном коде, если он отрицательный, то станет положительным.

Пример 3.

$$\begin{array}{rcl}
 A=0,10101011 & B=0,11001101 & \\
 -A=1,01010101 & -B=1,00110011 &
 \end{array}
 \qquad
 \begin{array}{r}
 A=0,10101011 \\
 + \quad -B=1,00110011 \\
 \hline
 A-B=1,11011110
 \end{array}$$

В примере 3 ответ получился отрицательный.

При выполнении операций в обратном коде необходимо использовать правило: при появлении переноса из старшего разряда добавляется 1 в младший разряд.

$$\begin{array}{r} A=0,10101011 \\ -A=1,01010100 \end{array}$$

B=0,11001101
-B=1,00110010

$$\begin{array}{r} + \text{ -A} = 1,01010100 \\ \text{B} = \underline{0,11001101} \\ 1_0,00100001 \\ \hline \text{B-A} = 0,00100010 \end{array}$$

1. Поясните понятия: прямой код, обратный код, дополнительный код.
2. Каковы свойства дополнительного кода ? Как его получить?
3. Как с помощью дополнительного кода в машинной арифметике реализуется вычитание?
4. С помощью дополнительного кода найти $C = A - B$ при $A = 0.11010101$, $B = 0.10101010$.

При выполнении некоторых операций, например сложения операндов одного знака, могут возникать переполнения, которые необходимо отслеживать. Признаком переполнения при сложении двух положительных чисел является появление 1 в знаковом разряде суммы, при сложении двух

отрицательных чисел – появление 0 в знаковом разряде суммы. При сложении двух чисел разных знаков переполнения невозможны.

Чтобы легче отследить переполнения, используются так называемые модифицированные коды, использующие два разряда для кодирования знака. Положительное число кодируется как 00, 1....., отрицательное – как 11, 0...., для нормализованного числа знаковый разряд должен отличаться от старшего разряда.

Пример 5.

$$\begin{array}{rcl} + & A=00,10101011 & + & -A=11,01010101 \\ & \underline{B=00,11001101} & & \underline{-B=11,00110011} \\ A+B= & 01,01111000 & -A-B= & 10,10001000 \end{array}$$

При сложении $A + B$ в примере 5 получаем положительное число с переполнением (01, – признак положительного переполнения), при сложении $(-A) + (-B)$ получаем отрицательное число с переполнением (10, – признак отрицательного переполнения). И в том, и в другом случае при использовании формата с плавающей запятой нужно нормализовать результат путем правого сдвига с сохранением знака (в сторону младших разрядов) и увеличения на единицу порядка результата.

Если в результате сложения получаем число с комбинацией знаковых разрядов 00, – число положительное без переполнения, 11, – число отрицательное без переполнения.

Модифицированные коды более удобны при выполнении операций, особенно при отслеживании возникающих переполнений, но требуют дополнительных аппаратных затрат.

Вопросы и задания

1. В каком случае возникает перенос из старшего разряда сумматора?
2. Что такое переполнение при суммировании?
3. Всегда ли перенос из старшего разряда сумматора соответствует переполнению чисел?
4. Для чего используются модифицированные коды?

2. ОСНОВЫ РАЗРАБОТКИ УСТРОЙСТВ ДВОИЧНОЙ АРИФМЕТИКИ

2.1. БАЗОВЫЕ ЦИФРОВЫЕ УЗЛЫ

Арифметические операции в вычислительных системах реализуются на арифметико-логических устройствах, которые строятся на основе типовых цифровых узлов: регистров, счетчиков сумматоров. Рассмотрим данные типовые цифровые узлы.

Элементной базой для построения цифровых узлов являются логические элементы и триггеры.

Триггер – бистабильная ячейка памяти, которая может находиться в одном из двух устойчивых состояний и способна хранить один бит информации.

Существует достаточно разветвленная классификация триггеров: по уравнению функционирования, по способу управления, по элементной базе и т. д. Но в данном случае будем считать, что триггер сохраняет один бит информации, поданный на его вход под воздействием управляющего сигнала загрузки триггера, и загруженный в триггер бит информации всегда доступен для прочтения. На рис. 1 изображен D-триггер с информационным входом D и управляющим C.

Цепочка триггеров образует регистр. Регистр – устройство, реализующее функции сверхоперативной памяти. Обычно в регистрах хранятся оперативные данные, которые используются непосредственно при выполнении операции. Кроме того, на регистрах выполняются операции сдвига и преобразования данных из последовательного в параллельный код и наоборот. При выполнении арифметических операций в регистрах хранятся операнды и промежуточные данные, также на них реализуются сдвиги. Сдвиг в сторону младших разрядов соответствует делению числа на два, сдвиг в сторону старших разрядов – умножению на два.

На рис. 2 приводится условное изображение регистра с именем RG2, в который загружается операнд А и который может выполнить сдвиг в сторону младших разрядов.

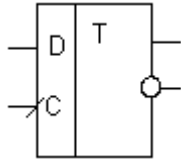


Рис. 1. Триггер

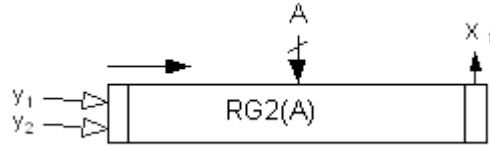


Рис. 2. Регистр

Стрелка, идущая от операнда А к регистру, перечеркнута, это означает, что шина передачи данных от А к регистру является многоразрядной. На регистр подаются два управляющих сигнала (например, y_1 – загрузка, y_2 – сдвиг вправо). С младшего разряда регистра снимается логическое условие x_1 ($x_1 = 0$, если младший разряд регистра равен 0, или $x_1 = 1$, если младший разряд регистра равен 1), которое может анализироваться алгоритмом обработки.

Сумматор – устройство, реализующее операцию сложения. При реализации арифметических операций удобнее использовать так называемые накапливающие сумматоры. При выполнении операции сложения на таком сумматоре содержимое сумматора складывается с данными, поступающими на его вход. Кроме операции сложения сумматор должен выполнять команду сброса, обнуляющую сумматор. Накапливающий сумматор содержит в своем составе регистр, соответственно может выполнять все операции, присущие регистрам, в частности сдвиги.

На рис. 3 приводится условное изображение сумматора с именем SM, на шину которого подается операнд А, в котором по окончании выполнения операции будет находиться результат выполнения операции – С и который соответственно может выполнить операцию $SM = SM + A$. При этом сумматор также может выполнить сдвиг вправо. На сумматор подаются управляющие сигналы (например, y_3 – сброс, т. е. $SM = 0$, y_4 – суммирование, y_5 – сдвиг вправо). Со старших разрядов сумматора снимаются

логические условия x_2, x_3 , позволяющие анализировать знак числа на сумматоре и его старший цифровой разряд, например, для проверки на предмет денормализации результата.

Счетчик – устройство, осуществляющее подсчет импульсов, поступающих на его вход. В приведенном случае (рис. 4) на счетчик подаются сигнал сброса y_6 ($CT = 0$) и сигнал y_7 , увеличивающий его содержимое на единицу (инкремент).

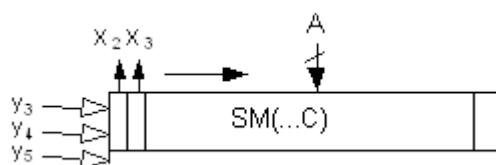


Рис. 3. Сумматор

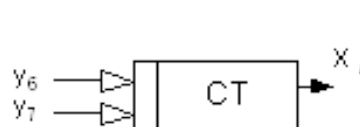


Рис. 4. Счетчик

Кроме перечисленных сигналов на счетчик могут подаваться сигнал загрузки (содержимое счетчика загружается подаваемыми на него внешними данными) и сигнал декремента, уменьшающего содержимое счетчика на единицу. Часто бывает удобнее загрузить начальное значение в счетчик, а потом подавать сигнал декремента, ожидая обнуление счетчика, которое сопровождается выдачей соответствующего выходного сигнала. При разработке арифметических устройств счетчики используются для подсчета необходимого числа циклов.

На базе рассмотренных типовых узлов – регистров, сумматоров, счетчиков – в некоторых случаях с привлечением отдельных триггеров и логических элементов реализуется построение операционной части арифметико-логических устройств, выполняющих арифметические операции.

Вопросы и задания

1. Назовите базовые цифровые узлы, используемые для построения арифметических устройств.
2. Приведите классификацию счетчиков.

3. Приведите классификацию сумматоров.

4. Постройте структурную схему устройства, содержащего сумматор, два регистра и счетчик: первый регистр имеет возможность сдвига влево, второй регистр – сдвига вправо, данные второго регистра подаются на сумматор.

2.2. РЕАЛИЗАЦИЯ ОПЕРАЦИИ УМНОЖЕНИЯ

Длинными операциями в машинной арифметике считаются операции умножения, деления, извлечения квадратного корня. Они называются «длинными», поскольку для их реализации требуется выполнение достаточно большого количества элементарных операций: сдвига, суммирования, установки разрядов.

2.2.1. Реализация операции умножения методом младшими разрядами вперед

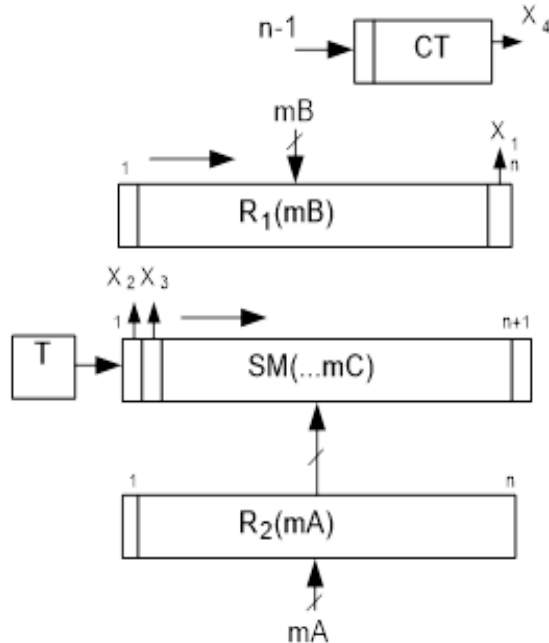


Рис. 5. Устройство умножения младшими разрядами вперед

Рассмотрим в качестве примера реализации операции умножения реализацию умножения двоичных чисел в формате с плавающей запятой в прямом коде с использованием алгоритма «младшими разрядами вперед» (он же алгоритм с подвижным сумматором). На рис. 5 приводится упрощенная структура устройства, реализующего выполнение данной операции. Устройство обработки мантисс содержит два регистра для хранения операндов и накапливающий сумматор с дополни-

тельным разрядом для округления. Дополнительно имеется триггер для хранения знака результата и счетчик для подсчета необходимого числа циклов, определяющегося количеством разрядов числа без учета знакового. Управляющие сигналы и часть устройства, выполняющего работу с порядками, не показаны.

Предполагается, что данные на вход устройства подаются в нормализованном виде. При выполнении операции $C = A \times B$ мантисса A загружается в регистр R_2 , мантисса B – в регистр R_1 , сумматор Sm обнуляется, в счетчик загружается число разрядов числа без учета знакового – $n-1$. При выполнении операции умножения порядки операндов должны складываться: $pC = pA + pB$, выполнение этой операции предполагается на некотором дополнительном сумматоре порядков (на рис. 5 не показано). Поскольку предполагается, что данные хранятся в прямом коде, то знаки операндов следует обработать отдельно и сохранить в триггере, то есть выполняется операция определения знака результата и сохранения его в триггере как

$$T = Z_n(A) \oplus Z_n(B),$$

где $\oplus$ – логическая операция «сумма по модулю два»;

$Z_n(A)$ – знак A (первый разряд мантиссы mA);

$Z_n(B)$ – знак B (первый разряд мантиссы mB).

После чего знаки операндов необходимо принудительно обнулить.

Собственно умножение реализуется следующим образом.

1. Анализируется младший разряд регистра R_1 (логическое условие x_1), и если разряд равен 1, выполняется операция на сумматоре $Sm = Sm + R_2$ для обеспечения данной операции в структуре предусмотрена многоразрядная связь от R_2 к сумматору.

2. Выполняются сдвиги сумматора и R_1 в сторону младших разрядов: $Sm \rightarrow, R_1 \rightarrow$.

3. Содержимое счетчика уменьшается на единицу СТ--, если $СТ \neq 0$, продолжить с 1.

4. Выполняется округление $S_m = S_m + 2^{-(n+1)}$.

Округление, реализуемое путем прибавления единицы к младшему дополнительному разряду сумматора, желательно выполнять для того, чтобы избежать накопления систематической ошибки отрицательного знака.

По окончании указанных действий на сумматоре должен находиться результат. Перед выдачей результата необходимо выполнить проверку на нормализацию результата (посмотреть второй разряд сумматора – логическое условие x_3 , он должен быть равен 1). Поскольку изначально операнды находятся в нормализованном виде, денормализация может быть не более чем на один знак. Если $x_3 = 0$, то необходимо перед выдачей выполнить нормализацию, то есть сдвинуть сумматор в сторону старших разрядов, что эквивалентно умножению на 2, и порядок результата p_C уменьшить на единицу.

Далее необходимо загрузить из триггера знак результата в первый разряд сумматора $S_m[1] = T$, после чего в сумматоре S_m находится мантисса результата операции умножения, предполагается, что порядок находится в специальном регистре или сумматоре p_C . Приведем пример выполнения данного алгоритма (пример 6).

В первом такте каждого цикла производится (или нет) операция сложения, а во втором такте, в любом случае, выполняется сдвиг в сторону младших разрядов.

Пример 6.

$mA = 0,10101101$ $pA = 00001$
 $mB = 0,11001001$ $pB = 00011$
 $S_m = 0,00000000 | 0$
 $R_2 = 0,10101101 | 0$
 $T = 0 \oplus 0 = 0$
1) $S_m = 0,10101101 | 0$ $R_1 = 0,11001001$
 $\overrightarrow{S_m} = 0,01010110 | 1$

$$\begin{aligned}
2) \overrightarrow{Sm} &= 0,00101011|0 & \overrightarrow{R_1} &= 0,01100100 \\
3) \overrightarrow{Sm} &= 0,00010101|1 & \overrightarrow{R_1} &= 0,00110010 \\
4) R_2 &= \underline{0,10101101|0} & \overrightarrow{R_1} &= 0,00011001 \\
Sm &= 0,11000010|1 \\
\overrightarrow{Sm} &= 0,01100001|0 \\
5) \overrightarrow{Sm} &= 0,00110000|1 & \overrightarrow{R_1} &= 0,00001100 \\
6) \overrightarrow{Sm} &= 0,00011000|0 & \overrightarrow{R_1} &= 0,00000110 \\
7) R_2 &= \underline{0,10101101|0} & \overrightarrow{R_1} &= 0,00000011 \\
Sm &= 0,11000101|0 \\
\overrightarrow{Sm} &= 0,01100010|1 \\
8) R_2 &= \underline{0,10101101|0} & \overrightarrow{R_1} &= 0,00000001 \\
Sm &= \underline{1,00001111|1} \\
\overrightarrow{Sm} &= 0,10000111|1 \\
& \quad \quad \quad \underline{1 \text{ (округление)}} \\
Sm &= 0,10001000|0 \\
Sm[1] &= T \text{ (установка знака)} \\
pA &= 00001 \\
pB &= 00011 \\
pC &= 00100 \text{ (порядок)}
\end{aligned}$$

Ответ: $mC=0,10001000$ $pC=00100$.

Перед выводом результата при необходимости выполняется нормализация (в нашем случае число и так получилось нормализованным) и устанавливается знак (в нашем случае $T = 0$, число положительное).

2.2.2. Реализация операции умножения методом старшими разрядами вперед

Существует и другой алгоритм умножения: старшими разрядами вперед, структура данного устройства приводится на рис. 6. При выполнении алгоритма данные множителя R_1 будут сдвигаться в сторону старших разрядов, данные множимого R_2 будут сдвигаться в сторону младших разрядов, поэтому в сумматоре и связанном с ним регистре R_2 необходимо предусмотреть дополнительные разряды. Сумматор остается неподвиж-

ным. Общее число разрядов соответствует $n + k$, где число дополнительных разрядов k исходя из обеспечения приемлемой погрешности ($\leq 2^n$) выбирается в соответствии с выражением

$$k \geq \log_2(n-k).$$

Работа с порядками и знаками производится аналогично предыдущему методу, собственно умножение реализуется следующим образом.

1. Выполняются сдвиги R_1 в сторону старших разрядов, R_2 в сторону младших разрядов: $R_1 \leftarrow$, $R_2 \rightarrow$.

2. Анализируется старший разряд регистра R_1 (логическое условие x_1), и если разряд равен 1, выполняется операция на сумматоре $Sm = Sm + R_2$, для обеспечения данной операции в структуре предусмотрена многоразрядная связь от R_2 к сумматору.

3. Содержимое счетчика уменьшается на единицу $CT--$, если $CT \neq 0$, продолжить с 1.

4. Выполняется округление $Sm = Sm + 2^{-(n+1)}$.

Пример 7.

$mA = 0,10101101$ $pA = 00001$

$mB = 0,11001001$ $pB = 00011$

$Sm = 0,00000000 | 000$

$R_2 = 0,10101101 | 000$

$R_1 = 0,11001001$

1) $\overrightarrow{R_2} = 0,01010110 | 100$

$\overleftarrow{R_1} = 1,10010010$

$Sm = 0,01010110 | 100$

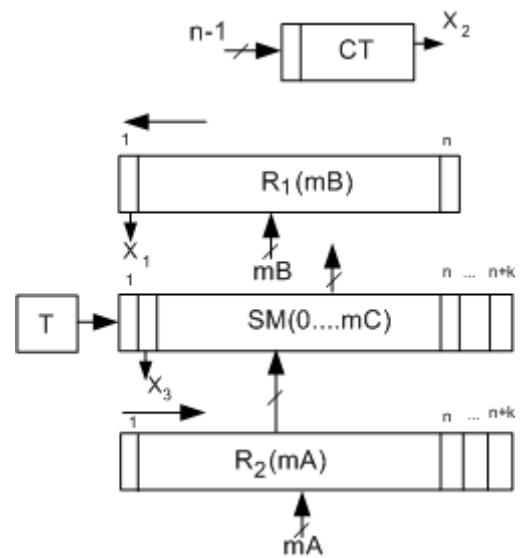


Рис. 6. Устройство умножения старшими разрядами вперед

$$2) \overrightarrow{R_2} = 0, 00101011 \mid 010$$

$S_m = 0, 10000001 \mid 110$

$$3) \overrightarrow{R_2} = 0, 00010101 | 101$$

4) $\vec{R}_2 = 0, 00001010 | 110$

$$5) \overrightarrow{R_2} = 0, 00000101 \mid 011$$

Sm= 0,100000001|110

Sm= 0, 10000111 | 001

$$6) \overrightarrow{R_2} = 0, 00000010 | 101$$

7) $\overrightarrow{R_2} = 0, 000000001 \mid 010$

8) $\overrightarrow{R_2} = 0, 00000000 | 101$

$$S_m = \underline{0, 10000111} \mid 001$$

$$S_m = 0, 10000111 | 110$$

1

Sm = 0,10000111|111

$$\overleftarrow{R}_1 = 1, 00100100$$

$$\overleftarrow{R}_1 = 0,01001000$$

$$\overleftarrow{R}_1 = 0,10010000$$

$$\overleftarrow{R_1} = \underline{1}, 00100000$$

$$\overleftarrow{R}_1 = 0,01000000$$

$$\overleftarrow{R_1} = 0,10000000$$

$$\overleftarrow{R}_1 = 1, 000000000$$

pA= 00001

pB= 00011

pC= 00100

ОТВЕТ: $mC = 0,10000111$ $pC=00100$.

В примере 7 работа со знаками не отображается.

Оценим погрешность:

2^{-n} – погрешность стандартного алгоритма (младшими разрядами вперед;

$2^{-(n+k)}$ – погрешность одного цикла алгоритма старшими разрядами вперед;

$(n-k) 2^{-(n+k)}$ – погрешность всего алгоритма старшими разрядами вперед, здесь учитываются только $(n-k)$ циклов, поскольку в первых k циклах за счет дополнительных разрядов вычисления идут без погрешно-

сти. Выбирая k из условия, чтобы погрешность алгоритма старшими разрядами вперед не превышала погрешности алгоритма младшими разрядами вперед, имеем:

$$\log_2(n - k) - n - k \leq -n$$

$$k \geq \log_2(n - k).$$

Таким образом, для $n = 8$ минимальное $k = 3$, для $n = 6$ минимальное $k = 2$.

Особенностью метода умножения старшими разрядами вперед является то, что можно использовать сумматор, не поддерживающий сдвиг.

Как будет показано в дальнейшем, операционное устройство, реализующее данный алгоритм без модификации, можно использовать для реализации операции деления методом с неподвижным сумматором.

Если при использовании алгоритма младшими разрядами вперед результат получается только по окончании всех n циклов (где n – число разрядов мантииссы после запятой), то при использовании алгоритма старшими разрядами вперед после каждого цикла получаем приблизительный результат, уточняемый с каждым последующим циклом.

2.2.3. Реализация операции умножения в дополнительных кодах

Рассмотрим особенности реализации алгоритмов умножения в дополнительных кодах:

- 1) при $A > 0, B > 0$ – можно использовать стандартные алгоритмы;
- 2) при $A < 0, B > 0$ – на сумматоре будет копиться отрицательная сумма, и также можно использовать стандартные алгоритмы;

3) при $B < 0$ будет происходить умножение не на само B , а на величину $(1 - B)$, отсюда имеем: $A(1 - B) = A - AB$, то есть чтобы получить правильный ответ, необходимо выполнить коррекцию сумматора на величину $(-A)$.

При использовании алгоритма младшими разрядами вперед коррекция при отрицательном множителе выполняется после основного цикла ($S_m - = R_2$), при использовании метода старшими разрядами вперед это удобно сделать до основного цикла, пока R_2 еще не сдвинут.

В примере 8 приводится умножение чисел в дополнительных кодах методом младшими разрядами вперед: $A > 0, B < 0$, операционное устройство аналогично рис. 5, только триггер знака не используется, поскольку знак результата получается автоматически.

Пример 8.

$$\begin{aligned}
 mA &= 0,101010 & A > 0 \\
 mB &= 1,001101 & B < 0 \\
 (|B| &= 0,110011) \\
 S_m &= 0,000000|0 \\
 R_2 &= \underline{0,101010|0} \\
 1) S_m &= 0,101010|0 & R_1 &= 1,00110\underline{1} \\
 \overrightarrow{S_m} &= 0,010101|0 & \overrightarrow{R_1} &= 1,10011\underline{0} \\
 2) \overrightarrow{S_m} &= 0,001010|1 & \overrightarrow{R_1} &= 1,11001\underline{1} \\
 R_2 &= \underline{0,101010|0} \\
 3) S_m &= 0,110100|1 \\
 \overrightarrow{S_m} &= 0,011010|0 & \overrightarrow{R_1} &= 1,11100\underline{1} \\
 4) R_2 &= \underline{0,101010|0} \\
 S_m &= 1,000100|0 \\
 \overrightarrow{S_m} &= 0,100010|0 & \overrightarrow{R_1} &= 1,11110\underline{0} \\
 5) \overrightarrow{S_m} &= 0,010001|0 & \overrightarrow{R_1} &= 1,11111\underline{0} \\
 6) \overrightarrow{S_m} &= 0,001000|1 & \overrightarrow{R_1} &= 1,11111\underline{1}
 \end{aligned}$$

$$-R_2 = \underline{1,010110|0} \quad (\text{коррекция})$$

$$Sm = 1,011110|1$$

$$\begin{array}{r} \\ \\ \hline Sm = 1,011111|0 \end{array}$$

Ответ: $mC = 1,011111$

В рассмотренном примере коррекция выполняется после основного цикла операции. Сдвиги модифицированные, с сохранением знака.

При работе в дополнительном коде триггер знака не используется, знак результата получается автоматически за счет коррекции.

2.2.4. Реализация операции умножения в обратных кодах

При выполнении операции умножения в обратных кодах в случае отрицательного множителя ($B < 0$) происходит реальное умножение не на сам множитель B , а на величину $(1 - B - 2^{-n})$, которая на единицу младшего разряда (2^{-n}) отличается от дополнительного кода $(1 - B)$, таким образом, умножая A на B , получается:

$$A \times (1 - B - 2^{-n}) = A - AB - 2^{-n}A.$$

Таким образом, при умножении в обратных кодах необходимо сделать две поправки при отрицательном B :

- 1) $-A$;
- 2) $2^{-n}A$ (сдвинутое на n разрядов A).

При использовании алгоритма младшими разрядами вперёд (пример 9) вначале в сумматор загружается R_2 , содержащий мантиссу A , которое в ходе алгоритма сдвигается на n разрядов (таким образом, реализуется коррекция 2), а после цикла выполняется коррекция 1 ($Sm - = R_2$), так же как в алгоритме для дополнительных кодов.

При использовании алгоритма старшими разрядами вперёд вначале выполняется коррекция 1 ($S_m - = R_2$), а после цикла, когда в R_2 как раз содержится сдвинутое на n разрядов A , выполняется коррекция 2 ($S_m + = R_2$). Операционное устройство аналогично рис. 5, только триггер знака, так же как в дополнительных кодах, не используется.

Пример 9.

$mA =$	$0, 10101101$	$pA =$	00001
$mB =$	$1, 01001101$	$pB =$	00010
$S_m =$	$0, 10101101 0$		
$R_2 =$	<u>$0, 10101101 0$</u>	$R_1 =$	$1, 01001101$
1) $S_m =$	$1, 01011010 0$		
$\overrightarrow{S_m} =$	$0, 10101101 0$	$\overrightarrow{R_1} =$	$1, 10100110$
2) $\overrightarrow{S_m} =$	$0, 01010110 1$	$\overrightarrow{R_1} =$	$1, 11010011$
3) $R_2 =$	<u>$0, 10101101 0$</u>		
$S_m =$	$1, 00000011 1$		
$\overrightarrow{S_m} =$	$0, 10000001 1$	$\overrightarrow{R_1} =$	$1, 11101001$
4) $R_2 =$	<u>$0, 10101101 0$</u>		
$S_m =$	$1, 00101110 1$		
$\overrightarrow{S_m} =$	$0, 10010111 0$	$\overrightarrow{R_1} =$	$1, 11110100$
5) $\overrightarrow{S_m} =$	$0, 01001011 1$	$\overrightarrow{R_1} =$	$1, 11111010$
6) $\overrightarrow{S_m} =$	$0, 00100101 1$	$\overrightarrow{R_1} =$	$1, 11111101$
7) $R_2 =$	<u>$0, 10101101 0$</u>		
$S_m =$	$0, 11010010 1$		
$\overrightarrow{S_m} =$	$0, 01101001 0$	$\overrightarrow{R_1} =$	$1, 11111110$
8) $\overrightarrow{S_m} =$	$0, 00110100 1$	$\overrightarrow{R_1} =$	$1, 11111111$
$-R_2 =$	<u>$1, 01010010 1$</u>	(коррекция)	
$S_m =$	$1, 10000111 1$		
	<u>1</u>	(округление)	
$S_m =$	<u>$1, 10001000 0$</u>	(норм.)	$pC = 00011$
$\overleftarrow{S_m} =$	$1, 00010000$	$pC =$	00010

ОТВЕТ: $mC = 1, 00010000$ $pC = 00010.$

При работе в обратном коде, как и в дополнительном, триггер знака не используется, знак результата получается автоматически за счет коррекции, сдвиги выполняются с сохранением знака.

При вычислениях необходимо учитывать особые правила сложения для обратного кода: при возникновении переноса из старшего разряда он прибавляется к младшему разряду.

2.2.5. Ускоренные методы умножения

Умножение является одной из базовых операций, большинство математических функций вычисляются посредством рядов Тейлора, основанных на использовании умножения, потому эффективное и быстрое выполнение этой операции крайне важно.

Существует два алгоритмических подхода ускорения операции умножения: 1) анализ цепочек единиц и 2) одновременный анализ нескольких разрядов множителя.

Идея метода, основанного на анализе цепочек единиц, достаточно проста, рассмотрим некоторый множитель B_0 , содержащий группу единиц, идущих подряд (сверху подписаны веса разрядов, разряды от $-k$ до $-m$):

$$\begin{array}{c} 0 \quad -1 \dots -k \quad \dots -m-(m+1) \\ B_0 = 0, 1xx1111110xx. \end{array}$$

Если выполнять стандартный алгоритм, то потребуется минимум шесть сложений в соответствующих циклах. Но B_0 можно представить в виде двух слагаемых B_1 и $-B_2$:

$$\begin{array}{l} B_0 = B_1 - B_2 \\ 0 \quad -1 \dots -k \quad \dots -m-(m+1) \\ B_0 = 0, 1xx1111110xx \\ 0 \quad -1 \dots -k \quad \dots -m-(m+1) \end{array}$$

$$B_1 = 0,1 \times 100000000xx$$

$$\quad \quad \quad 0 \quad -1 \dots -k \quad \dots -m-(m+1)$$

$$-B_2 = 0,1 \times 000000001xx.$$

То есть цепочку единиц можно представить разностью двух единиц: $2^{-(k-1)} - 2^{-(m+1)}$, и все сложения цепочки заменить одним сложением (в разряде $k-1$) и одним вычитанием (в разряде $m+1$).

Метод, основанный на анализе цепочек единиц, использует триггер (для пометки вхождения в режим анализа цепочек), однако эффект его использования невелик из-за статистически небольшой вероятности появления длинных цепочек единиц в операндах операции.

Метод, основанный на одновременном анализе нескольких разрядов операнда, используется намного чаще, известны методы, использующие 2, 3 и даже 4 разряда множителя. Суть этих методов состоит в обработке группы разрядов множителя в одном цикле.

Рассмотрим метод умножения с анализом двух разрядов множителя. В табл. 1 приводятся выполняемые в операционном устройстве операции в зависимости от комбинации младших разрядов множителя.

Таблица 1

Ускоренное умножение с анализом двух разрядов

p	R1[n-1]	n]	операция	p вых
0	0	0	$S_m = S_m + 0$	$p = 0$
0	0	1	$S_m = S_m + R_2$	$p = 0$
0	1	0	$S_m = S_m + 2R_2$	$p = 0$
0	1	1	$S_m = S_m - R_2$	$p = 0$
1	0	0	$S_m = S_m + R_2$	$p = 0$
1	0	1	$S_m = S_m + 2R_2$	$p = 0$
1	1	0	$S_m = S_m - R_2$	$p = 1$
1	1	1	$S_m = S_m + 0$	$p = 1$

Согласно табл. 1 при обработке комбинации 01 к сумматору прибавляется множимое R_2 , при 10 прибавляется $2R_2$ (сдвинутое на 1 разряд влево множимое), при 11 утроенное множимое не прибавляется, вместо этого

производится вычитание множимого и устанавливается перенос в следующую группу ($p = 1$), с учетом которого эта группа увеличивается на 1 (получается $4R_2 - R_2 = 3R_2$). После операции сдвига сумматора (S_m) и регистра множителя (R_1) производятся на 2 разряда сразу. Поскольку в вычислениях используются удвоенные значения нормализованных величин ($2R_2$), во избежание потери значащих цифр необходимо использовать модифицированные коды, содержащие два знаковых разряда и позволяющие работать в условиях переполнения.

Структура устройства, реализующего операцию умножения ускоренным методом с анализом двух разрядов, приводится на рис. 7. Здесь анализируются два младших разряда множителя. В структуре используется

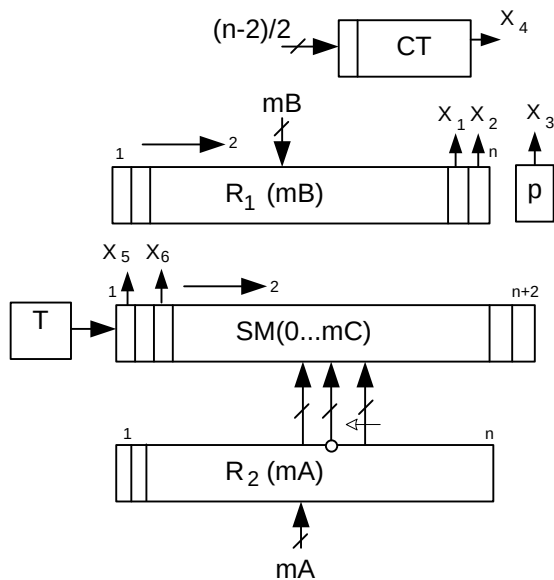


Рис. 7. Устройство ускоренного умножения с анализом двух разрядов старшими разрядами вперед

дополнительный триггер p , сохраняющий перенос в следующую секцию.

От R_2 к сумматору S_m идут две многоразрядные связи (слева направо: первая прямая обеспечивает операцию $+R_2$, вторая обратная $-R_2$, третья со сдвигом влево $+2R_2$).

По выполнению цикла, который в данном случае содержит $(n-2)/2$ итераций, где n – число разрядов, включая знаковые, и должно быть четным, при оставшемся неотработанном переносе (если $p = 1$), нужно выполнить дополнительный цикл $S_m = S_m + R_2$, но без сдвига.

Триггер T используется для хранения знака результата при работе в прямых кодах, при работе в дополнительных и обратных кодах не используется.

Пример 10 иллюстрирует выполнение умножения ускоренным методом с анализом двух разрядов в прямых кодах.

Пример 10.

$mA = 00, 11011001$

$pA = 000010$

$mB = 00, 11001001$

$pB = \underline{000011}$

$Sm = 00, 00000000 | 00$

$pC = 000100$

$R_2 = 00, 11011001 | 00$

$2R_2 = 01, 10110010 | 00$ $R_1 = 00, 1100100\underline{1}$

1) $Sm = 00, 11011001 | 00$

$\overrightarrow{Sm} = 00, 00110110 | 01$ $\overrightarrow{R_1} = 00, 001100\underline{10}$

2) $2R_2 = \underline{01, 10110010 | 00}$

$Sm = 01, 11101000 | 01$

$\overrightarrow{Sm} = 00, 01111010 | 00$ $\overrightarrow{R_1} = 00, 000011\underline{00}$

3) $\overrightarrow{Sm} = 00, 00011110 | 10$ $\overrightarrow{R_1} = 00, 000000\underline{11}$ ($p=1$)

4) $-R_2 = \underline{11, 00100111 | 00}$

$Sm = 11, 01000101 | 10$

$\overrightarrow{Sm} = 11, 11010001 | 01$ $\overrightarrow{R_1} = 00, 00000000$

$+R_2 = \underline{00, 11011001 | 00}$ (доп. цикл)

$Sm = 00, 10101010 | 01$

ОТВЕТ: $mC = 00, 10101010$ $pC = 000100$.

Данный метод с успехом можно также использовать при вычислениях в дополнительных кодах, только необходимо соблюдать правила коррекции. В примере 11 приводится укоренное умножение с анализом двух рядов в дополнительных кодах, в конце выполняется коррекция.

Пример 11.

$mA = 00, 11011001$

$pA = 000010$

$mB = 11, 00110111$

$pB = 000011$

$Sm = 00, 00000000 | 00$ $pC = 000100$

$R_2 = 00, 11011001 | 00$

$$\begin{aligned}
2R_2 &= 01, 10110010 | 00 \\
-R_2 &= 11, 00100111 | 00 \quad R_1 = 11, 001101\underline{11} (p=1) \\
1) S_m &= 11, 00100111 | 00 \\
\overrightarrow{S_m} &= 11, 11001001 | 11 \quad \overrightarrow{R_1} = 11, 110011\underline{01} \\
2) 2R_2 &= \underline{01, 10110010 | 00} \quad 10 \\
S_m &= 01, 01111011 | 11 \\
\overrightarrow{S_m} &= 00, 01011110 | 11 \quad \overrightarrow{R_1} = 11, 111100\underline{11} (p=1) \\
3) -R_2 &= \underline{11, 00100111 | 00} \\
S_m &= 11, 10000101 | 00 \\
\overrightarrow{S_m} &= 11, 11100001 | 01 \quad \overrightarrow{R_1} = 11, 111111\underline{00} \\
4) R_2 &= \underline{00, 11011001 | 00} \quad 01 \\
S_m &= 00, 10111010 | 01 \\
\overrightarrow{S_m} &= 00, 00101110 | 10 \quad \overrightarrow{R_1} = 11, 11111111 \\
-R_2 &= \underline{11, 00100111 | 00} \quad (\text{кор.}) \\
S_m &= 11, 01010101 | 10 \\
&\quad \underline{\hspace{10em} 1} \text{ (округление)} \\
S_m &= 11, 01010101 | 11
\end{aligned}$$

ОТВЕТ: $mC = 11, 01010101 | 11$ $pC = 000100$.

Во втором цикле обрабатывается комбинация 01, но поскольку в предыдущем цикле был установлен перенос ($p = 1$), выполняется операция $+2R_2$, что эквивалентно обрабатываемой комбинации 10.

Таким образом, ускоренный метод с анализом двух разрядов множителя за счет некоторого усложнения алгоритма выполнения операции увеличивает скорость выполнения операции примерно в 2 раза в устройстве, реализованном на тех же цифровых узлах (регистрах и сумматоре).

Развивая данную идею, можно предложить метод, основанный на анализе трех разрядов множителя.

В табл. 2 приводятся выполняемые в операционном устройстве действия.

Таблица 2

**Ускоренное умножение
с анализом трёх разрядов**

R1[n-2]	n-1	n]	Операция	p
0	0	0	$Sm=Sm+0$	0
0	0	1	$Sm=Sm+R_2$	0
0	1	0	$Sm=Sm+2R_2$	0
0	1	1	$Sm=Sm+3R_2$	0
1	0	0	$Sm=Sm+4R_2$	0
1	0	1	$Sm=Sm-3R_2$	1
1	1	0	$Sm=Sm-2R_2$	1
1	1	1	$Sm=Sm-R_2$	1

Для обработки трех последних комбинаций используются вычитание и переносы в следующую триаду ($p = 1$), при наличии которых следующая триада увеличивается на 1. После операции сдвиги сумматора (Sm) и регистра множителя (R_1) производятся на 3 разряда сразу. Используются расширенные модифицированные коды, содержащие 3 знаковых разряда и способные работать в условиях большого переполнения. Для удобства реализации алгоритма используется дополнительный регистр R_3 , в который рассчитывают и загружают утроенное значение множимого, удвоенные и учетверенные значения множимого можно получить передачей со сдвигом.

Структура устройства, выполняющего умножение ускоренным методом с анализом трёх разрядов, приводится на рис. 8. Здесь анализируются три младших разряда множителя. В структуре также используется дополнительный триггер p , сохраняющий перенос в следующую секцию. От R_2 к сумматору Sm идут семь многоразрядных связей (слева направо: первая прямая обеспечивает операцию $+R_2$, вторая обратная $-R_2$, третья со сдвигом влево $+2R_2$, четвертая обратная со сдвигом влево $-2R_2$, пятая с двойным сдвигом влево $+4R_2$, шестая $+R_3$ (в котором находится $+3R_2$), седьмая обратная $-R_3$ ($-3R_2$). Шестая связь двунаправленная, поскольку содержащее R_3 перед основным циклом операции формируется на сумматоре оче-

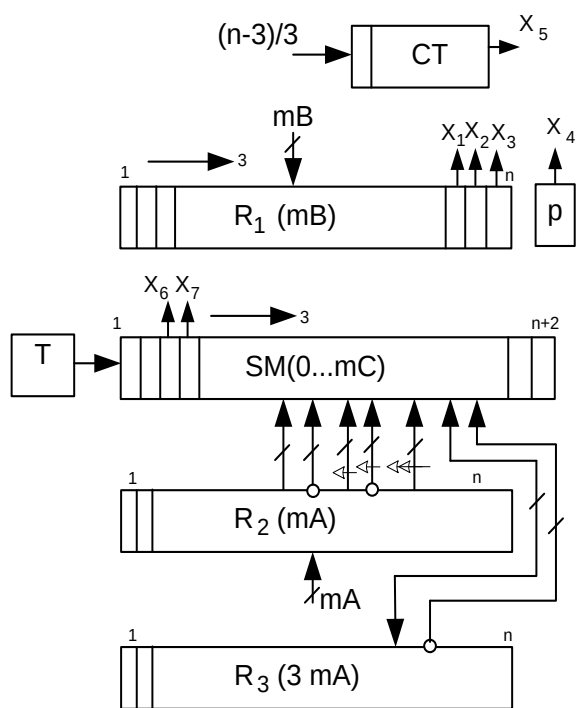


Рис. 8. Устройство ускоренного умножения с анализом трёх разрядов старшими разрядами вперед

видным набором команд: $S_m = 0$, $S_{m+} = 2R_2$, $S_{m+} = R_2$ (таким образом, на сумматоре получаем $+3R_2$), далее результат загружается в R_3 операцией $R_3 = S_m$ и перед началом первого цикла обнуляется сумматор $S_m = 0$.

Также по выполнению цикла, который в данном случае содержит $(n-3)/3$ итераций, где n – число разрядов, включая знаковые, и должно быть кратным трем, при оставшемся неотработанным переносе (если $p = 1$), нужно выполнить дополнительный цикл $S_m = S_m + R_2$, но без сдвига.

В примере 12 приводится умножение ускоренным методом с анализом трех разрядов в прямых кодах.

Пример 12.

$mA = 000, 110110010$

$mB = 000, 110010010$

$R_2 = 000, 110110010$

$2R_2 = 001, 101100100$

$3R_2 = 010, 100010110 \quad (R_3)$

$4R_2 = 011, 011001000$

1) $S_m = 000, 000000000 | 000$

$2R_2 = 001, 101100100 | 000$

$S_m = 001, 101100100 | 000$

$\overrightarrow{S_m} = 000, 001101100 | 100$

$R_1 = 000, 110010010$

ОТВЕТ: $mC = 000,101010100$.

Вопросы и задания

- a) младшими разрядами вперед;
- b) старшими разрядами вперед;
- c) ускоренным методом с анализом двух разрядов (самостоятельно привести операнды к модифицированному коду);
- d) ускоренным методом с анализом трех разрядов (самостоятельно привести операнды к расширенному модифицированному коду).

2. Для заданных A и B дать пошаговую иллюстрацию выполнения умножения $C = A \cdot B$ указанным методом в дополнительных кодах

($mA = 0.10101011$ $pA = 0010$, $mB = 1.00110101$ $pB = 1111$):

- a) младшими разрядами вперед;
- b) старшими разрядами вперед;
- c) ускоренным методом с анализом двух разрядов (самостоятельно привести операнды к модифицированному коду);
- d) ускоренным методом с анализом трех разрядов (самостоятельно привести операнды к расширенному модифицированному коду).

3. Для заданных A и B дать пошаговую иллюстрацию выполнения умножения $C = A \cdot B$ указанным методом в обратных кодах ($mA = 0.10101010$ $pA = 0010$, $mB = 1.00110111$ $pB = 1110$):

- a) младшими разрядами вперед;
- b) старшими разрядами вперед;
- c) ускоренным методом с анализом двух разрядов (самостоятельно привести операнды к модифицированному коду).

2.3. РЕАЛИЗАЦИЯ ОПЕРАЦИИ ДЕЛЕНИЯ

Операция деления, как и умножение, является длинной операцией и требует выполнения цикла команд.

Вспомним, как выполняется деление столбиком в привычной нам десятичной системе счисления:

1) имеем делимое (A) и делитель (B), вычисляем результат C : $C = 0, c_1c_2\dots c_n$ – его разряды;

2) выделяем часть цифр делимого, чтобы это частичное делимое превышало делитель (если, выбрав даже все цифры делимого, этого получить не удастся, дописываем к ним 0, а в результат записываем $c_1 = 0$, имея в виду $C = 0, \dots$);

3) подбираем такое наибольшее целое c_i (возможно, равное 0), чтобы при умножении на делитель результат не превышал текущее частичное

делимое (частичный остаток), это наибольшее целое c_i является очередной цифрой частного (начиная со старшего разряда);

4) из частичного делимого вычитается делитель, умноженный на c_i , и определяется частичный остаток, который дополняется очередной цифрой делимого (если цифры делимого кончились, то 0-м, что эквивалентно сдвигу влево);

5) процесс продолжается, начиная с пункта 3, пока не получим конечный ответ либо заданное количество цифр после запятой.

Если перенести эти действия на двоичные нормализованные вещественные числа в формате с плавающей запятой, то получим алгоритм машинной операции деления, при этом возможны только две альтернативы: либо сдвинутый на один разряд влево частичный остаток больше делителя, тогда очередная цифра частного $c_i = 1$, либо меньше, тогда $c_i = 0$. Как вариант, сравнение частичного остатка с делителем может быть реализовано на сумматоре путем вычитания из частичного остатка делителя, если результат получается положительный, то очередная цифра частного $c_i = 1$, а на сумматоре имеем частичный остаток для следующего цикла, если результат получается отрицательный, то очередная цифра частного $c_i = 0$ и выполняется восстановление сумматора до предыдущего значения путем прибавления к нему делителя.

С учетом требования к нормализации данных, в частности результата S (нормализованный результат должен быть меньше 1.0), перед выполнением операции необходимо выполнить проверку на то, что $|mA| < |mB|$, если это условие не удовлетворяется, необходимо произвести денормализацию делимого путем сдвига в сторону младших разрядов, соответственно порядок делимого увеличивается на 1 ($pA++$), если порядок результата уже посчитан как разность: $pS = pA - pB$, нужно увеличить порядок результата ($pS++$).

Рассмотрим варианты реализации операции деления на устройствах со сдвигом сумматора и без.

2.3.1. Реализация операции деления на устройстве с подвижным сумматором в прямых кодах

Алгоритм машинной операции деления методом со сдвигом сумматора и восстановления остатка для чисел в прямых кодах в формате с плавающей запятой, учитывающий работу с порядками и со знаками, будет выглядеть следующим образом.

1. В сумматор SM загружаем мантиссу делимого m_A , в регистр R_2 – мантиссу делителя m_B , в регистр результата R_1 – 0,0...0 (на выходе в нем сформируется m_C) в счетчик $CT = n-1$ (число разрядов после запятой).
2. Находим порядок результата $p_C = p_A - p_B$ (предполагается, что для этого используются регистры порядка).
3. В триггере сохраняем знак T результата $= SM[1] \oplus R_2[1]$.
4. Обнуляем знаки операндов $SM[1] = 0, R_2[1] = 0$.
5. Выполняем проверку на нормализацию результата как $SM < R_2$, если условие не выполняется, производится сдвиг сумматора вправо и p_C++ .
6. Выполняем сдвиг SM в сторону старших разрядов, $SM \leftarrow$.
7. Выполняем вычитание на сумматоре $SM = SM - R_2$.
8. Инверсия знакового разряда сумматора задвигается в регистр формирования результата при выполнении левого сдвига $R_1 \leftarrow$, $c_i = \overline{SM[1]}$.
9. Если сумматор отрицательный $SM[1] = 1$, выполняем восстановление остатка $SM = SM + R_2$.
10. Содержимое счетчика уменьшается на единицу $CT--$, если $CT > 0$, продолжить с 6.
11. Присваиваем знак результату $R_1[1] = T$.
12. Вывод R_1, p_C .

На рис. 9 приводится устройство, реализующее выполнение операции деления. Сумматор и регистр R_1 имеют возможность сдвига влево. Между регистром R_2 и сумматором SM указано две многоразрядные связи: прямая (обеспечивает реализацию $SM = SM + R_2$) и инверсная (обеспечивает реализацию $SM = SM - R_2$).

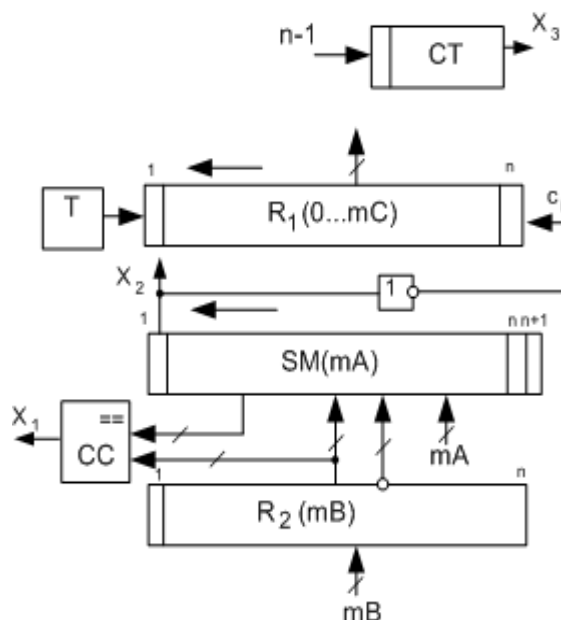


Рис. 9. Устройство деления
в прямых кодах со сдвигом
сумматора

Полученный в результате операции знак сумматора инвертируется и заносится в качестве очередного разряда частного при сдвиге регистра R_1 в сторону старших разрядов. Структура также содержит счетчик СТ, триггер знака результата Т и схему сравнения СС, обеспечивающую проверку на возможность деления $|mA| < |mB|$. Дополнительные регистры для работы с порядками на структуре не показаны.

В примере 13 приводится деление методом с восстановлением остатка на устройстве с подвижным сумматором.

Пример 13.

$mA = 0,10110101$	$pA = 0001$
$mB = 0,11001001$	$pB = 0010$
$Sm = 0,10110101$	$R_1 = 0,00000000 0$
$R_2 = 0,11001001$	
$-R_2 = 1,00110111$	
1) $\overleftarrow{Sm} = 1,01101010$	
$-R_2 = \underline{1,00110111}$	
$Sm = \underline{0,10100001}$	$\overleftarrow{R_1} = 0,00000000 1$
2) $\overleftarrow{Sm} = 1,01000010$	
$-R_2 = \underline{1,00110111}$	
$Sm = \underline{0,01111001}$	$\overleftarrow{R_1} = 0,00000001 1$
3) $\overleftarrow{Sm} = 0,11110010$	
$-R_2 = \underline{1,00110111}$	
$Sm = \underline{0,00101001}$	$\overleftarrow{R_1} = 0,00000011 1$

$$4) \overleftarrow{S_m} = 0, 01010010$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{1, 10001001} \quad \overleftarrow{R_1} = 0, 00000111 | 0$$

$$R_2 = \underline{0, 11001001} \text{ (восстановление)}$$

$$S_m = 0, 01010010$$

$$5) \overleftarrow{S_m} = 0, 10100100$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{1, 11011011} \quad \overleftarrow{R_1} = 0, 00001110 | 0$$

$$R_2 = \underline{0, 11001001} \text{ (восстановление)}$$

$$S_m = 0, 10100100$$

$$6) \overleftarrow{S_m} = 1, 01001000$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{0, 01111111} \quad \overleftarrow{R_1} = 0, 00011100 | 1$$

$$7) \overleftarrow{S_m} = 0, 11111110$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{0, 00110101} \quad \overleftarrow{R_1} = 0, 00111001 | 1$$

$$8) \overleftarrow{S_m} = 0, 01101010$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{1, 10100001} \quad \overleftarrow{R_1} = 0, 01110011 | 0$$

$$R_2 = \underline{0, 11001001} \text{ (восстановление)}$$

$$S_m = 0, 01101010$$

$$9) \overleftarrow{S_m} = 0, 11010100 \quad (\text{доп. цикл})$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{0, 00001011} \quad \overleftarrow{R_1} = 0, 11100110 | 1$$

$$\overleftarrow{R_1} = 0, 11100110 | 1$$

$$\underline{\quad\quad\quad 1} \quad (\text{округление})$$

$$R_1 = 0, 111001110$$

$$pA = 0001$$

$$\underline{-pB = 1110}$$

$$pC = 1111(-1 \text{ в доп.коде})$$

$$pC = 1001(-1 \text{ в прямом коде})$$

Ответ: $mC = 0,11100111$ $pC = 1001$.

В примере 13 в каждом цикле после сдвига сумматора производится вычитание делителя, далее анализируется знак сумматора и его инверсия заносится при сдвиге в регистр результата R_1 со стороны младших разрядов. Если после операции сумматор становится отрицательным, в результат заносится 0 и выполняется восстановление частичного остатка на сумматоре путем прибавления делимого к сумматору. В данном примере выполняется дополнительный 9-й цикл и находится дополнительная цифра частного, что позволяет сделать округление результата. Округление реализуется путем добавления единицы к дополнительному разряду, таким образом, в половине случаев получается ответ с избытком, в половине случаев – с недостатком, систематическая ошибка не накапливается. Однако следует отметить, в отличие от операции умножения округление при делении на реальных устройствах реализуется редко.

Существует метод деления без восстановления остатка, позволяющий сократить вычисления. Частичный остаток на сумматоре, используемый для определения очередной цифры частного можно выразить как

$$2SM - R_2,$$

где $2SM$ ничто иное, как сдвинутый сумматор в сторону старших разрядов. Если в случае отрицательного сумматора не делать восстановление, а сразу выполнить сдвиг в сторону старших разрядов, то получим

$$2(SM - R_2) = 2SM - 2R_2,$$

и если к этому прибавить R_2 , то получим

$$2SM - 2R_2 + R_2 = 2SM - R_2,$$

что и соответствует остатку в алгоритме с восстановлением.

Таким образом, для алгоритма без восстановления остатка имеем правило: в случае отрицательного сумматора восстановление в текущем цикле не выполняется, но в следующем цикле после сдвига сумматора выполняется сложение (уже не вычитание) делителя с сумматором. Структура устройства не изменилась и соответствует рис. 9.

В примере 14 приводится деление методом без восстановления остатка на устройстве с подвижным сумматором. В данном примере используются те же исходные данные, что для примера 13, можно убедиться, что значения частичных остатков на сумматоре после каждого цикла совпадают.

Пример 14.

$$\begin{array}{ll}
 mA = 0,10110101 & pA=0001 \\
 mB = 0,11001001 & pB=0010 \\
 Sm = 0,10110101 & R_1 = 0,00000000|0 \\
 R_2 = 0,11001001 \\
 -R_2 = 1,00110111 \\
 1) \overleftarrow{Sm} = 1,01101010 \\
 -R_2 = \underline{1,00110111} \\
 Sm = \underline{0},10100001 & \overleftarrow{R_1} = 0,00000000|1 \\
 2) \overleftarrow{Sm} = 1,01000010 \\
 -R_2 = \underline{1,00110111} \\
 Sm = \underline{0},01111001 & \overleftarrow{R_1} = 0,00000001|1 \\
 3) \overleftarrow{Sm} = 0,11110010 \\
 -R_2 = \underline{1,00110111} \\
 Sm = \underline{0},00101001 & \overleftarrow{R_1} = 0,00000011|1 \\
 4) \overleftarrow{Sm} = 0,01010010 \\
 -R_2 = \underline{1,00110111} \\
 Sm = \underline{1},10001001 & \overleftarrow{R_1} = 0,00000111|0
 \end{array}$$

$$5) \overleftarrow{S_m} = 1, 00010010$$

$$R_2 = \underline{0, 11001001}$$

$$S_m = \underline{1}, 11011011$$

$$\overleftarrow{R_1} = 0, 00001110 | 0$$

$$6) \overleftarrow{S_m} = 1, 10110110$$

$$R_2 = \underline{0, 11001001}$$

$$S_m = \underline{0}, 01111111$$

$$\overleftarrow{R_1} = 0, 00011100 | 1$$

$$7) \overleftarrow{S_m} = 0, 11111110$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{0}, 00110101$$

$$\overleftarrow{R_1} = 0, 00111001 | 1$$

$$8) \overleftarrow{S_m} = 0, 01101010$$

$$-R_2 = \underline{1, 00110111}$$

$$S_m = \underline{1}, 10100001$$

$$\overleftarrow{R_1} = 0, 01110011 | 0$$

$$9) \overleftarrow{S_m} = 1, 01000010$$

$$R_2 = \underline{0, 11001001}$$

$$S_m = \underline{0}, 00001011$$

$$\overleftarrow{R_1} = 0, 11100110 | 1$$

$$R_1 = 0, 11100110 | 1$$

$$\underline{\hspace{1.5cm}} | \underline{1}$$

$$R_1 = 0, 11100111 | 0$$

$$pA = 0001$$

$$-pB = \underline{1110}$$

$$pC = 1111 (-1 \text{ в доп. коде})$$

$$pC = 1001 (-1 \text{ в прямом коде})$$

$$\text{ОТВЕТ: } mC = 0, 11100111 \quad pC = 1001.$$

Если цикл операции деления с восстановлением остатка содержит 3 фазы: сдвиг, операция, восстановление, то цикл операции деления без восстановления остатка содержит 2 фазы: сдвиг, операция, что заметно сокращает объем вычислений. Тем не менее, рассмотренный метод без восстановления остатка не считается ускоренным.

2.3.2. Реализация операции деления на устройстве с неподвижным сумматором в прямых кодах

Рассмотрим реализацию операции деления двоичных чисел в формате с плавающей запятой в прямом коде с использованием алгоритма без восстановления остатка с неподвижным сумматором. На рис. 10 приводится упрощенная структура устройства, реализующего выполнение данной

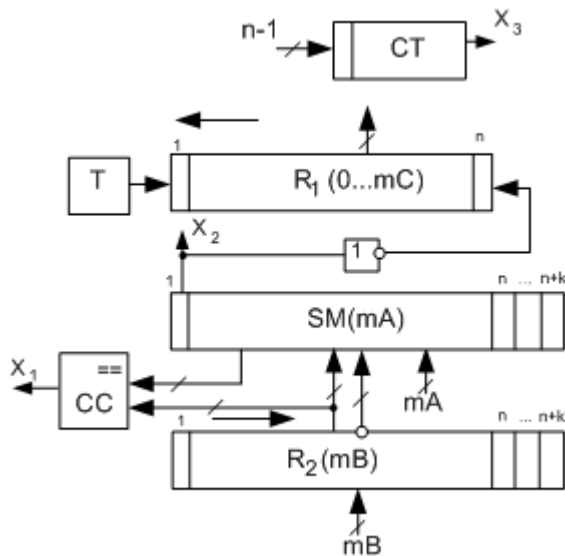


Рис. 10. Устройство деления в прямых кодах с неподвижным сумматором

операции. Устройство обработки мантисс также содержит два регистра для хранения операндов и накапливающий сумматор, дополнительно имеется триггер для хранения знака результата и счетчик для подсчета необходимого числа циклов, определяющегося количеством разрядов числа без учета знакового. Управляющие сигналы и часть устройства, выполняющего работу с порядками, не показаны.

Предполагается, что данные на вход устройства подаются в нормализованном виде. При выполнении операции $C = A/B$ мантисса mA загружается в сумматор SM , мантисса mB загружается в регистр R_2 , регистр формирования результата R_1 обнуляется, в счетчик загружается число разрядов числа без учета знакового – $n-1$.

К сумматору от R_2 подаются две многоразрядные связи, обеспечивающие выполнение двух операций на сумматоре: $SM = SM + R_2$ и $SM = SM - R_2$. Знак сумматора инвертируется и задвигается в регистр результата R_1 . Структура также содержит триггер знака T и схему сравнения CC , используемую перед основным циклом для определения возможности деления.

При выполнении операции деления порядки операндов должны вычитаться: $pC = pA - pB$, выполнение этой операции предполагается на некотором дополнительном сумматоре порядков (на рис. 10 не показано). Поскольку предполагается, что данные хранятся в прямом коде, то знаки операндов следует обработать отдельно и сохранить в триггере, то есть выполняется операция определения знака результата и сохранения его в триггере T так же, как при умножении:

$$T = Z_n(A) \oplus Z_n(B),$$

где $\oplus$ – логическая операция «сумма по модулю два»;

$Z_n(A)$ – знак A (первый разряд мантиссы mA);

$Z_n(B)$ – знак B (первый разряд мантиссы mB).

После чего знаки операндов необходимо принудительно обнулить.

При выполнении алгоритма данные будут сдвигаться в сторону младших разрядов, потому в сумматоре и связанном с ним регистре R_2 необходимо предусмотреть дополнительные разряды. Общее число разрядов соответствует $n + k$, где число дополнительных разрядов k исходя из обеспечения приемлемой погрешности ($\leq 2^n$) выбирается в соответствии с выражением

$$k \geq \log_2(n-k),$$

например, для $n = 8$ $k = 3$.

Исходя из необходимости получения нормализованного результата, перед выполнением непосредственно алгоритма деления необходимо сде-

лать проверку на $mA < mB$, данная проверка осуществляется с модулями операндов, при использовании прямых кодов после обнуления результатов на регистре и сумматоре как раз имеем соответственно модули операндов. Сравнение может быть выполнено с помощью специальной логической схемы сравнения (СС), либо, если таковая отсутствует, алгоритмически путем вычитания. В последнем случае из сумматора SM вычитается содержимое регистра R_2 , и если результат вычитания получается отрицательный (знаковый разряд сумматора, логическое условие $x_2 = 1$), то $mA < mB$, если положительный ($x_2 = 0$), то $mA < mB$ не выполняется и необходимо будет выполнить денормализацию операнда A, находящегося на сумматоре. После выполнения вычитания mA на сумматоре разрушается, и это можно компенсировать восстановлением сумматора, прибавив к нему R_2 , или, при использовании алгоритма без восстановления остатка, учесть это, и первый цикл начать со сложения на сумматоре. Денормализация заключается в сдвиге сумматора в сторону младших разрядов и увеличения порядка на единицу.

Собственно деление без восстановления остатка на устройстве с неподвижным сумматором реализуется следующим образом.

1. Выполняется сдвиг R_2 в сторону младших разрядов $R_2 \rightarrow$.
2. Если сумматор SM положительный ($SM > 0$), выполняется операция на сумматоре $SM = SM - R_2$, если сумматор отрицательный ($SM < 0$), выполняется $SM = SM + R_2$.
3. Инверсия знакового разряда сумматора задвигается в регистр формирования результата при выполнении левого сдвига R_1 ($R_1 \leftarrow$).
4. Содержимое счетчика уменьшается на единицу $CT--$, если $CT \neq 0$, продолжить с 1.

По окончании указанных действий на R_1 должен находиться результат. Если производилась предварительная проверка и приведение к $mA < mB$, результат получается нормализованным.

Далее необходимо загрузить из триггера знак результата в первый разряд $R_1[1] = T$, после чего в R_1 находится мантисса результата операции умножения, предполагается, что порядок находится в специальном регистре или сумматоре pC .

Приведем пример выполнения данного алгоритма (пример 15).

В первом такте каждого цикла производится сдвиг R_2 в сторону младших разрядов, во втором такте выполняется операция на сумматоре, в третьем такте – загрузка очередного разряда результата в R_1 .

Пример 15.

$$\begin{array}{ll}
 m_A = 0,10110101 & p_A = 00101 \quad (5) \\
 m_B = 0,11001001 & p_B = 10001 \quad (-1) \\
 -m_B = 1,00110111 & \\
 S_m = 0,10110101 | 000 & \\
 R_2 = 0,11001001 | 000 & \\
 1) \overrightarrow{R_2} = 0,01100100 | 100 & \\
 -R_2 = 1,10011011 | 100 & \\
 S_m = \underline{0,10110101 | 000} & \\
 S_m = \underline{0,01010000 | 100} & R_1 = 0,00000001 \\
 2) \overrightarrow{R_2} = 0,00110010 | 010 & \\
 -R_2 = 1,11001101 | 110 & \\
 S_m = \underline{0,01010000 | 100} & \\
 S_m = \underline{0,00011110 | 010} & R_1 = 0,00000011 \\
 3) \overrightarrow{R_2} = 0,00011001 | 001 & \\
 -R_2 = 1,11100110 | 111 & \\
 S_m = \underline{0,00011110 | 010} & \\
 S_m = \underline{0,00000101 | 001} & R_1 = 0,00000111 \\
 4) \overrightarrow{R_2} = 0,00001100 | 100 & \\
 -R_2 = 1,11110011 | 100 & \\
 S_m = \underline{0,00000101 | 001} & \\
 S_m = \underline{1,11111000 | 101} & R_1 = 0,00001110 \\
 5) \overrightarrow{R_2} = \underline{0,00000110 | 010} &
 \end{array}$$

$$\begin{array}{ll}
Sm = \underline{1}, 11111110 | 111 & R_1 = 0, 00011100 \\
6) \overrightarrow{R_2} = \underline{0}, 00000011 | 001 & \\
Sm = \underline{0}, 00000010 | 000 & R_1 = 0, 00111001 \\
7) \overrightarrow{R_2} = 0, 00000001 | 100 & \\
-R_2 = 1, 11111110 | 100 & \\
Sm = \underline{0}, 00000010 | 000 & \\
Sm = \underline{0}, 00000000 | 100 & R_1 = 0, 01110011 \\
8) \overrightarrow{R_2} = 0, 00000000 | 110 & R_1 = 0, 11100111 \\
-R_2 = 1, 11111111 | 100 & \\
Sm = \underline{0}, 00000000 | 100 & \\
Sm = \underline{0}, 00000000 | 000 & R_1 = 0, 11100111
\end{array}$$

$$\begin{array}{ll}
pA = 00101 & (5) \\
-pB = 00001 & (1) \\
\hline
pC = 00110 & (6)
\end{array}$$

ОТВЕТ: $mC = 0, 11100111$ $pC = 00110$.

На устройстве рис. 10 также может быть реализован и алгоритм с восстановлением остатка.

2.3.3. Реализация операции деления в дополнительных кодах

Рассмотрим реализацию операции деления $A / B = C$ в дополнительных кодах на устройствах. Можно использовать как устройство с подвижным сумматором (рис. 9), так и устройство с неподвижным сумматором (рис. 10), однако получение очередного разряда частного c_i выполняется по более сложным правилам. Как и во всех предыдущих структурах, перед началом выполнения операции мантисса делимого загружается в сумматор ($Sm = mA$), мантисса делителя – во второй регистр ($R_2 = mB$), первый регистр, где будет формироваться частное, обнуляется ($R_1 = 0$).

Существует обобщенный алгоритм машинной операции деления, работающий непосредственно в дополнительных кодах как с положительными, так и с отрицательными числами без перевода их в прямой код. Особенность данного метода заключается в том, что очередной знак результата определяется не просто как инверсия знакового разряда сумматора, как в прямом коде, а как результат сравнения знаков частичного остатка на сумматоре со знаком делимого на втором регистре по следующему правилу:

Если $Зн(Sm) = Зн(R_2)$, очередная цифра частного $c_i = 1$ (задвигается в R_1), в следующем цикле после сдвига сумматора выполняется операция $Sm = Sm - R_2$.

Иначе, если $Зн(Sm) \neq Зн(R_2)$, очередная цифра частного $c_i = 0$ (задвигается в R_1), в следующем цикле после сдвига сумматора выполняется операция $Sm = Sm + R_2$.

Вычисления начинаются с нулевой итерации цикла, в которой только проверяются знаки сумматора и делителя и определяется операция на первый цикл. Далее в первом и последующих циклах выполняется: сдвиг, операция (в зависимости от c_{i-1} , предыдущего цикла), сравнение знаков (уже после операции) сумматора и делителя, занесение c_i в R_1 . Число итераций без учета 0-й есть число цифр после запятой. В конце вычислений выполняется поправка по следующему правилу:

Если $Зн(A) = Зн(B)$, то выполняется $R_1 = R_1 + 1,00...00$,

иначе $R_1 = R_1 + 1,00...01$.

Таким образом, константа коррекции формируется в зависимости от соотношения знаков делимого и делителя. Поскольку делимое загружается в сумматор и в ходе операции разрушается, константу коррекции лучше сформировать перед основным циклом операции.

Рассмотренный алгоритм является алгоритмом деления чисел в дополнительном коде методом без восстановления остатка со сдвигом сум-

матора. Можно также реализовать алгоритмы с восстановлением остатка, со сдвигом и без сдвига сумматора.

Структура устройства деления чисел в дополнительном коде со сдвигом сумматора приводится на рис. 11. В отличие от соответствующего устройства для прямых кодов, где очередной разряд частного c_i находился как инверсия знакового разряда сумматора, в данном устройстве очередной разряд частного c_i формируется логическим элементом «исключающего или» (суммы по модулю 2) с инверсным выходом на основе знаковых разрядов сумматора и второго регистра (регистра делителя). Триггер знака в структуре отсутствует, поскольку знак результата операции получается автоматически после коррекции. На регистр результата R_1 подается слово коррекции (СК), которое формируется до основного цикла операции. Схема сравнения (СС) в устройстве также является опциональной, поскольку проверку на возможность деления $|mA| < |mB|$ можно провести, алгоритмически вычитая или складывая операнды и анализируя знак результата, хотя аппаратно с по-

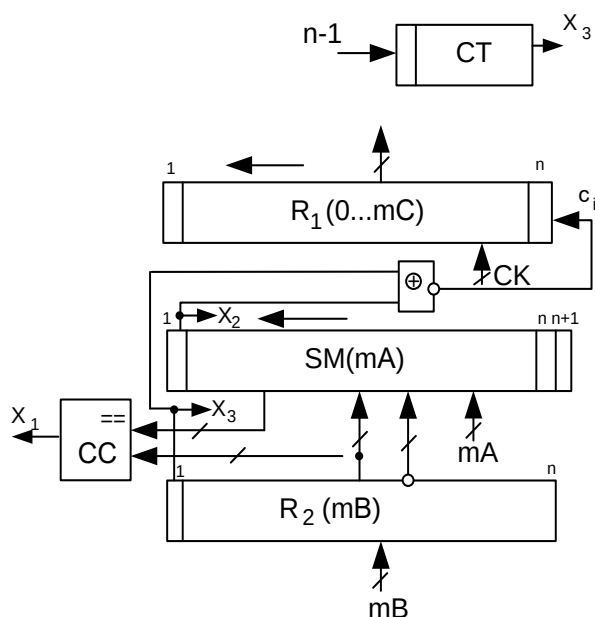


Рис. 11. Устройство деления в дополнительных кодах со сдвигом сумматора

мощью схемы сравнения это выполняется значительно быстрее.

Пример 16 иллюстрирует деление чисел в дополнительном коде без восстановления остатка со сдвигом сумматора.

Пример 16.

$$mA = 0,10110110$$

$$mB = 1,00110111$$

$$-mB = 0,11001001 \quad |mA| < |mB| \quad (\text{Для сравнения удобно } A \text{ и } B \text{ привести к положительным значениям})$$

$$\begin{aligned}
S_m &= 0,10110110 & R_1 &= 0,00000000 \\
R_2 &= \underline{1,00110111} \text{ (B)} \\
-R_2 &= 0,11001001 \text{ (-B)} & \overleftarrow{R_1} &= 0,0000000\underline{0} \\
1) \overleftarrow{S_m} &= 1,01101100 \\
R_2 &= \underline{1,00110111} \\
S_m &= \underline{0,10100011} & \overleftarrow{R_1} &= 0,000000\underline{00} \\
2) \overleftarrow{S_m} &= 1,01000110 \\
R_2 &= \underline{1,00110111} \\
S_m &= \underline{0,01111101} & \overleftarrow{R_1} &= 0,000000\underline{000} \\
3) \overleftarrow{S_m} &= 0,11111010 \\
R_2 &= \underline{1,00110111} \\
S_m &= \underline{0,00110001} & \overleftarrow{R_1} &= 0,000000\underline{0000} \\
4) \overleftarrow{S_m} &= 0,01100010 \\
R_2 &= \underline{1,00110111} \\
S_m &= \underline{1,10011001} & \overleftarrow{R_1} &= 0,000000\underline{0001} \\
5) \overleftarrow{S_m} &= 1,00110010 \\
-R_2 &= \underline{0,11001001} \\
S_m &= \underline{1,11111011} & \overleftarrow{R_1} &= 0,000000\underline{0011} \\
6) \overleftarrow{S_m} &= 1,11110110 \\
-R_2 &= \underline{0,11001001} \\
S_m &= \underline{0,10111111} & \overleftarrow{R_1} &= 0,00000\underline{0110} \\
7) \overleftarrow{S_m} &= 1,01111110 \\
R_2 &= \underline{1,00110111} \\
S_m &= \underline{0,10110101} & \overleftarrow{R_1} &= 0,0000\underline{1100} \\
8) \overleftarrow{S_m} &= 1,01101010 \\
R_2 &= \underline{1,00110111} & \overleftarrow{R_1} &= \underline{0,00011001} \\
S_m &= \underline{0,10100001} & \overleftarrow{R_1} &= \underline{0,00011001}
\end{aligned}$$

$3_n(A) \neq 3_n(B)$ (Коррекция)

$$R_1 = 0,00011001$$

1, 00000001 (Слово коррекции)

$$R_1 = 1, 00011010$$

Ответ: $mC = 1, 00011010$.

В рассмотренном примере первые цифры результата получаются нулевыми, потому для наглядности полученные цифры результата, заносимые в регистр R_1 , подчеркиваются.

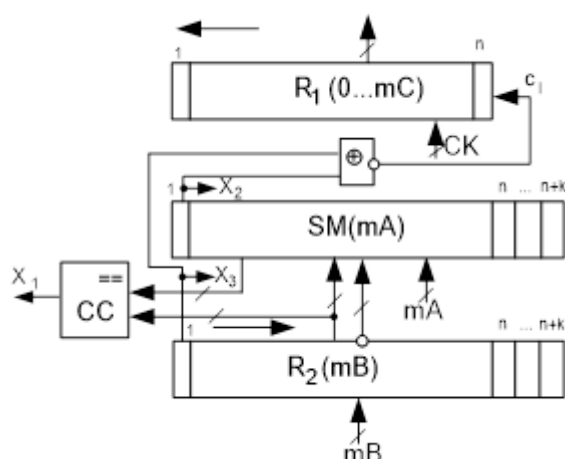


Рис. 12. Устройство деления
в дополнительных кодах
с неподвижным сумматором

Для реализации операции деления также можно использовать устройство с неподвижным сумматором (рис. 12). В данном устройстве сумматор неподвижен, второй регистр R_2 сдвигается в сторону младших разрядов, поэтому, чтобы избежать потери точности, необходимо в сумматоре и R_2 предусмотреть дополнительные разряды. Первый регистр, накапливающий результат, сдвигается в сторону старших разрядов.

Структура, так же как и предыдущая, содержит схему сравнения CC для определения возможности деления перед началом основного цикла; логический элемент сравнения $\oplus$ с инверсией для определения очередного разряда частного; формирователь слова коррекции $СК$, подключенный к первому регистру R_1 .

Пример 17 иллюстрирует деление чисел в дополнительном коде без восстановления остатка с неподвижным сумматором.

Пример 17.

$$mA = 0, 10110110$$

$$mB = 1, 00110111$$

$$\begin{array}{ll}
-mB = 0,11001001 & |mA| < |mB| \\
Sm = \underline{0,10110101|000} & R_1 = 0,00000000\underline{0} \\
R_2 = \underline{1,00110111|000} & \exists_H(Sm) \neq \exists_H(R_2), +R_2 \\
1) \overrightarrow{R_2} = \underline{1,10011011|100} & \\
Sm = \underline{0,10110101|000} & \\
Sm = \underline{0,01010000|100} & R_1 = 0,00000000\underline{0} \\
2) \overrightarrow{R_2} = \underline{1,11001101|110} & \\
Sm = \underline{0,00011110|010} & R_1 = 0,00000000\underline{0} \\
3) \overrightarrow{R_2} = \underline{1,11100110|111} & \\
Sm = \underline{0,00000101|001} & R_1 = 0,00000000\underline{0} \\
4) \overrightarrow{R_2} = \underline{1,11110011|100} & \\
Sm = \underline{1,11111000|101} & R_1 = 0,00000000\underline{1} \\
5) \overrightarrow{R_2} = \underline{1,11111001|110} & \\
-R_2 = 0,00000110|010 & \\
Sm = \underline{1,11111000|101} & \\
Sm = \underline{1,11111110|111} & R_1 = 0,00000000\underline{11} \\
6) \overrightarrow{R_2} = \underline{1,11111100|111} & \\
-R_2 = 0,00000011|001 & \\
Sm = \underline{1,11111110|111} & \\
Sm = \underline{0,00000010|000} & R_1 = 0,00000011\underline{0} \\
7) \overrightarrow{R_2} = \underline{1,11111110|100} & \\
Sm = \underline{0,00000010|000} & \\
Sm = \underline{0,00000000|100} & R_1 = 0,00001100\underline{0} \\
8) \overrightarrow{R_2} = \underline{1,11111111|010} & \\
Sm = \underline{1,11111111|110} & R_1 = 0,00011001 \\
& CK = \underline{1,00000001} \text{ (кор.)} \\
& R_1 = 1,00011010
\end{array}$$

ОТВЕТ: $mC = 1,00011010$.

Как видите, результаты примеров 16 и 17 совпадают.

2.3.4. Ускоренные методы деления

При выполнении операции деления прежде всего необходимо установить больше или меньше частичный остаток на сумматоре значения делителя, при работе в дополнительных кодах сравниваются модули этих значений. При реализации стандартных (не ускоренных) алгоритмов сравнение производится путем вычитания с последующим анализом знака результата, но это сравнение можно реализовать и путем анализа старших разрядов сумматора (на котором находится частичный остаток). Рассмотрим принцип ускоренного деления на примере прямых кодов (впоследствии распространим его и на дополнительные коды).

Имея в виду, что делитель в R_2 нормализован (то есть $R_2 = 0,1xxxxxxx$, где x – любая цифра), анализируя первые цифры сумматора, можно в некоторых случаях выполнить сравнение R_2 и Sm . Если сумматор положителен и его первая цифра после запятой равна нулю ($Sm = 0,0xxxxxxx$), то очевидно $|Sm| < |R_2|$, вычитание и восстановление можно не делать, а сразу записать очередную цифру результата как $c_i = 0$. При использовании метода без восстановления остатка сумматор может быть отрицательным, тогда если его первая цифра после запятой равна единице ($Sm = 1,1xxxxxxx$), то очевидно $|Sm| < |R_2|$ и прибавляя R_2 к такому сумматору, получим положительное число, в данном случае это сложение можно не делать, а сразу записать очередную цифру результата как $c_i = 1$. При выполнении операции следует иметь в виду, что после сдвига сумматора влево (по алгоритму деления) он может оказаться с переполнением, тогда в таком цикле операция обязательно должна быть выполнена, чтобы ликвидировать переполнение. В табл. 3 приводятся различные варианты разрядов сумматора, указывается выполняемая операция и, где это возможно, спрогнозированное значение c_i . При этом NOP (No Operation) – операция не выполняется, $c_i = *$ означает, что c_i определяется по результатам выполнения операции.

Таблица 3

Ускоренный метод деления с анализом одного разряда после запятой

Знак сумматора	Разряды сумматора	Операция	Очередной разряд результата c_i
SM>0	Sm=0,0xxxxxxx	NOP	0
	Sm=0,1xxxxxxx	Sm= Sm-R ₂	*
	Sm=1,xxxxxxx (переполнение)	Sm= Sm-R ₂	*
SM<0	Sm=1,0xxxxxxx	Sm= Sm + R ₂	*
	Sm=1,1xxxxxxx	NOP	1
	Sm=0,xxxxxxx (переполнение)	Sm= Sm + R ₂	*

Пример 18 иллюстрирует ускоренное деление чисел с анализом одного разряда после запятой в прямом коде без восстановления остатка со сдвигом сумматора.

Пример 18.

$$mA = 0,10110110$$

$$mB = 0,11101001 \quad |mA| < |mB|$$

$$Sm = 0,10110110 \quad R_1 = 0,00000000$$

$$R_2 = 0,11101001$$

$$-R_2 = 1,00010111$$

$$1) \overleftarrow{Sm} = 1,01101100$$

$$-R_2 = \underline{1,00010111}$$

$$Sm = \underline{0,10000011} \quad \overleftarrow{R_1} = 0,0000000\underline{1}$$

$$2) \overleftarrow{Sm} = 1,00000110 \quad (\text{переп. сумматора})$$

$$-R_2 = \underline{1,00010111}$$

$$Sm = \underline{0,00011101} \quad \overleftarrow{R_1} = 0,000000\underline{11}$$

$$3) \overleftarrow{Sm} = \underline{0,00111010} \quad \overleftarrow{R_1} = 0,00000\underline{110}$$

$$4) \overleftarrow{Sm} = \underline{0, 01110100} \quad \overleftarrow{R_1} = \quad 0, 0000\underline{1100}$$

$$5) \overleftarrow{Sm} = 0, 11101000$$

$$-R_2 = \underline{1, 00010111}$$

$$Sm = \underline{1, 11111111} \quad \overleftarrow{R_1} = \quad 0, 000\underline{11000}$$

$$6) \overleftarrow{Sm} = \underline{1, 11111110} \quad \overleftarrow{R_1} = \quad 0, 00\underline{110001}$$

$$7) \overleftarrow{Sm} = \underline{1, 11111110} \quad \overleftarrow{R_1} = \quad 0, 0\underline{1100011}$$

$$8) \overleftarrow{Sm} = \underline{1, 11111000} \quad \overleftarrow{R_1} = \quad 0, \underline{11000111}$$

Ответ: $mC = 0,11000111$.

В примере 18 во втором цикле после сдвига влево произошло переполнение сумматора и выполняется вычитание, текущий знак сумматора определяется до сдвига, после сдвига он может не сохраниться, как в данном цикле. В циклах 3 и 4 разряды результата находятся по ускоренному принципу (в соответствии с табл. 3 $c_i = 0$), в этих циклах операция не выполняется. Цикл 5 выполняется по стандартной схеме, c_i находится как инверсия знакового разряда сумматора после операции. Циклы 6–8 выполняются по ускоренному принципу для отрицательного сумматора с установкой $c_i = 1$, в этих циклах операция также не выполняется. Таким образом, в рассмотренном примере из восьми циклов операция выполнялась только в трех, что иллюстрирует значительный выигрыш в быстродействии в данном случае. Хотя вообще, анализируя варианты табл. 3 можно найти, что операция не выполняется в $\frac{2}{8} = \frac{1}{4}$ случаев (так как каждое переполнение соответствует двум комбинациям).

Также следует отметить, что ускоренный метод деления не уменьшает общего количества циклов, а только сокращает общее число операций.

Для реализации рассматриваемого метода для прямых кодов можно использовать структуру рис. 9, только к ней следует добавить анализ второго разряда сумматора.

Рассмотренный прием можно использовать и для работы в дополнительных кодах, только для указанных в табл. 3 случаев, когда операция не выполняется (NOP), разряд частного прогнозируется по более сложному правилу.

Пример 19 иллюстрирует ускоренное деление чисел с анализом одного разряда после запятой в дополнительном коде без восстановления остатка со сдвигом сумматора.

Пример 19.

$$mA = 0,10110110$$

$$mB = 1,00010111$$

$$Sm = \underline{0},10110110 \quad R_1 = 0,00000000$$

$$R_2 = \underline{1},00010111$$

$$-R_2 = 0,11101001 \quad |mA| < |mB|$$

$$1) \overleftarrow{Sm} = 1,01101100$$

$$+R_2 = \underline{1},00010111$$

$$Sm = \underline{0},10000011 \quad \overleftarrow{R_1} = 0,00000000$$

$$2) \overleftarrow{Sm} = 1,00000110$$

$$+R_2 = \underline{1},00010111$$

$$Sm = \underline{0},00011101 \quad \overleftarrow{R_1} = 0,00000000$$

$$3) \overleftarrow{Sm} = \underline{0},00111010 \quad \overleftarrow{R_1} = 0,00000001$$

$$4) \overleftarrow{Sm} = \underline{0},01110100 \quad \overleftarrow{R_1} = 0,00000011$$

$$5) \overleftarrow{Sm} = 0,11101000$$

$$+R_2 = \underline{1},00010111$$

$$Sm = \underline{1},11111111 \quad \overleftarrow{R_1} = 0,00000111$$

$$6) \overleftarrow{Sm} = \underline{1},11111110 \quad \overleftarrow{R_1} = 0,00001110$$

$$7) \overleftarrow{Sm} = \underline{1},11111110 \quad \overleftarrow{R_1} = 0,00011100$$

$$8) \overleftarrow{Sm} = \underline{1},11111000 \quad \overleftarrow{R_1} = 0,00111000$$

$$CK = \underline{1},00000001 \quad (\text{кор.})$$

$$R_1 = 1,00111001$$

Ответ: $mC = 1,00111001$.

В примере 19 в нулевом цикле сравниваются разряды делимого на сумматоре и делителя на R_2 , так как знаки не равны в результат заносится $c_i = 0$, при этом соответствующий разряд R_1 для наглядности подчеркивается, и в первом цикле выполняются операция сложения (в соответствии со стандартным алгоритмом работы в дополнительных кодах). В первом и втором циклах выполняются операции сложения, по окончании операции анализируются знаки сумматора, так как эти знаки не равны знаку делителя на R_2 в регистр результата заносятся $c_i = 0$ (также в соответствии со стандартным алгоритмом работы в дополнительных кодах). В третьем цикле после сдвига сумматор содержит число, гарантированно меньшее нормализованного делителя ($Sm = 0,0\dots$), поэтому, если из него вычесть нормализованный делитель, можно безошибочно спрогнозировать, что результат будет отрицательный, то есть знак результата на сумматоре будет совпадать со знаком делителя на R_2 , и можно, не выполняя операции, сразу записать $c_i = 1$. Четвертый цикл аналогичен третьему. Пятый цикл выполняется по стандартному алгоритму: в результате сложения с делителем получаем отрицательное число, по знаку совпадающее с делителем, и записываем $c_i = 1$. Циклы 6, 7, 8 выполняются по ускоренному принципу: во всех этих циклах сумматор после сдвига содержит малое отрицательное число, поэтому, прибавляя к нему положительное нормализованное число ($-R_2$), гарантированно получим положительный результат, который по знаку не будет совпадать с отрицательным делителем на R_2 , операция в этих циклах не выполняется, а в результат сразу помещается $c_i = 0$. В конце вычислений выполняется коррекция.

В примере 19 использованы те же операнды по модулю, что и в примере 18, только в нем делитель переведен в дополнительный код и является отрицательным, соответственно и результат в нем получается отрицательным и соответствует результату примера 18, переведенному в дополнительный код. Более того, если сравнивать элементарные операции примеров 18 и 19, они тоже по операндам совпадают, что является лишним доказательством правомерности метода в дополнительном коде, основанном на нахождении разности модулей операндов.

При выполнении ускоренного деления в дополнительных кодах, в тех циклах, в которых пропускается операция, очередная цифра частного c_i определяется в соответствии со знаком частичного остатка на сумматоре и знаком делителя на R_2 .

Табл. 4 является уточнением табл. 3 для дополнительных кодов. В случае отрицательного делимого при малых значениях положительного сумматора ($Sm = 0,0\dots$) очередной разряд частного $c_i = 1$, при малых значениях отрицательного сумматора ($Sm = 1,1\dots$) $c_i = 0$, как в примере 19. Соответственно при положительном делимом будет наоборот: в случае отрицательного делимого при малых значениях положительного сумматора ($Sm = 0,0\dots$) очередной разряд частного $c_i = 0$, при малых значениях отрицательного сумматора ($Sm = 1,1\dots$) $c_i = 1$, как в прямых кодах. Таким образом, можно построить общее правило: в случае отрицательного делимого при малых значениях положительного сумматора ($Sm = 0,0\dots$) очередной разряд частного определяется знаком делителя $c_i = R_2[1]$, при малых значениях отрицательного сумматора ($Sm = 1,1\dots$) очередной разряд частного определяется инверсией знака делителя $c_i = \overline{R_2[1]}$.

Таблица 4

**Ускоренный метод деления с анализом одного разряда после запятой
в дополнительных кодах**

Знак сумматора	Разряды сумматора	Операция	Очередной разряд результата c_i
SM>0	Sm = 0,0xxxxxxx	NOP	$R_2[1]$
	Sm = 0,1xxxxxxx	OP*	*
	Sm = 1,xxxxxxx (переполнение)	OP*	*
SM<0	Sm = 1,0xxxxxxx	OP*	*
	Sm = 1,1xxxxxxx	NOP	$\overline{R_2[1]}$
	Sm = 0,xxxxxxx (переполнение)	OP*	*

Для реализации рассматриваемого метода для дополнительных кодов можно использовать структуру рис. 11, только к ней следует добавить анализ второго разряда сумматора.

Анализируя большее число разрядов после запятой, можно еще снизить количество выполняемых операций. Известен алгоритм ускоренного деления на основе анализа двух разрядов после запятой, принцип которого приводится в табл. 5. Данная таблица включает варианты, соответствующие разрядам как сумматора, так и делителя, на пересечении строк и столбцов указывается, нужно ли выполнять операцию (OP), или операция пропускается (NOP), очередная цифра частного устанавливается путем сравнений знака сумматора и знака делителя по принципу, рассмотренному в предыдущем алгоритме (ускоренный метод с анализом одного разряда после запятой).

Таблица 5

**Ускоренный метод деления с анализом двух разрядов после запятой
в дополнительных кодах**

Разряды сумматора	Разряды делителя $R_2 = 0.10\dots$	Разряды делителя $R_2 = 0.11\dots$
$S_m = 0,00\dots$	$S_m < R_2$, NOP	$S_m < R_2$, NOP
$S_m = 0,01\dots$	$S_m < R_2$, NOP	$S_m < R_2$, NOP
$S_m = 0,10\dots$	OP	$S_m < R_2$, NOP
$S_m = 0,11\dots$	OP	OP
$S_m = 1,xx\dots$ (переполнение)	OP	OP

В табл. 5 приводятся только положительные значения разрядов сумматора и делителя, если в вычислениях используются отрицательные числа в дополнительном коде, необходимо проинвертировать их первые разряды.

Имея в виду, что вариант переполнения занимает половину возможных комбинаций, операция может быть пропущена в $\frac{5}{16}$ случаев, что только на $\frac{1}{16}$ случаев больше, чем для предыдущего алгоритма.

Для реализации данного алгоритма к структуре рис. 11 нужно добавить как анализ второго разряда сумматора, так и анализ двух разрядов делителя.

Пример 20 иллюстрирует ускоренное деление чисел с анализом двух разрядов после запятой в прямом коде без восстановления остатка со сдвигом сумматора.

Пример 20.

$$mA = 0,10100110$$

$$mB = 0,11101001 \quad |mA| < |mB|$$

$$Sm = 0,10100110$$

$$R_1 = 0,00000000$$

$$R_2 = 0,11101001$$

$$-R_2 = 1,00010111$$

$$1) \overleftarrow{Sm} = 1,01001100$$

$$-R_2 = \underline{1,00010111}$$

$$Sm = \underline{0,01100011}$$

$$\overleftarrow{R_1} = 0,0000000\underline{1}$$

$$2) \overleftarrow{Sm} = 0,11000110$$

$$-R_2 = \underline{1,00100111}$$

$$Sm = \underline{1,11101101}$$

$$\overleftarrow{R_1} = 0,000000\underline{10}$$

$$3) \overleftarrow{Sm} = \underline{1,11011010}$$

$$0,00$$

$$\overleftarrow{R_1} = 0,00000\underline{101}$$

$$4) \overleftarrow{Sm} = \underline{1,10110100}$$

$$0,01$$

$$\overleftarrow{R_1} = 0,0000\underline{1011}$$

$$5) \overleftarrow{Sm} = \underline{1,01101000}$$

$$0,10$$

$$\overleftarrow{R_1} = 0,000\underline{10111}$$

$$6) \overleftarrow{Sm} = 0,11010000$$

$$-R_2 = \underline{1,00010111}$$

$$Sm = \underline{1,11100111}$$

$$\overleftarrow{R_1} = 0,00\underline{101110}$$

$$7) \overleftarrow{Sm} = \underline{1,11001110}$$

$$\begin{array}{ll}
0,00 & \overleftarrow{R_1} = 0,0\underline{1011101} \\
8) \overleftarrow{Sm} = \underline{1,10011100} & \\
0,01 & \overleftarrow{R_1} = 0,\underline{10111011}
\end{array}$$

ОТВЕТ: $mC = 0,10111011$.

В примере 20 циклы 1 и 2 выполняются по стандартному алгоритму (без ускорения). В циклах 3, 4, 5 сумматор отрицательный, поэтому, чтобы выполнить быстрое сравнение, старшие разряды инвертируются (подписаны ниже), в 5-м цикле операция пропускается за счет анализа двух разрядов после запятой ($0,10 Sm < 011R_2$), при использовании алгоритма с анализом одного разряда после запятой операция выполнялась бы. В 6-м цикле после сдвига сумматор меняет знаковый разряд, то есть произошло переполнение, здесь операция в обязательном порядке выполняется. Циклы 7 и 8 выполняются по ускоренному правилу для отрицательного сумматора.

Ускоренный метод деления с анализом двух разрядов также можно распространить на случай дополнительных кодов, для этого с учетом поправки на знак можно пользоваться табл. 5 и с учетом использования двух разрядов способом определения очередного разряда частного, приведённого в табл. 4.

Пример 21 иллюстрирует ускоренное деление чисел с анализом двух разрядов после запятой в дополнительном коде без восстановления остатка со сдвигом сумматора.

Пример 21.

$$\begin{array}{ll}
mA = 0,10100110 & \\
mB = 1,00011101 & \\
Sm = 0,10100110 & \\
R_2 = \underline{1},00011101 & R_1 = 0,00000000 \\
-R_2 = \underline{0},\underline{11}100011 & (\text{модуль } mB, |mA| < |mB|)
\end{array}$$

- 1) $\overleftarrow{Sm} = 1, 01001100$
 $R_2 = \underline{1, 00011101}$
 $Sm = \underline{0, 01101001}$ $\overleftarrow{R_1} = 0, 00000000$
- 2) $\overleftarrow{Sm} = 0, 11010010$
 $R_2 = \underline{1, 00011101}$
 $Sm = \underline{1, 11101111}$ $\overleftarrow{R_1} = 0, 00000001$
- 3) $\overleftarrow{Sm} = \underline{1, 11011110}$ (0,00 < 0,11, прогноз: $3H Sm = 0, c_i = 0$)
 $\overleftarrow{R_1} = 0, 00000010$
- 4) $\overleftarrow{Sm} = \underline{1, 10111100}$ (0,01 < 0,11, прогноз $3H Sm = 0, c_i = 0$)
 $\overleftarrow{R_1} = 0, 00000100$
- 5) $\overleftarrow{Sm} = \underline{1, 01111000}$ (0,10 < 0,11, прогноз $3H Sm = 0, c_i = 0$)
 $\overleftarrow{R_1} = 0, 00001000$
- 6) $\overleftarrow{Sm} = 0, 11110000$
 $-R_2 = \underline{0, 11100011}$
 $Sm = \underline{1, 11010011}$ ($3H Sm = 3H R_2$)
 $\overleftarrow{R_1} = 0, 00010001$
- 7) $\overleftarrow{Sm} = \underline{1, 10100110}$ (0,01 < 0,11, прогноз $3H Sm = 0, c_i = 0$)
 $\overleftarrow{R_1} = 0, 00100010$
- 8) $\overleftarrow{Sm} = \underline{1, 01001100}$ (0,10 < 0,11 прогноз: $3H Sm = 0, c_i = 0$)
 $\overleftarrow{R_1} = 0, 01000100$
 $CK = \underline{1, 00000001}$ (кор.)
 $\underline{1, 01000101}$

ОТВЕТ: $mC = 1, 01000101$.

В примере 21 действия начинаются со сравнения знаков операндов в 0-м цикле. В циклах 3–5, 7–8 при малых значениях сумматора (по модулю) знак результата операции на сумматоре прогнозируется (в данных случаях как 0, то есть положительный), и он не равен знаку делителя, в результат заносится $c_i = 0$. Или можно, воспользовавшись принципом, опи-

санным в табл. 4, в результат занести инверсию знака делителя, то есть $c_i = \overline{R_2[1]}$, что эквивалентно. В 6-м цикле после сдвига сумматор оказывается с переполнением, операция выполняется. В конце производится коррекция.

Рассмотренные алгоритмы деления для дополнительных кодов, как ускоренные, так и не ускоренные, вполне применимы и для работы в обратных кодах, только следует выполнять все правила арифметики для обратного кода: отрицание $-A$ формируется из A путем инверсии всех разрядов (без прибавления единицы к младшему разряду), при выполнении сложения в случае переноса из старшего разряда прибавляется единица к младшему, существует -0 (111111), который сразу нужно обращать в 0 (000000) и слово коррекции $СК = 1,00...0$ вне зависимости от соотношения знаков операндов.

Вопросы и задания

1. Для заданных A и B дать пошаговую иллюстрацию выполнения деления $C = A/B$ указанным методом в прямых кодах ($mA = 0.10010011$ $pA = 0110$, $mB = 0.11010101$ $pB = 0011$):

- а) с подвижным сумматором с восстановлением остатка;
- б) с подвижным сумматором без восстановления остатка;
- в) с неподвижным сумматором с восстановлением остатка;
- г) с неподвижным сумматором без восстановления остатка;
- е) ускоренным методом с анализом одного разряда после запятой;
- ф) ускоренным методом с анализом двух разрядов после запятой.

2. Для заданных A и B дать пошаговую иллюстрацию выполнения деления $C = A/B$ указанным методом в дополнительных кодах ($mA = 0.10110011$ $pA = 0110$, $mB = 1.00111101$ $pB = 1110$):

- а) с подвижным сумматором с восстановлением остатка;
- б) с подвижным сумматором без восстановления остатка;
- в) с неподвижным сумматором с восстановлением остатка;

- d) с неподвижным сумматором без восстановления остатка;
- e) ускоренным методом с анализом одного разряда после запятой;
- f) ускоренным методом с анализом двух разрядов после запятой.

2.4. РЕАЛИЗАЦИЯ ОПЕРАЦИИ ИЗВЛЕЧЕНИЯ КВАДРАТНОГО КОРНЯ

Операция извлечения квадратного корня также является типовой длинной арифметической операцией, как будет вскоре показано, по реализации ненамного сложнее операции деления. Известен ручной метод извлечения квадратного корня «в столбик». Во всех программных системах обычно присутствует функция извлечения квадратного корня (обычно $\text{sqrt}(x)$), которую по возможности рекомендуется использовать, чем возведение в степень $1/2$.

Рассмотрим принцип реализации арифметической операции извлечения квадратного корня $Y = \sqrt{X}$, где X – подкоренное выражение; Y – результат извлечения корня. Пусть $Y = 0, y_1 y_2 \dots y_n$, где y_i – i -я цифра результата.

Поскольку $(0,1)^2 = 0,01$, можно определить первую после запятой цифру результата y_1 исходя из проверки: если $X \geq 0,01$, то $y_1 = 1$, иначе $y_1 = 0$. Поскольку на входе операнд X нормализован, или в случае нечетного порядка денормализуется на один разряд, условие автоматически выполняется, то есть $y_1 = 1$, что обеспечивает нормализацию результата. Сравнение $X \geq 0,01$, реализуется путем вычитания из сумматора Sm , в который изначально загружается мантисса X содержимого вспомогательного регистра R_2 , в который изначально загружается $0.010\dots 0$ ($0,01$ остальные 0), и анализа знака сумматора после операции.

Далее определим вторую цифру результата y_2 путем следующего сравнения:

если $X - 0, y_1 1^2 \geq 0$, то $y_2 = 1$, иначе $y_2 = 0$. Преобразуем данное выражение:

$$X - (0, y_1 + 2^{-2})^2 \geq 0,$$

$$X - 0, y_1^2 - 2 \cdot 0, y_1 \cdot 2^{-2} - 2^{-4} \geq 0,$$

$$X - 0, y_1^2 - 2^{-1}(0, y_1 + 2^{-3}) \geq 0, \text{ где первая часть выражения}$$

$X_1 = X - 0, y_1^2$ есть ничто иное, как разность, выполняемая на сумматоре в первом цикле, отсюда

$$X_1 - 2^{-1}(0, y_1 + 2^{-3}) \geq 0,$$

$$2X_1 - (0, y_1 + 2^{-3}) \geq 0,$$

$2X_1 - 0, y_1 01 \geq 0$, где второй операнд разности также формируется на вспомогательном регистре $R_2 = 0, y_1 010 \dots 0$, то есть первая цифра результата с дописанной к ним парой 01.

Рассуждая подобным образом для k -й цифры частного y_k , получим правило:

$$\text{если } 2X_{k-1} - 0, y_1 y_2 \dots y_{k-1} 010 \dots 0 \geq 0, \text{ то } y_k = 1, \text{ иначе } y_k = 0,$$

где $2X_{k-1}$ – сдвинутая влево разность на сумматоре, полученная в предыдущем цикле (если разность получается отрицательной, ее необходимо восстановить путем прибавления к сумматору вспомогательного регистра $S_m = S_m + R_2$; $0, y_1 y_2 \dots y_{k-1} 010 \dots 0$ – формируемый на вспомогательном регистре R_2 код, получаемый на основе полученных в предыдущих циклах разрядов результата (начиная со старшего) с дописыванием к ним пары 01 со стороны младших разрядов, остальные 0 (до конца разрядной сетки).

На основе приведенных рассуждений имеем алгоритм извлечения квадратного корня с восстановлением остатка на устройстве с подвижным сумматором.

Подкоренное выражение загружается в сумматор $S_m = mA$, в вспомогательный регистр R_2 загружается $0.010 \dots 0$, в регистр результата R_1 в начале загружается $0.0 \dots 0$, в R_1 будет формироваться результат, в счетчик $CT = n-1$.

Перед основным циклом находим порядок результата. Если порядок операнда pX четный, порядок результата определяем сдвигом вправо:

$pY = \overrightarrow{pX}$, что соответствует делению на 2; если порядок операнда pX нечетный выполняем денормализацию мантиссы на сумматоре $\overrightarrow{Sm}$, порядок увеличиваем на 1 ($pX++$) и он становится четным, также находим $pY = \overrightarrow{pX}$.

Замечание. Если при реализации операции умножения работу с порядками можно выполнить как до, так и после основного цикла, то при извлечении квадратного корня обязательно до, поскольку в случае нечетного порядка содержимое сумматора изменяется.

Основной цикл операции:

1. Формируется содержимое R_2 (в первом цикле 0.010...0; во втором 0. y_1 010...0, где y_1 – цифра результата, найденная в 1-м цикле; в k -м цикле 0, $y_1y_2...y_{k-1}$ 010 ... 0, где y_i – цифра результата, найденная в i -м цикле).
2. Выполняется вычитание на сумматоре $Sm = Sm - R_2$.
3. Инверсия знакового разряда сумматора задвигается в регистр формирования результата при выполнении левого сдвига $R_1 \leftarrow$, $y_i = \overrightarrow{SM}[1]$.
4. Если сумматор отрицательный $Sm[1]=1$, выполняется восстановление остатка $Sm = Sm + R_2$.
5. Выполняется сдвиг SM в сторону старших разрядов, $Sm \leftarrow$;
6. Содержимое счетчика уменьшается на единицу $CT--$, если $CT > 0$, продолжить с 1.
7. Вывод R_1 , pY .

Замечание. В отличие от деления сдвиг сумматора влево выполняется в конце цикла, таким образом, первое вычитание производится без сдвига.

Структура устройства, реализующего операцию извлечения квадратного корня с восстановлением остатка и сдвигом сумматора, приводится на рис. 13. Здесь в регистр порядков RP заносится порядок подкоренного выражения, младший разряд которого анализируется: если $RP[m] = 1$, порядок нечетный и следует выполнить денормализацию X , регистр имеет возможность правого сдвига, что позволяет определить порядок результата pY . В счетчик CT заносится $n-1$ (общее число разрядов минус

знаковый). В сумматор Sm изначально заносится мантисса подкоренного выражения mX , потом в ходе вычисления формируется частичный остаток, сумматор имеет возможность как сдвига вправо (используется при денормализации), так и сдвига влево (используется в цикле алгоритма).

Регистр R_2 – вспомогательный регистр, принцип формирования его содержимого в зависимости от номера цикла указан под регистром. В регистре R_1 формируется мантисса результата mY , R_1 сдвигается влево, очередная цифра результата y_i заносится со стороны младших разрядов.

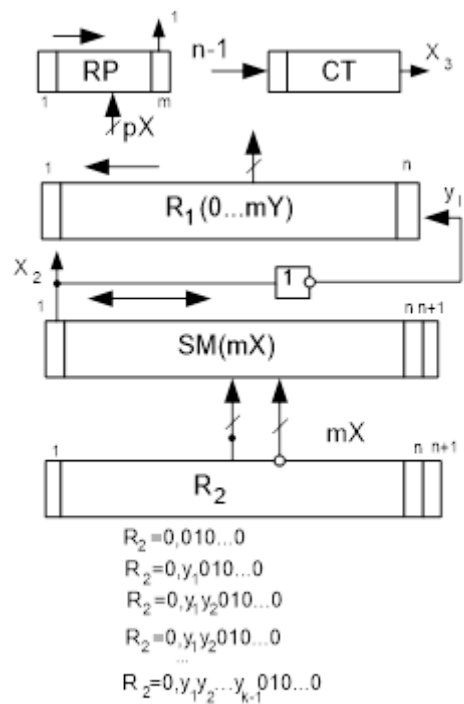


Рис. 13. Устройство извлечения квадратного корня

Пример 22 иллюстрирует выполнение операции извлечения квадратного корня с восстановлением остатка и сдвигом сумматора.

Пример 22.

$$mX = 0,10101101 \quad pX = 00100(4)$$

$$pY = 00010(2)$$

$$Sm = 0,10101101 | 0 \quad R_1 = 0,00000000$$

$$1) R_2 = 0,01000000 | 0$$

$$-R_2 = 1,11000000 | 0$$

$$Sm = \underline{0,10101101} | 0$$

$$Sm = \underline{0,01101101} | 0 \quad \overleftarrow{R_1} = 0,0000000\underline{1}$$

$$2) R_2 = 0,10100000 | 0$$

$$-R_2 = 1,01100000 | 0$$

$$\overleftarrow{Sm} = \underline{0,11011010} | 0$$

$$Sm = \underline{0,00111010} | 0 \quad \overleftarrow{R_1} = 0,000000\underline{11}$$

$$\begin{aligned}
3) R_2 &= 0, 11010000 | 0 \\
-R_2 &= 1, 00110000 | 0 \\
\overleftarrow{S_m} &= \underline{0, 01110100 | 0} \\
S_m &= \underline{1, 10100100 | 0} \quad \overleftarrow{R_1} = 0, 00000\underline{110} \\
R_2 &= \underline{0, 11010000 | 0} \quad (\text{восстановление}) \\
S_m &= 0, 01110100 | 0 \\
4) R_2 &= 0, 11001000 | 0 \\
-R_2 &= 1, 00111000 | 0 \\
\overleftarrow{S_m} &= \underline{0, 11101000 | 0} \\
S_m &= \underline{0, 00100000 | 0} \quad \overleftarrow{R_1} = 0, 0000\underline{1101} \\
5) R_2 &= 0, 11010100 | 0 \\
-R_2 &= 1, 00101100 | 0 \\
S_m &= \underline{0, 01000000 | 0} \\
S_m &= \underline{1, 01101100 | 0} \quad \overleftarrow{R_1} = 0, 000\underline{11010} \\
R_2 &= \underline{0, 11010100 | 0} \quad (\text{восстановление}) \\
S_m &= 0, 01000000 | 0 \\
6) R_2 &= 0, 11010010 | 0 \\
-R_2 &= 1, 00101110 | 0 \\
\overleftarrow{S_m} &= \underline{0, 10000000 | 0} \\
S_m &= \underline{1, 10101110 | 0} \quad \overleftarrow{R_1} = 0, 00\underline{110100} \\
R_2 &= \underline{0, 11010010 | 0} \quad (\text{восстановление}) \\
S_m &= 0, 10000000 | 0 \\
7) R_2 &= 0, 11010001 | 0 \\
-R_2 &= 1, 00101111 | 0 \\
\overleftarrow{S_m} &= \underline{1, 00000000 | 0} \\
S_m &= \underline{0, 00101111 | 0} \quad \overleftarrow{R_1} = 0, 0\underline{1101001} \\
8) R_2 &= 0, 11010010 | 1 \\
-R_2 &= 1, 00101101 | 1 \\
\overleftarrow{S_m} &= \underline{0, 01011110 | 0}
\end{aligned}$$

$$\begin{aligned}
S_m &= \underline{1}, 10001011 | 1 \quad \overleftarrow{R_1} = 0, \underline{11010010} \\
R_2 &= \underline{0, 11010010 | 1} \quad (\text{восстановление}) \\
S_m &= 0, 01011110 | 0
\end{aligned}$$

ОТВЕТ: $mY = 0, 11010010$, $pY = 00010 (2)$.

В примере 22 для сокращения записи сдвиг сумматора из конца цикла перенесен в следующий цикл. В начале первого цикла в R_2 заносится 0,01, то есть $R_2 = 0,01000000$, в первом цикле получена первая цифра результата $y_1 = 1$, поэтому формируется $R_2 = 0, \underline{10100000}$, во втором цикле получена ещё и вторая цифра результата $y_2 = 1$, формируется $R_2 = 0, \underline{11010000}$, в третьем $y_3 = 0$, формируется $R_2 = 0, \underline{11001000}$ и т. д., в последнем цикле используется дополнительный разряд. Порядок операнда четный, порядок результата получается путем правого сдвига порядка операнда (рассчитывается в самом начале примера).

Рассмотренный ранее прием ускорения операции деления вполне может быть применен при реализации операции извлечения квадратного корня. Как уже отмечалось, при нормализованных данных на входе или денормализованных на один разряд вправо первая цифра результата всегда 1, то есть результат при этих условиях гарантированно нормализован, и содержимое R_2 после второго цикла также нормализовано. Перед тем как выполнять в очередном цикле операцию на сумматоре, а в начале цикла он всегда положительный, можно проанализировать его старшие разряды, если $S_m = 0.0....$ (сумматор без переполнения и после запятой 0), его содержимое гарантировано меньше нормализованного содержимого R_2 , поэтому можно сразу записать очередную цифру частного $y_i = 0$, а вычитание и последующее восстановление не выполнять.

Пример 23 иллюстрирует реализацию ускоренного алгоритма операции извлечения квадратного корня с восстановлением остатка и сдвигом сумматора.

Пример 23.

$$mX = 0,10101101 \quad pX = 00101(5)$$

$$Sm = 0,10101101|0 \quad R_1 = 0,00000000$$

$$\overrightarrow{Sm} = 0,01010110|1 \quad pX = 00110(6)$$

$$1) R_2 = 0,01000000|0$$

$$-R_2 = 1,11000000|0$$

$$Sm = \underline{0,01010110|1}$$

$$Sm = \underline{0,00010110|1} \quad \overleftarrow{R_1} = 0,0000000\underline{1}$$

$$2) \overleftarrow{Sm} = \underline{0,00101101|0} \quad \overleftarrow{R_1} = 0,000000\underline{10}$$

$$3) \overleftarrow{Sm} = \underline{0,01011010|0} \quad \overleftarrow{R_1} = 0,00000\underline{100}$$

$$4) R_2 = 0,10001000|0$$

$$-R_2 = 1,01111000|0$$

$$\overleftarrow{Sm} = \underline{0,10110100|0}$$

$$Sm = \underline{0,00101100|0} \quad \overleftarrow{R_1} = 0,0000\underline{1001}$$

$$5) \overleftarrow{Sm} = \underline{0,01011000|0} \quad \overleftarrow{R_1} = 0,000\underline{10010}$$

$$6) R_2 = 0,10010010|0$$

$$-R_2 = 1,01101110|0$$

$$\overleftarrow{Sm} = \underline{0,10110000|0}$$

$$Sm = \underline{0,00011110|0} \quad \overleftarrow{R_1} = 0,00\underline{100101}$$

$$7) \overleftarrow{Sm} = \underline{0,00111100|0} \quad \overleftarrow{R_1} = 0,0\underline{1001010}$$

$$8) \overleftarrow{Sm} = \underline{0,01111000|0} \quad \overleftarrow{R_1} = 0,\underline{10010100}$$

$$pY = \overrightarrow{pX} = 00011(3)$$

ОТВЕТ: $mY = 0,10010100$, $pY = 00011(3)$.

В примере 22 в циклах 2, 3, 5 и 7, 8 операция не выполняется (используется ускоренный прием. Обратите внимание, что мантисса операндов для примеров 22 и 23 совпадала, однако в примере 22 порядок был четный, в примере 23 – нечетный и результаты сильно отличаются друг от

друга (это эффект четной и нечетной степени). Так, $\sqrt{4} = \sqrt{4 \cdot 10^0} = 2 = 2 \cdot 10^0$, $\sqrt{400} = \sqrt{4 \cdot 10^2} = 20 = 2 \cdot 10^1$, но $\sqrt{40} = \sqrt{4 \cdot 10^1} = 6,3245553$.

Итак, были рассмотрены варианты реализации базовых арифметических операций двоичной арифметики на устройствах, однако приведенными методами и структурами устройств далеко не исчерпывается весь возможный арсенал их реализации, можно привести еще достаточно много вариантов различных алгоритмов выполнения арифметических операций и соответствующих им структур устройств.

Вопросы и задания

1. Для заданного X дать пошаговую иллюстрацию выполнения операции извлечения квадратного корня в прямых кодах методом без ускорения ($mX = 0.11010011$ $pX = 0110$).
2. Для заданного X дать пошаговую иллюстрацию выполнения операции извлечения квадратного корня в прямых кодах методом с ускорением ($mX = 0.11010011$ $pX = 0111$).

3. ОСНОВЫ РАЗРАБОТКИ УСТРОЙСТВ ДВОИЧНО-ДЕСЯТИЧНОЙ АРИФМЕТИКИ

3.1. ДВОИЧНО-ДЕСЯТИЧНЫЕ КОДЫ

Существует два принципиально различных типа задач: технические и экономические. Задачи технического типа характеризуются относительно небольшим числом входных и выходных данных, но достаточно большой сложностью вычислений (вычисление сложных функций, решение систем дифференциальных уравнений, сложные итерационные вычислительные алгоритмы). Задачи экономического типа, напротив, характеризуются относительно несложными вычислениями (посчитать сумму, найти среднее), но большим количеством входных и выходных данных. При использовании непосредственно двоичной арифметики при решении задач экономического типа значительный вычислительный ресурс тратится на перевод чисел из привычной человеку десятичной системы исчисления в двоичную и обратно. В этой связи и были предложены так называемые двоично-десятичные коды, позволяющие минимизировать затраты на перевод из десятичной системы счисления в двоичную и обратно, и соответственно двоично-десятичная арифметика, имеющая ряд специфических особенностей.

Ряд вычислительных систем непосредственно поддерживают форматы десятичных чисел. На основе десятичной арифметики реализованы практически все микрокалькуляторы, работа с которыми связана с постоянным вводом и выводом данных пользователей.

Двоично-десятичные коды позволяют однозначно представлять десятичные числа в двоичном виде, при этом каждой десятичной цифре числа соответствует тетрада (четыре разряда) двоично-десятичного кода.

В данном пособии будет рассмотрена реализация арифметических операций с числами в двоично-десятичных кодах D8421 и D8421+3, получивших наибольшее распространение. В табл. 6 приводится соответствие цифр десятичного кода кодам D8421 и D8421+3, хотя существует доволь-

но много разновидностей двоично-десятичных кодов. Основные требования, предъявляемые к двоично-десятичным кодам, следующие:

- 1) однозначность, каждой десятичной цифре должен соответствовать уникальная комбинация двоичных цифр;
- 2) упорядоченность первого рода, большей десятичной цифре соответствует большая комбинация двоичных цифр;
- 3) упорядоченность второго рода, четной (нечетной) десятичной цифре соответствует четная (нечетная) комбинация двоичных цифр;
- 4) самодополняемость, дополнительный код двоично-десятичного числа вычисляется по стандартным правилам двоичной арифметики (поразрядная инверсия плюс единица к младшему разряду);
- 5) соответствие двоичному порядку, десятичная цифра кодируется своим двоичным эквивалентом.

Таблица 6

Двоично-десятичные коды D8421и D8421+3

Десятичное число	D8421	D8421+3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Не существует такого кода, для которого бы все требования выполнялись, поэтому их и используется довольно большое количество. Для двоично-десятичного кода D8421 выполняются все требования, кроме само-

дополняемости, что ограничивает его применения, тогда как в D8421+3 это требование соблюдается, что делает его привлекательным.

Выполнение операции сложения в двоично-десятичных кодах требует коррекции.

Правила сложения чисел в двоично-десятичном коде D8421 следующие:

1) выполняется сложение чисел в двоично-десятичном коде по общим правилам двоичной арифметики;

2) выполняется коррекция тетрады, из которой был перенос или значение кода которой превышает 9, коррекция заключается в прибавлении к тетраде кода 0110 (6). Межтетрадные цепи переноса при коррекции должны быть замкнуты.

В D8421 используется 10 двоичных комбинаций из 16, 6 не используется, именно 6 и прибавляется при коррекции.

В некоторых случаях может потребоваться провести коррекцию повторно. Сложение двух чисел A и B в D8421 иллюстрирует пример 24, сложение выполняется побитно, для наглядности тетрады разделены пробелами.

Пример 24.

A=0001 1000 0110	A=186
B= <u>0011 1001 0101</u>	B= <u>395</u>
0101 <u>0001</u> 1011	C=581
0110 0110	
C=0101 1000 0001	

В примере 24 в младшей тетраде после сложения получается **1011** (>9) и к ней прибавляется коррекция; во второй тетраде код допустим, но возникает перенос в следующую тетраду, поэтому и к ней прибавляется коррекция; в старшей тетраде коррекция не требуется.

В примере 25 требуется повторная коррекция.

Пример 25.

A=0001 1001 1001	A=199
B= <u>0000 0000 0001</u>	B= <u>001</u>
0001 1001 1010	C=200
<u>0110</u>	
0001 1010 <u>0000</u>	
<u>0110</u>	
C=0010 <u>0000</u> 0000	

В примере 25 после первой коррекции возникла тетрада с кодом (>9), поэтому потребовалось еще сделать коррекцию. Если при коррекции возникает межтетрадный перенос, это само по себе не требует дополнительной коррекции.

Как в двоичной арифметике, в двоично-десятичных кодах вычитание реализуется через дополнительный код, то есть отрицательные числа представляются в дополнительном коде, например -0.123 в дополнительном коде будет представлено как 9.877 , непосредственное сложение этих чисел обнуляет разрядную сетку и дает перенос из старшего разряда.

0.123
<u>9.877</u>
<u>0.000</u>

Здесь используются вещественные числа, на знак отводится целая тетрада, 9 перед запятой указывает, что число отрицательное. Поскольку D8421 не является самодополняемым, для получения дополнительного кода требуется специфическая коррекция. В примере 26 иллюстрируется получение дополнительного кода в D8421.

Пример 26.

A = 0000. 0001 0010 0011	A = 0.123
A _д = 1111. 1110 1101 1101	
СК = <u>1010</u> . <u>1010</u> <u>1010</u> <u>1010</u>	
-A = 1001. 1000 0111 0111	-A = 9.877

В примере 26 сначала находится дополнительный код исходного числа A по стандартным правилам двоичной арифметики A_д, затем к нему прибавляется слово коррекции СК, состоящее из тетрад 1010(-6). Цепи межатетрадных переносов при коррекции должны быть заблокированы (иллюстрируется разрывами в черте суммирования).

В примере 27 иллюстрируется вычитание в D8421

Пример 27. C=B-A

B = 0000. 0101 0001 0111	B = 0.517	B = 0.517
-A = <u>1001</u> . <u>1000</u> <u>0111</u> <u>0111</u>	-A = <u>-0.123</u>	-A = <u>9.877</u>
1001. 1101 1000 1110	C = 0.394	C = <u>0.394</u>
0110 0110		
<u>1010.</u> <u>0011</u> 1001 <u>0100</u>		
<u>0110</u>		
C = <u>0000</u> . 0011 1001 0100		

В примере 27 -A подставляется сразу в дополнительном коде, после суммирования выполняется коррекция в двух тетрадах (цепи межатетрадного переноса замкнуты) и потом дополнительная коррекция старшей тетрады.

Правила сложения чисел в двоично-десятичном коде D8421+3 следующие:

1) выполняется сложение чисел в двоично-десятичном коде по общим правилам двоичной арифметики;

2) выполняется коррекция для тетрад, из которых был перенос 0011 (+3), и для тетрад, из которых не было переноса 1101 (–3). Межтетрадные цепи переноса при коррекции должны быть разомкнуты.

В D8421+3 используется опять же 10 двоичных комбинаций из 16, но не используются 3 начальные комбинации и 3 конечные. D8421+3 есть код с избытком 3, при сложении двух чисел имеем $(A + 3) + (B + 3) = (A + B + 3) + 3$, то есть одна 3 лишняя, поэтому если переноса нет – корректируем на –3, если перенос возникает, то необходимо скорректировать эту секцию на +3 для обеспечения необходимого избытка.

Слово коррекции не содержит нулевых секций, но зато не возникает необходимость в повторной коррекции. Сложение двух чисел A и B в D8421+3 иллюстрирует пример 28, сложение выполняется побитно, для наглядности тетрады разделены пробелами.

Пример 28.

A=0100 1011 1001	A=186
B= <u>0110 1100 1000</u>	B= <u>395</u>
1011_1000_0001	C=581
<u>1101 0011 0011</u>	
C=1000 1011 0100	

В примере 28 из двух младших тетрад возникает перенос, поэтому к ним прибавляется коррекция 0011, из старшей секции переноса не возникает, поэтому коррекция 1101, цепи межтетрадного переноса при коррекции разомкнуты, что иллюстрируется разрывом суммирующей черты.

Повторная коррекция в D8421+3 отсутствует, что является дополнительным преимуществом данного кода.

Поскольку D8421+3 является самодополняемым, то для получения дополнительного кода не требуется коррекции. В примере 29 иллюстрируется получение дополнительного кода в D8421+3,

Пример 29.

A = 0011. 0100 0101 0110	A = 0.123
-A = 1100. 1011 1010 1010	-A = 9.877

В примере 29 дополнительный код исходного числа A находится по стандартным правилам двоичной арифметики и не требует коррекции, этот момент является главным преимуществом D8421+3.

В примере 30 иллюстрируется вычитание в D8421+3.

Пример 30. C=B-A

B = 0011. 1000 0100 1010	B = 0.517	B = 0.517
-A = <u>1100. 1011 1010 1010</u>	-A = <u>-0.123</u>	-A = <u>9.877</u>
<u> 0000. 0011 1111 0100</u>	C = 0.394	C = <u>0.394</u>
<u> 0011. 0011 1101 0011</u>		
C = 0011. <u>0110 1100 0111</u>		

В примере 30 -A подставляется сразу в дополнительном коде, после суммирования выполняется коррекция во всех тетрадах (как и всегда в D8421+3), цепи межатетрадного переноса разомкнуты.

Вопросы и задания

1. Что такое двоично-десятичные коды?
2. Для какого типа задач удобно использовать двоично-десятичные коды?
3. Что означает самодополняемость двоично-десятичного кода?
4. Укажите преимущества и недостатки кода D8421.
5. Укажите преимущества и недостатки кода D8421+3.

3.2. УМНОЖЕНИЕ В ДВОИЧНО-ДЕСЯТИЧНОМ КОДЕ

Как и в устройствах умножения двоичной арифметики, здесь применяются две базовые схемы: младшими разрядами вперед и старшими разрядами вперед, только в двоично-десятичных устройствах обработка идет секциями (по 4 разряда) и в структуры устройств включается дополнительно 4-разрядный счетчик.

3.2.1. Умножение в двоично-десятичном коде младшей секцией вперед

Структура устройства умножения в двоично-десятичном коде младшей секцией вперед представлена на рис. 14. Сдвиги выполняются на всю десятичную секцию (4 разряда).

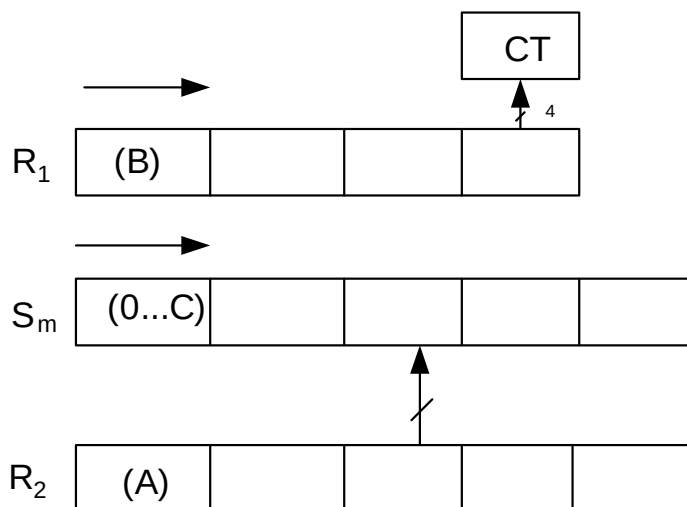


Рис. 14. Устройство умножения младшей секцией вперед

При выполнении операции умножения $C = A \cdot B$ операнд A загружается в регистр R_2 , операнд B – в R_1 , сумматор S_m обнуляется. R_2 и S_m содержат дополнительные секции для снижения погрешности. В структуре используется дополнительный счетчик CT (не путать со счетчиком циклов, счетчик циклов на рис. 14 не показан), который используется для загрузки в него младшей секции множителя и подсчета числа сложений в цикле.

В каждом цикле:

1. Содержимое младшей секции R_1 загружается в счетчик СТ.

2. Далее к сумматору Sm прибавляется содержимое R_2 (и при необходимости выполняется коррекция), после каждого такого сложения содержимое счетчика СТ уменьшается на 1, и так до тех пор, пока содержимое СТ не станет равным 0.

3. В конце цикла выполняются сдвиги сумматора Sm и регистра R_1 на одну секцию вправо (в сторону младших разрядов).

Число циклов – число десятичных знаков множителя (B) после десятичной точки (в нашем случае 3).

В конце рекомендуется сделать округление по стандартным правилам десятичной арифметики с использованием дополнительных разрядов.

Замечание: в одном цикле может быть несколько сложений, но сдвиги выполняются только один раз в конце цикла.

Пример 31 иллюстрирует пошаговое выполнение умножения двух десятичных чисел $A = 0,862$ и $B = 0,312$ в D8421, в правом столбце производится проверка всех действий в десятичной системе.

Пример 31.

0	$Sm=0000,0000\ 0000\ 0000\ 0000\ 0000$ $R_2=0000,1000\ 0110\ 0010\ 0000$	$R_1=0000,0011\ 0001\ \underline{0010}$ $CT=0010$	$Sm=0,000\ 0$ $R_2=0,862\ 0$
1	$Sm=0000,0000\ 0000\ 0000\ 0000\ 0000$ $R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0000,1000\ 0110\ 0010\ 0000$	$CT=0010$ $CT=0001$	$Sm=0,000\ 0$ $R_2=0,862\ 0$ $Sm=0,862\ 0$
	$R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0001,0000\ 1100\ 0100\ 0000$ $Кор\ \underline{\hspace{1cm}},0110\ 0110\ \hspace{1cm}\ $ $Sm=0001,0111\ 0010\ 0100\ 0000$	$CT=0000\ (\text{конец цикла})$ $R_1=0000,0000\ 0011\ \underline{0001}$	$Sm=0,862\ 0$ $R_2=0,862\ 0$ $Sm=1,724\ 0$
	$Сдвиги:$ $SM=0000,0001\ 0111\ 0010\ 0100$		
			$Sm=0,172\ 4$

2	$Sm=0000,0001\ 0111\ 0010\ 0100$ $R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0000,1001\ 1101\ 0100\ 0100$ $Кор\ \underline{\hspace{1cm}},\ 0110\ \hspace{1cm} \hspace{1cm}=\hspace{1cm}$ $Sm=0000,1010\ 0011\ 0100\ 0100$ $Кор\ \underline{\hspace{1cm}},\ 0110\ \hspace{1cm} \hspace{1cm}=\hspace{1cm}$ $Sm=0001,0000\ 0011\ 0100\ 0100$ Сдвиги: $Sm=0000,0001\ 0000\ 0011\ 0100$	$CT=0001$ $CT=0000$ (конец цикла) $R_1=0000,0000\ 0000\ \underline{0011}$	$Sm=0,172\ 4$ $R_2=0,862\ 0$ $Sm=1,034\ 4$ $Sm=0,103\ 4$
	$Sm=0000,0001\ 0000\ 0011\ 0100$ $R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0000,1001\ 0110\ 0101\ 0100$ $R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0001,0001\ 1100\ 0111\ 0100$ $Кор\ \underline{\hspace{1cm}},\ 0110\ 0110\ \hspace{1cm} \hspace{1cm}=\hspace{1cm}$ $R_2=0001,1000\ 0010\ 0111\ 0100$ $R_2=0000,1000\ 0110\ 0010\ 0000$ $Sm=0010,0000\ 1000\ 1001\ 0100$ $Кор\ \underline{\hspace{1cm}},\ 0110\ \hspace{1cm} \hspace{1cm}=\hspace{1cm}$ $Sm=0010,0110\ 1000\ 1001\ 0100$ Сдвиги: $Sm=0000,0010\ 0110\ 1000\ 1001$	$CT=0011$ $CT=0010$ $CT=0001$ $CT=0000$ (конец цикла) $R_1=0000,0000\ 0000\ 0000$	$Sm=0,103\ 4$ $R_2=0,862\ 0$ $Sm=0,965\ 4$ $Sm=0,965\ 4$ $R_2=0,862\ 0$ $Sm=1,827\ 4$ $Sm=1,827\ 4$ $R_2=0,862\ 0$ $Sm=2,689\ 4$ $Sm=0,268\ 9(4)$

Результат, полученный по правилам округления: **0,269**.

В примере 31 в первом цикле выполняются 2 сложения, во втором – 1 сложение, в третьем – 3 сложения (в соответствии со значениями секций множителя, начиная с младшей). Во втором цикле выполняется дополнительная коррекция.

Пример 32 иллюстрирует пошаговое выполнение умножения двух десятичных чисел $A = 0,862$ и $B = 0,312$ в D8421+3, в правом столбце производится проверка всех действий в десятичной системе.

Напомним правила коррекции в двоично-десятичном коде D8421+3: при наличии переноса из тетрады к ней прибавляется 0011 (+3), при отсутствии – прибавляется 1101 (–3), цепи межтетрадного переноса при коррекции (только при коррекции) должны быть разорваны.

Пример 32.

0	$S_m=0011, 0011 \ 0011 \ 0011 \ 0011$ $R_2=0011, 1011 \ 1001 \ 0101 \ 0011$	$R_1=0011, 0110 \ 0100 \ 0101$ $CT=2$	$S_m=0, 000 \mid 0$ $R_2=0, \underline{862} \mid 0$
1	$S_m=0011, 0011 \ 0011 \ 0011 \ 0011$ $R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=0110, 1110 \ 1100 \ 1000 \ 0110$ $Кор\ 1101, 1101 \ 1101 \ 1101 \ 1101$ $S_m=0011, 1011 \ 1001 \ 0101 \ 0011$	$CT=2$ (прибавление 0 тоже требует коррекции) $CT=1$	$S_m=0, 000 \mid 0$ $R_2=0, \underline{862} \mid 0$ $S_m=0, 862 \mid 0$
	$R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=0111, 0111 \ 0010 \ 1010 \ 0110$ $Кор\ 1101, 0011 \ 0011 \ 1101 \ 1101$ $S_m=0100, 1010 \ 0101 \ 0111 \ 0011$ Сдвиги: $S_m=0011, 0100 \ 1010 \ 0101 \ 0111$	$CT=0$ (конец цикла) $R_1=0011, 0011 \ 0110 \ 0100$	$S_m=0, 862 \mid 0$ $R_2=0, \underline{862} \mid 0$ $S_m=1, 724 \mid 0$ $S_m=0, 172 \mid 4$
2	$S_m=0011, 0100 \ 1010 \ 0101 \ 0111$ $R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=0111, 0000 \ 0011 \ 1010 \ 1010$ $Кор\ 1101, 0011 \ 0011 \ 1101 \ 1101$ $S_m=0100, 0011 \ 0110 \ 0111 \ 0111$ Сдвиги: $S_m=0011, 0100 \ 0011 \ 0110 \ 0111$	$CT=1$ $CT=0$ (конец цикла) $R_1=0011, 0011 \ 0011 \ 0110$	$S_m=0, 172 \mid 4$ $R_2=0, \underline{862} \mid 0$ $SM=1, 034 \mid 4$ $SM=0, 103 \mid 4$
	$S_m=0011, 0100 \ 0011 \ 0110 \ 0111$ $R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=0110, 1111 \ 1100 \ 1011 \ 1010$ $Кор\ 1101, 1101 \ 1101 \ 1101 \ 1101$ $S_m=0011, 1100 \ 1001 \ 1000 \ 0111$	$CT=3$ $CT=2$	$SM=0, 103 \mid 4$ $R_2=0, \underline{862} \mid 0$ $SM=0, 965 \mid 4$
3	$R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=0111, 1000 \ 0010 \ 1101 \ 1010$ $Кор\ 1101, 0011 \ 0011 \ 1101 \ 1101$ $S_m=0100, 1011 \ 0101 \ 1010 \ 0111$	$CT=1$	$SM=0, 965 \mid 4$ $R_2=0, \underline{862} \mid 0$ $SM=1, 827 \mid 4$
	$R_2=0011, 1011 \ 1001 \ 0101 \ 0011$ $S_m=1000, 0110 \ 1110 \ 1111 \ 1010$ $Кор\ 1101, 0011 \ 1101 \ 1101 \ 1101$ $S_m=0101, 1001 \ 1011 \ 1100 \ 0111$ Сдвиги: $S_m=0011, 0101 \ 1001 \ 1011 \ 1100$	$CT=0$ (конец цикла) $R_1=0011, 0011 \ 0011 \ 0011$	$SM=1, 827 \mid 4$ $R_2=0, \underline{862} \mid 0$ $SM=2, 689 \mid 4$
	$S_m=0011, 0101 \ 1001 \ 1011 \ 1100$		$SM=0, 268 \mid 9(4)$

Результат, полученный по правилам округления: **0,269**.

В примере 32 для упрощения содержимое вспомогательного счетчика приводится в десятичной системе. Обратите внимание, что в D8421+3 сложение с нулевым значением представляет полноценную операцию с коррекцией.

3.2.2. Умножение в двоично-десятичном коде старшей секцией вперед

Структура устройства умножения в двоично-десятичном коде старшей секцией вперед представлена на рис. 15. Сдвиги выполняются на всю десятичную секцию (4 разряда).

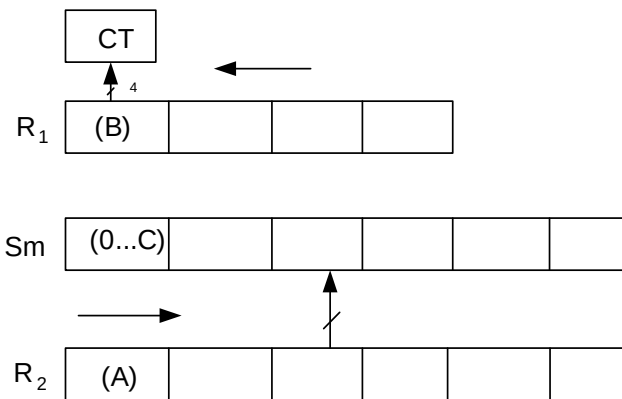


Рис. 15. Умножение старшей секцией вперед

При выполнении операции умножения $C = A \cdot B$ операнд A загружается в регистр R₂, операнд B – в R₁, сумматор Sm обнуляется. R₂ и Sm содержат дополнительные секции для снижения погрешности. В структуре используется дополнительный счетчик СТ (не путать со счетчиком циклов, счетчик циклов на рис. 15 не показан), который используется для загрузки в него старшей секции множителя и подсчета числа сложений в цикле.

В каждом цикле:

1. Выполняются сдвиги регистра R₂ вправо и регистра R₁ влево на одну секцию.
2. Содержимое старшей секции R₁ загружается в счетчик СТ.

3. Далее к сумматору S_m прибавляется содержимое R_2 (и при необходимости выполняется коррекция), после каждого такого сложения содержимое счетчика СТ уменьшается на 1, и так до тех пор, пока содержимое СТ не станет равным 0.

Число циклов – число десятичных знаков множителя (В) после десятичной точки (в нашем случае 3).

В конце рекомендуется сделать округление по стандартным правилам десятичной арифметики с использованием дополнительных разрядов.

Замечание: в одном цикле может быть несколько сложений, но сдвиги выполняются только один раз в начале цикла.

Пример 33 иллюстрирует пошаговое выполнение умножения старшими разрядами вперед двух десятичных чисел $A = 0,862$ и $B = 0,312$ в D8421, в правом столбце производится проверка всех действий в десятичной системе.

Пример 33.

0	$S_m = 0000, 0000 \ 0000 \ 0000 \ 0000 \ 0000$ $R_2 = 0000, 1000 \ 0110 \ 0010 \ 0000 \ 0000$	$R_1 = 0000, 0011 \ 0001 \ 0010$	$S_m = 0, 000 \ 00$ $R_2 = 0, 862 \ 00$
1	Сдвиги: $R_2 = 0000, 0000 \ 1000 \ 0110 \ 0010 \ 0000$ $S_m = 0000, 0000 \ 0000 \ 0000 \ 0000 \ 0000$ $S_m = 0000, 0000 \ 1000 \ 0110 \ 0010 \ 0000$	$R_1 = 0011, 0001 \ 0010 \ 0000$ $CT = 0011$ $CT = 0010$	$S_m = 0, 000 \ 00$ $R_2 = 0, 086 \ 20$ $S_m = 0, 086 \ 20$
	$R_2 = 0000, 0000 \ 1000 \ 0110 \ 0010 \ 0000$ $S_m = 0000, 0001 \ 0000 \ 1100 \ 0100 \ 0000$ Кор $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$ $0110 \ 0110$ $\underline{\hspace{1cm}}$ $S_m = 0000, 0001 \ 0111 \ 0010 \ 0100 \ 0000$	$CT = 0001$	$S_m = 0, 086 \ 20$ $R_2 = 0, 086 \ 20$ $S_m = 0, 172 \ 40$
	$R_2 = 0000, 0000 \ 1000 \ 0110 \ 0010 \ 0000$ $S_m = 0000, 0001 \ 1111 \ 1000 \ 0110 \ 0000$ Кор $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$ 0110 $\underline{\hspace{1cm}}$ $S_m = 0000, 0010 \ 0101 \ 1000 \ 0110 \ 0000$	$CT = 0000$ (конец цикла)	$S_m = 0, 172 \ 40$ $R_2 = 0, 086 \ 20$ $S_m = 0, 258 \ 60$
2	Сдвиги: $R_2 = 0000, 0000 \ 0000 \ 1000 \ 0110 \ 0010$ $S_m = 0000, 0010 \ 0101 \ 1000 \ 0110 \ 0000$ $S_m = 0000, 0010 \ 0110 \ 0000 \ 1100 \ 0010$ Кор $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$ $0110 \ 0110$ $\underline{\hspace{1cm}}$ $S_m = 0000, 0010 \ 0110 \ 0111 \ 0010 \ 0010$	$R_1 = 0001, 0010 \ 0000 \ 0000$ $CT = 0001$ $CT = 0000$ (конец цикла)	$R_2 = 0, 008 \ 62$ $S_m = 0, 258 \ 60$ $S_m = 0, 267 \ 22$

3	Сдвиги: $R_2 = 0000,0000 \ 0000 \ 0000 \mid 1000 \ 0110$ $Sm = 0000,0010 \ 0110 \ 0111 \mid 0010 \ 0010$ $Sm = 0000,0010 \ 0110 \ 0111 \mid 1010 \ 1000$ Кор $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$ $\mid 0110 \ \underline{\hspace{1cm}}$ $Sm = 0000,0010 \ 0110 \ 1000 \mid 0000 \ 1000$	$R_1 = 0010,0000 \ 0000 \ 0000$ CT=0010 CT=0001	$R_2 = 0,000 \mid 86$ $Sm = 0,267 \mid 22$ $Sm = 0,268 \mid 08$
	$R_2 = 0000,0000 \ 0000 \ 0000 \mid 1000 \ 0110$ $Sm = 0000,0010 \ 0110 \ 1000 \mid 1000 \ 1110$ Кор $\underline{\hspace{1cm}}$, $\underline{\hspace{1cm}}$ $\mid \hspace{1cm} 0110$ $Sm = \mathbf{0000,0010 \ 0110 \ 1000} \mid 1001 \ 0100$	CT=0000 (конец цикла)	$Sm = 0,268 \mid 08$ $R_2 = 0,000 \mid 86$ $Sm = \mathbf{0,268} \mid 94$

Результат, полученный по правилам округления: **0,269**.

Пример 34 иллюстрирует пошаговое выполнение умножения старшими разрядами вперед двух десятичных чисел $A = 0,862$ и $B = 0,312$ в D8421+3, в правом столбце производится проверка всех действий в десятичной системе, для упрощения записи содержимое счетчика также представлено в десятичной системе.

Пример 34.

0	$Sm = 0011,0011 \ 0011 \ 0011 \mid 0011 \ 0011$ $R_2 = 0011,1011 \ 1001 \ 0101 \mid 0011 \ 0011$	$R_1 = 0011,0110 \ 0100 \ 0101$	$Sm = 0,000 \mid 00$ $R_2 = 0,862 \mid 00$
1	Сдвиги: $R_2 = 0011,0011 \ 1011 \ 1001 \mid 0101 \ 0011$ $Sm = 0011,0011 \ 0011 \ 0011 \mid 0011 \ 0011$ $Sm = 0110,0110 \ 1110 \ 1100 \mid 1000 \ 0110$ Кор $\underline{1101,1101 \ 1101 \ 1101} \mid 1101 \ 1101$ $Sm = 0011,0011 \ 1011 \ 1001 \mid 0101 \ 0011$	$R_1 = 0110,0100 \ 0101 \ 0011$ CT=3 CT=2	$Sm = 0,000 \mid 00$ $R_2 = 0,086 \mid 20$ $Sm = 0,086 \mid 20$
	$R_2 = 0011,0011 \ 1011 \ 1001 \mid 0101 \ 0011$ $Sm = 0110,0111 \ 0111 \ 0010 \mid 1010 \ 0110$ Кор $\underline{1101,1101 \ 0011 \ 0011} \mid 1101 \ 1101$ $Sm = 0011,0100 \ 1010 \ 0101 \mid 0111 \ 0011$	(R₂ не сдвигаем) CT=1	$Sm = 0,086 \mid 20$ $R_2 = 0,086 \mid 20$ $Sm = 0,172 \mid 40$
	$R_2 = 0011,0011 \ 1011 \ 1001 \mid 0101 \ 0011$ $Sm = 0110,1000 \ 0101 \ 1110 \mid 1100 \ 0110$ Кор $\underline{1101,1101 \ 0011 \ 1101} \mid 1101 \ 1101$ $Sm = 0011,0101 \ 1000 \ 1011 \mid 1001 \ 0011$	CT=0 (конец цикла)	$Sm = 0,172 \mid 40$ $R_2 = 0,086 \mid 20$ $Sm = 0,258 \mid 60$
	Сдвиги: $R_2 = 0011,0011 \ 0011 \ 1011 \mid 1001 \ 0101$ $Sm = 0011,0101 \ 1000 \ 1011 \mid 1001 \ 0011$ $Sm = 0110,1000 \ 1100 \ 0111 \mid 0010 \ 1000$ Кор $\underline{1101,1101 \ 1101 \ 0011} \mid 0011 \ 1101$ $Sm = 0011,0101 \ 1001 \ 1010 \mid 0101 \ 0101$	$R_1 = 0100,0101 \ 0011 \ 0011$ CT=1 CT=0 (конец цикла)	$R_2 = 0,008 \mid 62$ $Sm = 0,258 \mid 60$ $Sm = 0,267 \mid 22$

3	Сдвиги: $R_2 = 0011, 0011 \ 0011 \ 0011 \mid 1011 \ 1001$ $Sm = 0011, 0101 \ 1001 \ 1010 \mid 0101 \ 0101$ $Sm = 0110, 1000 \ 1100 \ 1110 \mid 0000 \ 1110$ $Kop \ 1101, 1101 \ 1101 \ 1101 \mid 0011 \ 1101$ $Sm = 0011, 0101 \ 1001 \ 1011 \mid 0011 \ 1011$	$R_1 = 0101, 0011 \ 0011 \ 0011$ CT=2 CT=1	$R_2 = 0, 000 \mid 86$ $Sm = 0, 267 \mid 22$ $Sm = 0, 268 \mid 08$
	$R_2 = 0011, 0011 \ 0011 \ 0011 \mid 1011 \ 1001$ $Sm = 0110, 1000 \ 1100 \ 1110 \mid 1111 \ 0100$ $Kop \ 1101, 1101 \ 1101 \ 1101 \mid 1101 \ 0011$ $Sm = 0011, 0101 \ 1001 \ 1011 \mid 1100 \ 0111$	CT=0 (конец цикла)	$Sm = 0, 268 \mid 08 \ R_2$ $= 0, 000 \mid 86$ $Sm = 0, 268 \mid 94$

Результат, полученный по правилам округления: **0,269**.

Согласно рассмотренным примерам число сложений в цикле зависит от соответствующей секции делителя, то есть при умножении на $B = 0,999$ потребовалось бы в каждом цикле выполнить 9 операций сложения. С целью ускорения выполнения операции умножения в двоично-десятичных кодах используются заранее заготовленные значения $2A$ и $4A$, в табл. 7 приводятся выполняемые операции при всех возможных значениях секции множителя.

Таблица 7

Ускорение умножения

в двоично-десятичных кодах

Секция В	Операции
0	NOP
1	+A
2	+2A
3	+2A+A
4	+4A
5	+4A+A
6	+4A+2A
7	+4A+2A+A
8	+4A+4A
9	+4A+4A+A

Из табл. 7 видно, что при использовании ускоренного метода максимальное число сложений в каждом цикле не превышает трех.

Вопросы и задания

1. Умножить указанными методами два ненулевых десятичных числа в D8421 с тремя десятичными знаками после запятой (типа 0.973 и 0.432), сумма цифр множителя (В) должна быть не менее 7, на каждом шаге выполнять десятичную проверку:

- а) младшими разрядами вперед;
- б) старшими разрядами вперед.

2. Умножить указанными методами два ненулевых десятичных числа в D8421+3 с тремя десятичными знаками после запятой (типа 0.973 и 0.432), сумма цифр множителя (В) должна быть не менее 7, на каждом шаге выполнять десятичную проверку:

- а) младшими разрядами вперед;
- б) старшими разрядами вперед.

Для заданий выполнить общую проверку на микрокалькуляторе.

3.3. ДЕЛЕНИЕ В ДВОИЧНО-ДЕСЯТИЧНОМ КОДЕ

Как и в устройствах деления двоичной арифметики, здесь применяются базовые схемы: с восстановлением остатка, без восстановления остатка, со сдвигом сумматора, с неподвижным сумматором только в двоично-десятичных устройствах обработка идет секциями (по 4 разряда) и в структуре устройств включается дополнительно 4-разрядный счетчик.

3.3.1. Деление в двоично-десятичном коде со сдвигом сумматора

Структура устройства деления в двоично-десятичном коде со сдвигом сумматора представлена на рис. 16. Сдвиги выполняются на всю десятичную секцию (4 разряда).

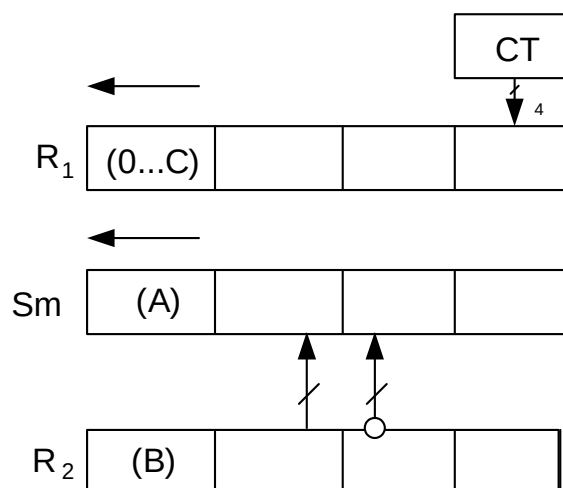


Рис. 16. Структура устройства деления
со сдвигом сумматора

Рассмотрим сначала алгоритм с восстановлением остатка.

При выполнении операции деления $C = A / B$ операнд A загружается в сумматор Sm , операнд B – в регистр R_2 , R_1 обнуляется (по окончании выполнения операции здесь будет находиться результат C). Дополнительные секции не используются. В структуре используется дополнительный счетчик $СТ$ (не путать со счетчиком циклов, счетчик циклов на рис. 16 не показан), который используется для нахождения очередной цифры частного, которая загружается в младшую секцию регистра R_1 .

Перед вычислением производится проверка $|A| < |B|$, если условие не соблюдается, производится деморализация A (на целую секцию).

В каждом цикле:

1. Выполняются сдвиги сумматора Sm и регистра R_1 влево и на одну секцию, счетчик устанавливается на ноль ($СТ = 0$).

2. Из сумматора Sm вычитается содержимое R_2 (для этого находится $-R_2$ в дополнительном коде, данная операция требует специальной коррекции), после каждого прибавления $-R_2$, и при необходимости выполнения коррекции содержимое счетчика $СТ$ увеличивается на 1, и так до тех пор, пока содержимое сумматора не станет меньше 0 (первый бит сумматора станет равным 1). После этого сумматор Sm восстанавливается путем прибавления к нему содержимого регистра R_2 .

3. В конце цикла содержимое счетчика СТ передается в младшую секцию регистра R₁, так находится очередная цифра частного.

Число циклов – число десятичных знаков находимого результата (С) после десятичной точки (в нашем случае 3).

Замечание: в одном цикле может быть несколько вычитаний, но сдвиги выполняются только один раз в начале цикла.

Пример 35 иллюстрирует пошаговое выполнение деления двух десятичных чисел A = 0,291 и B = 0,829 в D8421 методом с восстановлением остатка на структуре со сдвигом сумматора, в правом столбце производится проверка всех действий в десятичной системе.

Пример 35.

0	$\begin{array}{r} \text{Sm}=0000,0010 \ 1001 \ 0001 \\ \text{R}_2=0000,1000 \ 0010 \ 1001 \\ \text{ДК}=1111,0111 \ 1101 \ 0111 \\ \text{Кор}=\underline{1010,1010 \ 1010 \ 1010} \\ -\text{R}_2=\underline{1001,0001 \ 0111 \ 0001} \end{array}$	$\begin{array}{l} \text{R}_1=0000,0000 \ 0000 \ 0000 \\ \text{(при данной коррекции} \\ \text{межтетрадные цепи пе-} \\ \text{реносов разорваны)} \end{array}$	$\begin{array}{l} \text{Sm}=0,291 \\ \text{R}_2=0,829 \\ -\text{R}_2=\underline{9,171} \end{array}$
1	Сдвиги: $\begin{array}{r} \text{Sm}=0010,1001 \ 0001 \ 0000 \\ -\text{R}_2=\underline{1001,0001 \ 0111 \ 0001} \\ \text{Sm}=\underline{1011,1010 \ 1000 \ 0001} \\ \text{Кор} \ \underline{0110,0110} = \\ \text{Sm}=0010,0000 \ 1000 \ 0001 \end{array}$	$\begin{array}{l} \text{R}_1=0000,0000 \ 0000 \ 0000 \\ \text{СТ}=0 \\ \text{(при данной коррекции} \\ \text{и далее межтетрадные} \\ \text{цепи переносов соеди-} \\ \text{нены)} \\ \text{СТ}=1 \end{array}$	$\begin{array}{l} \text{Sm}=2,910 \\ -\text{R}_2=\underline{9,171} \\ \text{Sm}=\underline{2,081} \end{array}$
	$\begin{array}{r} -\text{R}_2=\underline{1001,0001 \ 0111 \ 0001} \\ \text{Sm}=\underline{1011,0001 \ 1111 \ 0010} \\ \text{Кор}=\underline{0110, \quad 0110} = \\ \text{Sm}=\underline{0001,0010 \ 0101 \ 0010} \end{array}$	СТ=2	$\begin{array}{l} \text{Sm}=2,081 \\ -\text{R}_2=\underline{9,171} \\ \text{Sm}=\underline{1,252} \end{array}$
	$\begin{array}{r} -\text{R}_2=\underline{1001,0001 \ 0111 \ 0001} \\ \text{Sm}=\underline{1010,0011 \ 1100 \ 0011} \\ \text{Кор}=\underline{0110, \quad 0110} = \\ \text{SM}=\underline{0000,0100 \ 0010 \ 0011} \\ (*) \end{array}$	СТ=3	$\begin{array}{l} \text{Sm}=1,252 \\ -\text{R}_2=\underline{9,171} \\ \text{Sm}=\underline{0,423} \end{array}$
	$\begin{array}{r} -\text{R}_2=\underline{1001,0001 \ 0111 \ 0001} \\ \text{Sm}=\underline{1001,0101 \ 1001 \ 0100} \\ +\text{R}_2=\underline{0000,1000 \ 0010 \ 1001} \\ \text{Sm}=\underline{1001,1101 \ 1011 \ 1101} \\ \text{Кор}=\underline{\quad,0110 \ 0110 \ 0110} \\ \text{Sm}=\underline{1010,0100 \ 0010 \ 0011} \\ \text{Кор}=\underline{0110, \quad \quad \quad} = \\ \text{Sm}=\underline{0000,0100 \ 0010 \ 0011} \\ (*) \end{array}$	$\begin{array}{l} \text{(Sm}<0, \text{ конец цикла)} \\ \text{R}_1=0000,0000 \ 0000 \ \underline{0011} \end{array}$	$\begin{array}{l} \text{Sm}=0,423 \ (*) \\ -\text{R}_2=\underline{9,171} \\ \text{Sm}=\underline{9,594}_{(\text{SM}<0)} \\ +\text{R}_2=\underline{0,829} \\ \text{Sm}=\underline{0,423} \ (*) \end{array}$

2	Сдвиги: $\begin{array}{r} Sm=0100,0010 \ 0011 \ 0000 \\ -R_2=1001,0001 \ 0111 \ 0001 \\ \hline Sm=1101,0011 \ 1010 \ 0001 \\ \text{Кор}=0110, \quad 0110 \quad = \\ Sm=0011,0100 \ 0000 \ 0001 \end{array}$	$R_1=0000,0000 \ 0011 \ 0000$ $CT=0$	$Sm=4,230$ $-R_2=9,171$ $Sm=3,401$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1100,0101 \ 0111 \ 0010 \\ \text{Кор}=0110, \quad = \\ Sm=0010,0101 \ 0111 \ 0010 \end{array}$	$CT=2$	$Sm=3,401$ $-R_2=9,171$ $Sm=2,572$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1011,0110 \ 1110 \ 0011 \\ \text{Кор}=0110, \quad 0110 \quad = \\ Sm=0001,0111 \ 0100 \ 0011 \end{array}$	$CT=3$	$Sm=2,572$ $-R_2=9,171$ $Sm=1,743$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1010,1000 \ 1011 \ 0100 \\ \text{Кор}=0110, \quad 0110 \quad = \\ Sm=0000,1001 \ 0001 \ 0100 \end{array}$	$CT=4$	$Sm=1,743$ $-R_2=9,171$ $Sm=0,914$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1001,1010 \ 1000 \ 0101 \\ \text{Кор}= \quad , 0110 \quad = \\ Sm=1010,0000 \ 1000 \ 0101 \\ \text{Кор}=0110, \quad = \\ Sm=0000,0000 \ 1000 \ 0101 \\ (*) \end{array}$	$CT=5$	$Sm=0,914$ $-R_2=9,171$ $Sm=0,085$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1001,0001 \ 1111 \ 0110 \\ \text{Кор}= \quad , \quad 0110 \quad = \\ Sm=1001,0010 \ 0101 \ 0110 \\ \\ Sm=1001,0010 \ 0101 \ 0110 \\ +R_2=0000,1000 \ 0010 \ 1001 \\ Sm=1001,1010 \ 0111 \ 1111 \\ \text{Кор}= \quad , 0110 \quad 0110 \\ Sm=1010,0000 \ 1000 \ 0101 \\ \text{Кор}=0110, \quad = \\ Sm=0000,0000 \ 1000 \ 0101 \\ (*) \end{array}$	$(Sm < 0, \text{конец цикла})$ $R_1=0000,0000 \ 0011 \ 0101$	$Sm=0,085 \ (*)$ $-R_2=9,171$ $Sm=9,256_{(SM<0)}$ $Sm=9,256_{(SM<0)}$ $+R_2=0,829$ $Sm=0,085 \ (*)$
3	Сдвиги: $\begin{array}{r} Sm=0000,1000 \ 0101 \ 0000 \\ -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1001,1001 \ 1100 \ 0001 \\ \text{Кор}= \quad , \quad 0110 \quad = \\ Sm=1001,1010 \ 0010 \ 0001 \\ \text{Кор}= \quad , 0110 \quad = \\ Sm=1010,0000 \ 0010 \ 0001 \\ \text{Кор}=0110, \quad = \\ Sm=0000,0000 \ 0010 \ 0001 \\ (*) \end{array}$	$R_1=0000,0011 \ 0101 \ 0000$ $CT=1$	$Sm=0,850$ $-R_2=9,171$ $Sm=0,021$
	$\begin{array}{r} -R_2=1001,0001 \ 0111 \ 0001 \\ Sm=1001,0001 \ 1001 \ 0010 \\ +R_2=0000,1000 \ 0010 \ 1001 \end{array}$	$(Sm < 0, \text{конец цикла})$ $R_1=0000,0011 \ 0101 \ 0001$	$SM=0,021 \ (*)$ $-R_2=9,171$

$ \begin{array}{r} Sm=1001,1001\ 1011\ 1011 \\ \text{Кор}=\underline{\hspace{1cm}},\ 0110\ 0110 \\ Sm=1001,1010\ 0010\ 0001 \\ \text{Кор}=\underline{\hspace{1cm}},\ 0110 \\ Sm=1010,0000\ 0010\ 0001 \\ \text{Кор}=\underline{0110},\hspace{1cm} \\ Sm=0000,0000\ 0010\ 0001 \\ (*) \end{array} $		$ \begin{array}{r} SM=9,192\ (SM<0) \\ +R_2=\underline{0,829} \\ SM=0,021\ (*) \\ R_1=\underline{0,351} \end{array} $
--	--	--

Результат : **0,351.**

В примере 35 сначала выполняется перевод R_2 в дополнительный код и соответствующая коррекция. В конце каждого цикла выполняется восстановления остатка $Sm = SM + R_2$, в начале каждого цикла частичный остаток положителен и счетчик $CT = 0$. Обратите внимание на то, что в некоторых циклах довольно много дополнительных коррекций.

Пример 36 иллюстрирует пошаговое выполнение деления двух десятичных чисел $A = 0,291$ и $B = 0,829$ в D8421+3 методом с восстановлением остатка на структуре со сдвигом сумматора, в правом столбце производится проверка всех действий в десятичной системе. Содержимое счетчика приводится в десятичной системе. Напомним правила коррекции в двоично-десятичном коде D8421+3: при наличии переноса из тетрады к ней прибавляется 0011 (+3), при отсутствии – прибавляется 1101 (–3), цепи межтетрадного переноса при коррекции (только при коррекции) должны быть разорваны.

Пример 36.

0	$ \begin{array}{r} Sm=0011,0101\ 1100\ 0100 \\ R_2=0011,1011\ 0101\ 1100 \\ -R_2=1100,0100\ 1010\ 0100 \end{array} $	$ \begin{array}{r} R_1=0011,0011\ 0011\ 0011 \\ (R_1=0,000) \end{array} $	$ \begin{array}{r} Sm=0,291 \\ -R_2=\underline{0,829} \\ -R_2=9,171 \end{array} $
1	Сдвиги: $ \begin{array}{r} Sm=0101,1100\ 0100\ 0011 \\ -R_2=\underline{1100,0100\ 1010\ 0100} \\ Sm=0010,0000\ 1110\ 0111 \\ \text{Кор}\ \underline{0011,0011\ 1101\ 1101} \\ Sm=0101,0011\ 1011\ 0100 \end{array} $	$ \begin{array}{r} R_1=0011,0011\ 0011\ 0011 \\ CT=0 \\ \text{(при данной коррекции} \\ \text{и далее межтетрадные} \\ \text{цепи разорваны)} \\ CT=1 \end{array} $	$ \begin{array}{r} Sm=2,910 \\ -R_2=\underline{9,171} \\ Sm=2,081 \end{array} $

	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0001, 1000 \ 0101 \ 1000$ $Kop \ 0011, 1101 \ 0011 \ 1101$ $Sm = 0100, 0101 \ 1000 \ 0101$	CT=2	$Sm = 2, 081$ $-R_2 = 9, 171$ $Sm = 1, 252$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0000, 1010 \ 0010 \ 1001$ $Kop \ 0011, 1101 \ 0011 \ 1101$ $Sm = 0011, 0111 \ 0101 \ 0110$ (*)	CT=3	$Sm = 1, 252$ $-R_2 = 9, 171$ $Sm = 0, 423$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 1111, 1011 \ 1111 \ 1010$ $Kop \ 1101, 1101 \ 1101 \ 1101$ $Sm = 1100, 1000 \ 1100 \ 0111$ $+R_2 = 0011, 1011 \ 0101 \ 1100$ $Sm = 0000, 0100 \ 0010 \ 0011$ $Kop \ 0011, 0011 \ 0011 \ 0011$ $Sm = 0011, 0111 \ 0101 \ 0110$ (*)	(Sm<0, конец цикла) $R_1 = 0011, 0011 \ 0011 \ 0110$ (R ₁ =0, 003)	$Sm = 0, 423 \ (*)$ $-R_2 = 9, 171$ $Sm = 9, 594_{(Sm<0)}$ $+R_2 = 0, 829$ $Sm = 0, 423 \ (*)$
2	Сдвиги: $Sm = 0111, 0101 \ 0110 \ 0011$ $-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0011, 1010 \ 0000 \ 0111$ $Kop \ 0011, 1101 \ 0011 \ 1101$ $Sm = 0110, 0111 \ 0011 \ 0100$	$R_1 = 0011, 0011 \ 0110 \ 0011$ (R ₁ =0, 030) CT=0 CT=1	$Sm = 4, 230$ $-R_2 = 9, 171$ $Sm = 3, 401$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0010, 1011 \ 1101 \ 1000$ $Kop \ 0011, 1101 \ 1101 \ 1101$ $Sm = 0101, 1000 \ 1010 \ 0101$	CT=2	$Sm = 3, 401$ $-R_2 = 9, 171$ $Sm = 2, 572$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0001, 1101 \ 0100 \ 1001$ $Kop \ 0011, 1101 \ 0011 \ 1101$ $Sm = 0100, 1010 \ 0111 \ 0110$	CT=3	$Sm = 2, 572$ $-R_2 = 9, 171$ $Sm = 1, 743$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0000, 1111 \ 0001 \ 1010$ $Kop \ 0011, 1101 \ 0011 \ 1101$ $Sm = 0011, 1100 \ 0100 \ 0111$	CT=4	$Sm = 1, 743$ $-R_2 = 9, 171$ $Sm = 0, 914$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 0000, 0000 \ 1110 \ 1011$ $Kop \ 0011, 0011 \ 1101 \ 1101$ $Sm = 0011, 0011 \ 1011 \ 1000$ (*)	CT=5	$Sm = 0, 914$ $-R_2 = 9, 171$ $Sm = 0, 085$
	$-R_2 = 1100, 0100 \ 1010 \ 0100$ $Sm = 1111, 1000 \ 0101 \ 1100$ $Kop \ 1101, 1101 \ 0011 \ 1101$ $Sm = 1100, 0101 \ 1000 \ 1001$ $+R_2 = 0011, 1011 \ 0101 \ 1100$ $Sm = 0000, 0000 \ 1110 \ 0101$ $Kop \ 0011, 0011 \ 1101 \ 0011$ $Sm = 0011, 0011 \ 1011 \ 1000$ (*)	(Sm<0, конец цикла) $R_1 = 0011, 0011 \ 0110 \ 1000$ (R ₁ =0, 035)	$Sm = 0, 085 \ (*)$ $-R_2 = 9, 171$ $Sm = 9, 256_{(Sm<0)}$ $Sm = 9, 256_{(Sm<0)}$ $+R_2 = 0, 829$ $Sm = 0, 085 \ (*)$

3	Сдвиги: $Sm=0011, 1011 \ 1000 \ 0011$ $-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=0000, 0000 \ 0010 \ 0111$ $Kop \ 0011, 0011 \ 0011 \ 1101$ $Sm=0011, 0011 \ 0101 \ 0100$ (*)	$R_1=0011, 0110 \ 1000 \ 0011$ $(R_1=0, 350)$ CT=1	$Sm=0, 850$ $-R_2=9, 171$ $Sm=0, 021$
	$-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=1111, 0111 \ 1111 \ 1000$ $Kop \ 1101, 1101 \ 1101 \ 1101$ $Sm=1100, 0100 \ 1100 \ 0101$ $+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=0000, 0000 \ 0010 \ 0001$ $Kop=0011, 0011 \ 0011 \ 0011$ $Sm=0011, 0011 \ 0101 \ 0100$ (*)	$(SM<0, \text{конец цикла})$ $R_1=0011, 0110 \ 1000 \ 0100$ $(R_1=0, 351)$	$Sm=0, 021 \ (*)$ $-R_2=9, 171$ $Sm=9, 192 (SM<0)$ $+R_2=0, 829$ $SM=0, 021 \ (*)$ $R_1=0, 351$

Результат : **0,351.**

Восстановление остатка в конце каждого цикла требует определенного времени и поглощает значительный вычислительный ресурс, метод деления без восстановления остатка лишен данных недостатков.

В отличие от метода деления с восстановлением остатка, метод без восстановления остатка различает нечетные циклы (1, 3, 5...), в которых счетчик начинается с 0, после каждого вычитания из сумматора R_2 счетчик увеличивается на 1, пока сумматор не поменяет знак (аналогично методу с восстановлением остатка). Когда сумматор меняет знак, его восстановление не производится, начинается четный цикл (2, 4, 6...), после сдвигов счетчик устанавливается в 9, и к сумматору прибавляется R_2 , счетчик уменьшается на 1 и так, до тех пор, пока сумматор не поменяет знак (теперь на положительный), что означает конец цикла. Остальные действия аналогичны.

Пример 37 иллюстрирует пошаговое выполнение деления двух десятичных чисел $A = 0,291$ и $B = 0,829$ в D8421 методом без восстановления остатка на структуре со сдвигом сумматора, в правом столбце производится проверка всех действий в десятичной системе. В данном примере 1 и 3 циклы (нечетные) аналогичны предыдущему алгоритму. В конце этих циклов восстановление сумматора не выполняется. Как можно заметить, особенно-

стью вычислений в двоично-десятичном коде D8421 является необходимость в некоторых случаях производить многократную коррекцию.

Пример 37.

0	$\begin{array}{r} Sm=0000,0010 \ 1001 \ 0001 \\ R_2=0000,1000 \ 0010 \ 1001 \\ ДК=1111,0111 \ 1101 \ 0111 \\ Кор=\underline{1010,1010 \ 1010 \ 1010} \\ -R_2=\underline{1001,0001 \ 0111 \ 0001} \end{array}$	$\begin{array}{l} R_1=0000,0000 \ 0000 \ 0000 \\ \text{(при данной коррекции} \\ \text{межтетрадные цепи пе-} \\ \text{реносов разорваны)} \end{array}$	$\begin{array}{l} Sm=0,291 \\ R_2=0,829 \\ -R_2=\underline{9,171} \end{array}$
1	Сдвиги: $\begin{array}{r} Sm=0010,1001 \ 0001 \ 0000 \\ -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=\underline{1011,1010 \ 1000 \ 0001} \\ Кор \ \underline{0110,0110} \quad \quad \quad = \\ Sm=\underline{0010,0000 \ 1000 \ 0001} \end{array}$	$\begin{array}{l} R_1=0000,0000 \ 0000 \ 0000 \\ CT=0 \\ \text{(при данной коррекции} \\ \text{и далее межтетрадные} \\ \text{цепи переносов соеди-} \\ \text{нены)} \\ CT=1 \end{array}$	$\begin{array}{l} Sm=2,910 \\ -R_2=\underline{9,171} \\ Sm=\underline{2,081} \end{array}$
	$\begin{array}{r} -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=\underline{1011,0001 \ 1111 \ 0010} \\ Кор=\underline{0110, \quad \quad \quad 0110} \quad \quad \quad = \\ Sm=\underline{0001,0010 \ 0101 \ 0010} \end{array}$	CT=2	$\begin{array}{l} Sm=2,081 \\ -R_2=\underline{9,171} \\ Sm=\underline{1,252} \end{array}$
	$\begin{array}{r} -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=\underline{1010,0011 \ 1100 \ 0011} \\ Кор=\underline{0110, \quad \quad \quad 0110} \quad \quad \quad = \\ Sm=\underline{0000,0100 \ 0010 \ 0011} \\ (*) \end{array}$	CT=3	$\begin{array}{l} Sm=1,252 \\ -R_2=\underline{9,171} \\ Sm=\underline{0,423} \end{array}$
	$\begin{array}{r} -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=\underline{1001,0101 \ 1001 \ 0100} \end{array}$	$\begin{array}{l} (Sm < 0, \text{ конец цикла}) \\ R_1=0000,0000 \ 0000 \ \underline{0011} \end{array}$	$\begin{array}{l} Sm=0,423(*) \\ -R_2=\underline{9,171} \\ SM=\underline{9,594} \\ (Sm < 0) \end{array}$
2	Сдвиги: $\begin{array}{r} Sm=0101,1001 \ 0100 \ 0000 \\ +R_2=\underline{0000,1000 \ 0010 \ 1001} \\ Sm=\underline{0110,0001 \ 0110 \ 1001} \\ Кор=\underline{\quad \quad \quad ,0110} \quad \quad \quad = \\ Sm=\underline{0110,0111 \ 0110 \ 1001} \end{array}$	$\begin{array}{l} R_1=0000,0000 \ 0011 \ 0000 \\ CT=9 \\ \\ CT=8 \end{array}$	$\begin{array}{l} Sm=5,940 \\ +R_2=\underline{0,829} \\ Sm=\underline{6,769} \end{array}$
	$\begin{array}{r} +R_2=\underline{0000,1000 \ 0010 \ 1001} \\ Sm=\underline{0110,1111 \ 1001 \ 0010} \\ Кор=\underline{\quad \quad \quad ,0110} \quad \quad \quad 0110 \\ Sm=\underline{0111,0101 \ 1001 \ 1000} \end{array}$	CT=7	$\begin{array}{l} SM=6,769 \\ +R_2=\underline{0,829} \\ SM=\underline{7,598} \end{array}$
	$\begin{array}{r} +R_2=\underline{0000,1000 \ 0010 \ 1001} \\ Sm=\underline{0111,1101 \ 1100 \ 0001} \\ Кор=\underline{\quad \quad \quad ,0110 \ 0110 \ 0110} \\ SM=\underline{1000,0100 \ 0010 \ 0111} \end{array}$	CT=6	$\begin{array}{l} SM=7,598 \\ +R_2=\underline{0,829} \\ SM=\underline{8,427} \end{array}$

	$\begin{array}{r} +R_2=0000,1000 \ 0010 \ 1001 \\ \hline Sm=1000,1100 \ 0101 \ 0000 \\ \text{Кор}=\underline{\hspace{1cm}},0110 \hspace{1cm} 0110 \\ Sm=1001,0010 \ 0101 \ 0110 \end{array}$	CT=5	$\begin{array}{r} SM=8,427 \\ +R_2=\underline{0,829} \\ SM=9,256 \end{array}$
	$\begin{array}{r} +R_2=0000,1000 \ 0010 \ 1001 \\ \hline Sm=1001,1010 \ 0111 \ 1111 \\ \text{Кор}=\underline{\hspace{1cm}},0110 \hspace{1cm} 0110 \\ Sm=1010,0000 \ 1000 \ 0101 \\ \text{Кор}=\underline{0110},\hspace{1cm}=\hspace{1cm} \\ Sm=\underline{0000},0000 \ 1000 \ 0101 \end{array}$	(SM>0, конец цикла) R1=0000,0000 0011 0101	$\begin{array}{r} SM=9,256 \\ +R_2=\underline{0,829} \\ Sm=\underline{0,085} \end{array}$
3	Сдвиги: $\begin{array}{r} Sm=0000,1000 \ 0101 \ 0000 \\ -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=1001,1001 \ 1100 \ 0001 \\ \text{Кор}=\underline{\hspace{1cm}},\hspace{1cm} 0110 \hspace{1cm}=\hspace{1cm} \\ Sm=1001,1010 \ 0010 \ 0001 \\ \text{Кор}=\underline{\hspace{1cm}},0110 \hspace{1cm}=\hspace{1cm} \\ Sm=1010,0000 \ 0010 \ 0001 \\ \text{Кор}=\underline{0110},\hspace{1cm}=\hspace{1cm} \\ Sm=\underline{0000},0000 \ 0010 \ 0001 \\ (*) \end{array}$	R1=0000,0011 0101 0000 CT=1	$\begin{array}{r} SM=0,850 \\ -R_2=\underline{9,171} \\ SM=\underline{0,021} \end{array}$
	$\begin{array}{r} -R_2=\underline{1001,0001 \ 0111 \ 0001} \\ Sm=1001,0001 \ 1001 \ 0010 \\ +R_2=\underline{0000,1000 \ 0010 \ 1001} \\ Sm=1001,1001 \ 1011 \ 1011 \\ \text{Кор}=\underline{\hspace{1cm}},\hspace{1cm} 0110 \ 0110 \\ Sm=1001,1010 \ 0010 \ 0001 \\ \text{Кор}=\underline{\hspace{1cm}},0110 \hspace{1cm}=\hspace{1cm} \\ Sm=1010,0000 \ 0010 \ 0001 \\ \text{Кор}=\underline{0110},\hspace{1cm}=\hspace{1cm} \\ Sm=\underline{0000},0000 \ 0010 \ 0001 \\ (*) \end{array}$	(SM<0, конец цикла) R1=0000,0011 0101 0001	$\begin{array}{r} SM=0,021 \\ (*) \\ -R_2=\underline{9,171} \\ SM=9,192 (SM<0) \\ +R_2=\underline{0,829} \\ SM=\underline{0,021} \\ (*) \\ R1=\underline{0,351} \end{array}$

Результат : **0,351.**

Пример 38 иллюстрирует пошаговое выполнение деления двух десятичных чисел $A = 0,291$ и $B = 0,829$ в D8421+3 методом без восстановления остатка на структуре со сдвигом сумматора, в правом столбце производится проверка всех действий в десятичной системе. Как можно заметить, при вычислении в двоично-десятичном коде D8421+3 нет необходимости производить многократную коррекцию.

Пример 38.

0	$Sm=0011, 0101 \ 1100 \ 0100$ $R_2=0011, 1011 \ 0101 \ 1100$ $-R_2=1100, 0100 \ 1010 \ 0100$	$R_1=0011, 0011 \ 0011 \ 0011$	$Sm=0, 291$ $R_2=0, 829$ $-R_2=9, 171$
1	Сдвиги: $Sm=0101, 1100 \ 0100 \ 0011$ $-R_2=1100, 0100 \ 1010 \ 0100$ $SM=0010, 0000 \ 1110 \ 0111$ Кор $0011, 0011 \ 1101 \ 1101$ $Sm=0101, 0011 \ 1011 \ 0100$	$R_1=0011, 0011 \ 0011 \ 0011$ $CT=0$ (при данной коррекции и далее междетрадные цепи разорваны) $CT=1$	$Sm=2, 910$ $-R_2=9, 171$ $Sm=2, 081$
	$-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=0001, 1000 \ 0101 \ 1000$ Кор $0011, 1101 \ 0011 \ 1101$ $Sm=0100, 0101 \ 1000 \ 0101$	$CT=2$	$Sm=2, 081$ $-R_2=9, 171$ $Sm=1, 252$
	$-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=0000, 1010 \ 0010 \ 1001$ Кор $0011, 1101 \ 0011 \ 1101$ $Sm=0011, 0111 \ 0101 \ 0110$	$CT=3$	$Sm=1, 252$ $-R_2=9, 171$ $Sm=0, 423$
	$-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=1111, 1011 \ 1111 \ 1010$ Кор $1101, 1101 \ 1101 \ 1101$ $Sm=1100, 1000 \ 1100 \ 0111$	$(Sm < 0, \text{конец цикла})$ $R_1=0011, 0011 \ 0110 \ 0111$	$Sm=0, 423$ $-R_2=9, 171$ $Sm=9, 594$ $(Sm < 0)$
	Сдвиги: $Sm=1000, 1100 \ 0111 \ 0011$ $+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=0100, 0111 \ 1100 \ 1111$ Кор $1101, 0011 \ 1101 \ 1101$ $Sm=1001, 1010 \ 1001 \ 1100$	$R_1=0011, 0011 \ 0110 \ 0111$ $CT=9$	$Sm=5, 940$ $+R_2=0, 829$ $Sm=6, 769$
2	$+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=1101, 0101 \ 1111 \ 1000$ Кор $1101, 0011 \ 1101 \ 0011$ $Sm=1010, 1000 \ 1100 \ 1011$	$CT=8$	$Sm=6, 769$ $+R_2=0, 829$ $SM=7, 598$
	$+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=0110, 0100 \ 0010 \ 0111$ Кор $0011, 0011 \ 0011 \ 0011$ $Sm=1011, 0111 \ 0101 \ 1010$	$CT=7$	$Sm=7, 598$ $+R_2=0, 829$ $Sm=8, 427$
	$+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=0110, 0100 \ 0010 \ 0111$ Кор $0011, 0011 \ 0011 \ 0011$ $Sm=1011, 0111 \ 0101 \ 1010$	$CT=6$	$Sm=8, 427$ $+R_2=0, 829$ $Sm=9, 256$
	$+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=1111, 0010 \ 1011 \ 0110$ Кор $1101, 0011 \ 1101 \ 0011$ $Sm=1100, 0101 \ 1000 \ 1001$	$CT=5$	$Sm=9, 256$ $+R_2=0, 829$ $Sm=0, 085$
	$+R_2=0011, 1011 \ 0101 \ 1100$ $Sm=0000, 0000 \ 1110 \ 0101$ Кор $0011, 0011 \ 1101 \ 0011$ $Sm=0011, 0011 \ 1011 \ 1000$	$(Sm > 0, \text{конец цикла})$ $R_1=0011, 0011 \ 0110 \ 1000$	$Sm=9, 256$ $+R_2=0, 829$ $Sm=0, 085$

3	Сдвиги: $Sm=0011, 1011 \ 1000 \ 0011$ $-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=0000, 0000 \ 0010 \ 0111$ Кор $0011, 0011 \ 0011 \ 1101$ $Sm=0011, 0011 \ 0101 \ 0100$	$R_1=0011, 0110 \ 1000 \ 0011$ <u>CT=1</u>	$Sm=0, 850$ $-R_2=9, 171$ $Sm=0, 021$
	$-R_2=1100, 0100 \ 1010 \ 0100$ $Sm=1111, 0111 \ 1111 \ 1000$ Кор $1101, 1101 \ 1101 \ 1101$ $Sm=1100, 0100 \ 1100 \ 0101$	$(Sm < 0, \text{конец цикла})$ $R_1=0011, 0110 \ 1000 \ 0100$ <u>$R_1=0, 351$</u>	$S = 0, 021$ $-R_2=9, 171$ $Sm=9, 192$ $(Sm < 0)$

Результат : **0,351**.

3.3.2. Деление в двоично-десятичном коде с неподвижным сумматором

Структура устройства представлена на рис. 17. Сдвиги выполняются на всю десятичную секцию (4 разряда).

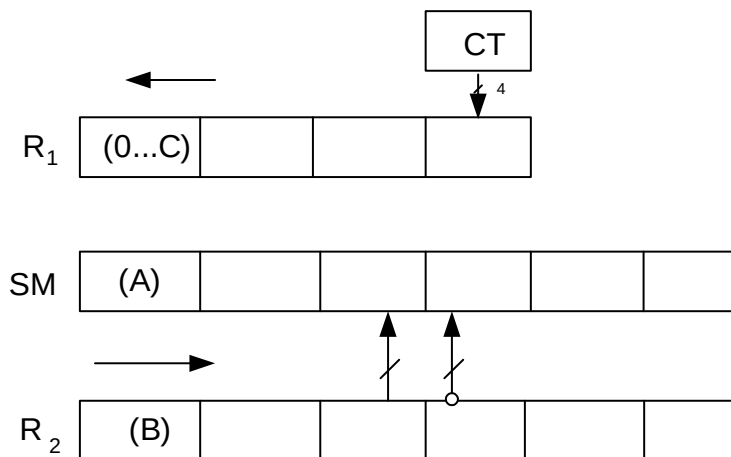


Рис. 17. Структура устройства деления
с неподвижным сумматором

При выполнении операции деления методом с неподвижным сумматором, сумматор Sm остается неподвижным, а сдвигается регистр R₂, R₂ и Sm содержат дополнительные секции для снижения погрешности. Можно использовать как метод с восстановлением остатка, так и метод без

восстановления остатка. В приведённом ниже примере используется метод без восстановления остатка.

Пример 39 иллюстрирует пошаговое выполнение деления двух десятичных чисел $A = 0,291$ и $B = 0,829$ в D8421 методом без восстановления остатка на структуре с неподвижным сумматором, в правом столбце производится проверка всех действий в десятичной системе.

Пример 39.

0	$S_m = 0000, 0010 \ 1001 \ 0001 \mid 0000 \ 0000$ $R_2 = 0000, 1000 \ 0010 \ 1001 \mid 0000 \ 0000$	$R_1 = 0000, 0000 \ 0000 \ 0000$	$S_m = 0, 291 \mid 00$ $R_2 = 0, 829 \mid 00$
1	Сдвиги: $R_2 = 0000, 0000 \ 1000 \ 0010 \mid 1001 \ 0000$ $\neg R_2 = 1111, 1111 \ 0111 \ 1101 \mid 0111 \ 0000$ $Kop = 1010, 1010 \ 1010 \ 1010 \mid 1010 \ 0000$ $\neg R_2 = 1001, 1001 \ 0001 \ 0111 \mid 0001 \ 0000$ $S_m = 0000, 0010 \ 1001 \ 0001 \mid 0000 \ 0000$ $S_m = 1001, 1011 \ 1010 \ 1000 \mid 0001 \ 0000$ $Kop = \underline{\hspace{1cm}}, 0110 \ 0110 \mid \underline{\hspace{1cm}} -$ $S_m = 1010, 0010 \ 0000 \ 1000 \mid 0001 \ 0000$ $Kop = 0110, \underline{\hspace{1cm}} \mid \underline{\hspace{1cm}} -$ $S_m = 0000, 0010 \ 0000 \ 1000 \mid 0001 \ 0000$	$R_1 = 0000, 0000 \ 0000 \ 0000$ $CT = 0$	$R_2 = 0, 082 \mid 90$ $\neg R_2 = 9, 917 \mid 10$ $S_m = 0, 291 \mid 00$ $S_m = 0, 208 \mid 10$
	$\neg R_2 = 1001, 1001 \ 0001 \ 0111 \mid 0001 \ 0000$ $S_m = 1001, 1011 \ 0001 \ 1111 \mid 0010 \ 0000$ $Kop = \underline{\hspace{1cm}}, 0110 \hspace{1cm} 0110 \mid \underline{\hspace{1cm}} -$ $S_m = 1010, 0001 \ 0010 \ 0101 \mid 0010 \ 0000$ $Kop = 0110, \underline{\hspace{1cm}} \mid \underline{\hspace{1cm}} -$ $S_m = 0000, 0001 \ 0010 \ 0101 \mid 0010 \ 0000$	$CT = 1$	$S_m = 0, 208 \mid 10$ $\neg R_2 = 9, 917 \mid 10$ $S_m = 0, 125 \mid 20$
	$\neg R_2 = 1001, 1001 \ 0001 \ 0111 \mid 0001 \ 0000$ $S_m = 1001, 1010 \ 0011 \ 1100 \mid 0011 \ 0000$ $Kop = \underline{\hspace{1cm}}, 0110 \hspace{1cm} 0110 \mid \underline{\hspace{1cm}} -$ $S_m = 1010, 0000 \ 0100 \ 0010 \mid 0011 \ 0000$ $Kop = 0110, \underline{\hspace{1cm}} \mid \underline{\hspace{1cm}} -$ $S_m = 0000, 0000 \ 0100 \ 0010 \mid 0011 \ 0000$	$CT = 2$	$S_m = 0, 125 \mid 20$ $\neg R_2 = 9, 917 \mid 10$ $S_m = 0, 042 \mid 30$
	$\neg R_2 = 1001, 1001 \ 0001 \ 0111 \mid 0001 \ 0000$ $S_m = 1001, 1001 \ 0101 \ 1001 \mid 0100 \ 0000$	$CT = 3$	$S_m = 0, 042 \mid 30$ $\neg R_2 = 9, 917 \mid 10$ $S_m = 9, 959 \mid 40$
	($S_m < 0$, конец цикла) $R_1 = 0000, 0000 \ 0000 \ 0011$		

2	Сдвиги: $R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $-R_2=1001,1001\ 1001\ 0001\ \ 0111\ 0001$ $Sm=1001,1001\ 0101\ 1001\ \ 0100\ 0000$ $+R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $Sm=1001,1001\ 0110\ 0001\ \ 0110\ 1001$ Кор=_____, _____ 0110 _____ $Sm=1001,1001\ 0110\ 0111\ \ 0110\ 1001$	$R_1=0000,0000\ 0011\ 0000$ CT=9	$R_2=0,008\ \ 29$ $-R_2=9,991\ \ 71$ $Sm=9,959\ \ 40$ $+R_2=0,008\ \ 29$ $Sm=9,967\ \ 69$
	$+R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $Sm=1001,1001\ 0110\ 1111\ \ 1001\ 0010$ Кор=_____, _____ 0110 _____ 0110 $Sm=1001,1001\ 0111\ 0101\ \ 1001\ 1000$	CT=8	$Sm=9,967\ \ 69$ $+R_2=0,008\ \ 29$ $Sm=9,975\ \ 98$
	$+R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $Sm=1001,1001\ 0111\ 1101\ \ 1100\ 0001$ Кор=_____, _____ 0110 _____ 0110 $Sm=1001,1001\ 1000\ 0100\ \ 0010\ 0111$	CT=7	$SM=9,975\ \ 98$ $+R_2=0,008\ \ 29$ $SM=9,984\ \ 27$
	$+R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $Sm=1001,1001\ 1000\ 1100\ \ 0101\ 0000$ Кор=_____, _____ 0110 _____ 0110 $Sm=1001,1001\ 1001\ 0010\ \ 0101\ 0110$	CT=6	$Sm=9,984\ \ 27$ $+R_2=0,008\ \ 29$ $SM=9,992\ \ 56$
	$Sm=1001,1001\ 1001\ 0010\ \ 0101\ 0110$ $+R_2=0000,0000\ 0000\ 1000\ \ 0010\ 1001$ $Sm=1001,1001\ 1001\ 1010\ \ 0111\ 1111$ Кор=_____, _____ 0110 _____ 0110 $Sm=1001,1001\ 1010\ 0000\ \ 1000\ 0101$ Кор=_____, _____ 0110 _____ 0 $Sm=1001,1010\ 0000\ 0000\ \ 1000\ 0101$ Кор=_____, 0110 _____ $Sm=1010,0000\ 0000\ 0000\ \ 1000\ 0101$ Кор=0110, _____ _____ $Sm=0000,0000\ 0000\ 0000\ \ 1000\ 0101$	CT=5	$Sm=9,992\ \ 56$ $+R_2=0,008\ \ 29$ $Sm=0,000\ \ 85$
		(Sm>0, конец цикла) $R_1=0000,0000\ 0011\ 0101$	
3	Сдвиги: $R_2=0000,0000\ 0000\ 0000\ \ 1000\ 0010$ $-R_2=1111,1111\ 1111\ 1111\ \ 0111\ 1110$ Кор=1010,1010 1010 1010 1010 1010 $-R_2=1001,1001\ 1001\ 1001\ \ 0001\ 1000$ $Sm=0000,0000\ 0000\ 0000\ \ 1000\ 0101$ $Sm=1001,1001\ 1001\ 1001\ \ 1001\ 1101$ Кор=_____, _____ 0110 $Sm=1001,1001\ 1001\ 1001\ \ 1010\ 0011$ Кор=_____, _____ 0110 $Sm=1001,1001\ 1001\ 1010\ \ 0000\ 0011$ Кор=_____, _____ 0110 _____	$R_1=0000,0011\ 0101\ 0000$ CT=0	$R_2=0,000\ \ 82$ $-R_2=9,999\ \ 18$ $Sm=0,000\ \ 85$ $Sm=0,000\ \ 03$

$Sm = 1001, 1001 \ 1010 \ 0000 \mid 0000 \ 0011$ $Kop = \underline{\hspace{1cm}}, 0 \quad 0110 \mid \underline{\hspace{1cm}} -$ $Sm = 1001, 1010 \ 0000 \ 0000 \mid 0000 \ 0011$ $Kop = \underline{\hspace{1cm}}, 0110 \mid \underline{\hspace{1cm}} -$ $Sm = 1010, 0000 \ 0000 \ 0000 \mid 0000 \ 0011$ $Kop = 0110, \mid \underline{\hspace{1cm}} -$ $Sm = 0000, 0000 \ 0000 \ 0000 \mid 0000 \ 0011$	CT=1	
$-R_2 = 1001, 1001 \ 1001 \ 1001 \mid 0001 \ 0111$ $Sm = 1001, 1001 \ 1001 \ 1001 \mid 0001 \ 1010$ $Kop = \underline{\hspace{1cm}}, \mid \quad 0110$ $Sm = 1001, 1001 \ 1001 \ 1001 \mid 0010 \ 0000$	$(Sm < 0 \text{ конец цикла})$ $R_1 = 0000, 0011 \ 0101 \ 0001$	$Sm = 0, 000 \mid 03$ $-R_2 = 9, 999 \mid 17$ $Sm = 9, 999 \mid 20$

Результат : **0,351.**

Пример 40 иллюстрирует пошаговое выполнение деления двух десятичных чисел $A = 0,291$ и $B = 0,829$ в D8421+3 методом без восстановления остатка на структуре с неподвижным сумматором, в правом столбце производится проверка всех действий в десятичной системе.

Пример 40.

0	$Sm = 0011, 0101 \ 1100 \ 0100 \mid 0011 \ 0011$ $R_2 = 0011, 1011 \ 0101 \ 1100 \mid 0011 \ 0011$	$R_1 = 0011, 0011 \ 0011 \ 0011$	$Sm = 0, 291 \mid 00$ $R_2 = 0, 829 \mid 00$
1	Сдвиги: $R_2 = 0011, 0011 \ 1011 \ 0101 \mid 1100 \ 0011$ $-R_2 = 1100, 1100 \ 0100 \ 1010 \mid 0100 \ 0011$ $Sm = 0011, 0101 \ 1100 \ 0100 \mid 0011 \ 0011$ $Sm = 0000, 0010 \ 0000 \ 1110 \mid 0111 \ 0110$ $Kop = 0011, 0011 \ 0011 \ 1101 \mid 1101 \ 1101$ $Sm = 0011, 0101 \ 0011 \ 1011 \mid 0100 \ 0011$	$R_1 = 0011, 0011 \ 0011 \ 0011$ CT=0 CT=1	$R_2 = 0, 082 \mid 90$ $-R_2 = 9, 917 \mid 10$ $Sm = 0, 291 \mid 00$ $Sm = 0, 208 \mid 10$
	$-R_2 = 1100, 1100 \ 0100 \ 1010 \mid 0100 \ 0011$ $Sm = 0000, 0001 \ 1000 \ 0101 \mid 1000 \ 0110$ $Kop = 0011, 0011 \ 1101 \ 0011 \mid 1101 \ 1101$ $Sm = 0011, 0100 \ 0101 \ 1000 \mid 0101 \ 0011$	CT=2	$Sm = 0, 208 \mid 10$ $-R_2 = 9, 917 \mid 10$ $SM = 0, 125 \mid 20$
	$-R_2 = 1100, 1100 \ 0100 \ 1010 \mid 0100 \ 0011$ $Sm = 0000, 0000 \ 1010 \ 0010 \mid 1001 \ 0110$ $Kop = 0011, 0011 \ 1101 \ 0011 \mid 1101 \ 1101$ $Sm = 0011, 0011 \ 0111 \ 0101 \mid 0110 \ 0011$	CT=3	$Sm = 0, 125 \mid 20$ $-R_2 = 9, 917 \mid 10$ $Sm = 0, 042 \mid 30$
	$-R_2 = 1100, 1100 \ 0100 \ 1010 \mid 0100 \ 0011$		$Sm = 0, 042 \mid 30$

	$Sm=1111, 1111 \ 1011 \ 1111$ $Kop=1101, 1101 \ 1101 \ 1101$ $Sm=1100, 1100 \ 1000 \ 1100$	$1010 \ 0110$ $1101 \ 1101$ $0111 \ 0011$	$(Sm < 0, \text{конец цикла})$ $R_1=0011, 0011 \ 0011 \ 0110$	$-R_2=9, 917$ $Sm=9, 959$	10 40
2	Сдвиги: $R_2=0011, 0011 \ 0011 \ 1011$ $-R_2=1100, 1100 \ 1100 \ 0100$ $Sm=1100, 1100 \ 1000 \ 1100$ $+R_2=0011, 0011 \ 0011 \ 1011$ $Sm=1111, 1111 \ 1100 \ 0111$ $Kop=1101, 1101 \ 1101 \ 0011$ $Sm=1100, 1100 \ 1001 \ 1010$	$0101 \ 1100$ $1010 \ 0100$ $0111 \ 0011$ $0101 \ 1100$ $1100 \ 1111$ $1101 \ 1101$ $1001 \ 1100$	$R_1=0011, 0011 \ 0110 \ 0011$ $CT=9$ $CT=8$	$R_2=0, 008$ $-R_2=9, 991$ $Sm=9, 959$ $+R_2=0, 008$ $Sm=9, 967$	29 71 40 29 69
	$+R_2=0011, 0011 \ 0011 \ 1011$ $Sm=1111, 1111 \ 1101 \ 0101$ $Kop=1101, 1101 \ 1101 \ 0011$ $Sm=1100, 1100 \ 1010 \ 1000$	$0101 \ 1100$ $1111 \ 1000$ $1101 \ 0011$ $1100 \ 1011$	 $CT=7$	$Sm=9, 967$ $+R_2=0, 008$ $Sm=9, 975$	69 29 98
	$+R_2=0011, 0011 \ 0011 \ 1011$ $Sm=1111, 1111 \ 1110 \ 0100$ $Kop=1101, 1101 \ 1101 \ 0011$ $Sm=1100, 1100 \ 1011 \ 0111$	$0101 \ 1100$ $0010 \ 0111$ $0011 \ 0011$ $0101 \ 1010$	 $CT=6$	$Sm=9, 975$ $+R_2=0, 008$ $Sm=9, 984$	98 29 27
	$+R_2=0011, 0011 \ 0011 \ 1011$ $SM=1111, 1111 \ 1111 \ 0010$ $Kop=1101, 1101 \ 1101 \ 0011$ $SM=1100, 1100 \ 1100 \ 0101$	$0101 \ 1100$ $1011 \ 0110$ $1101 \ 0011$ $1000 \ 1001$	 $CT=5$	$Sm=9, 984$ $+R_2=0, 008$ $Sm=9, 992$	27 29 56
	$+R_2=0011, 0011 \ 0011 \ 1011$ $Sm=0000, 0000 \ 0000 \ 0000$ $Kop=0011, 0011 \ 0011 \ 0011$ $Sm=0011, 0011 \ 0011 \ 0011$	$0101 \ 1100$ $1110 \ 0101$ $1101 \ 0011$ $1011 \ 1000$	 $(Sm > 0, \text{конец цикла})$ $R_1=0011, 0011 \ 0110 \ 1000$	$Sm=9, 992$ $+R_2=0, 008$ $Sm=0, 000$	56 29 85
3	Сдвиги: $R_2=0011, 0011 \ 0010 \ 0011$ $-R_2=1100, 1100 \ 1100 \ 1100$ $Sm=0011, 0011 \ 0011 \ 0011$ $Sm=0000, 0000 \ 0000 \ 0000$ $Kop=0011, 0011 \ 0011 \ 0011$ $Sm=0011, 0011 \ 0011 \ 0011$	$1011 \ 0101$ $0100 \ 1011$ $1011 \ 1000$ $0000 \ 0011$ $0011 \ 0011$ $0011 \ 0110$	$R_1=0011, 0110 \ 1000 \ 0011$ $CT=0$ $CT=1$	$R_2=0, 000$ $-R_2=9, 999$ $Sm=0, 000$ $Sm=0, 000$	82 18 85 03
	$R_2=1100, 1100 \ 1100 \ 1100$ $Sm=1111, 1111 \ 1111 \ 1111$ $Kop=1101, 1101 \ 1101 \ 0011$ $Sm=0011, 0011 \ 0011 \ 0011$	$0100 \ 1011$ $1000 \ 0001$ $1101 \ 0011$ $0101 \ 0100$	 $(Sm < 0 \text{ конец цикла})$ $R_1=0011, 0110 \ 1000 \ 0100$	$Sm=0, 000$ $-R_2=9, 999$ $Sm=9, 999$	03 18 21

Результат : **0,351.**

Вопросы и задания

1. Поделить указанными методами два ненулевых десятичных числа в D8421 с тремя десятичными знаками после запятой ($|A| < |B|$), на каждом шаге выполнять десятичную проверку:

- a) с подвижным сумматором с восстановлением остатка;
- b) с подвижным сумматором без восстановления остатка;
- c) с неподвижным сумматором с восстановлением остатка;
- d) с неподвижным сумматором без восстановления остатка.

2. Поделить указанными методами два ненулевых десятичных числа в D8421+3 с тремя десятичными знаками после запятой ($|A| < |B|$), на каждом шаге выполнять десятичную проверку:

- a) с подвижным сумматором с восстановлением остатка;
- b) с подвижным сумматором без восстановления остатка;
- c) с неподвижным сумматором с восстановлением остатка;
- d) с неподвижным сумматором без восстановления остатка.

Выполнить общую проверку на микрокалькуляторе.

4. РАЗРАБОТКА СТРУКТУРЫ И АЛГОРИТМА ФУНКЦИОНИРОВАНИЯ АРИФМЕТИЧЕСКОГО УСТРОЙСТВА

В качестве примера рассмотрим более детально разработку устройства деления в прямых кодах без восстановления остатка с неподвижным сумматором и управляющего алгоритма (см. 2.3.2, пример 15).

4.1. РАЗРАБОТКА СТРУКТУРЫ УСТРОЙСТВА

Полная структура устройства (рис. 18), выполняющего операцию деления по указанному алгоритму должна содержать часть, отвечающую за работу с мантиссами, и часть, отвечающую за работу с порядками, триггер для хранения знака результата.

Часть, отвечающая за работу с мантиссами, содержит сумматор мантисс SM, регистры мантисс RM1, RM2, триггер для хранения знака мантиссы Т и счетчик циклов СТ.

При реализации операции деления $C = A / B$ в сумматор SM загружается мантисса делимого (mA), в регистр RM2 загружается мантисса делителя, регистр RM1 обнуляется, в нем будет формироваться мантисса частного (mC).

Сигналы с первых (знаковых) разрядов SM и RM2 подключаются к логическому элементу «сумма по модулю два», выход элемента подключен ко входу триггера. Данная схема позволяет вычислить знак результата на основе знаков операндов и сохранить его в триггере. После сохранения знака результата в триггере знаки операндов можно обнулить.

Сумматор SM и регистр RM2 имеют индикаторы нуля. При $SM=0$ формируется логическое условие x_1 , при $RM2 = 0$ формируется логическое условие x_2 .

От регистра RM2 к сумматору SM идут две многоразрядные связи – прямая и инверсная (предполагается подключение через блок инверторов).

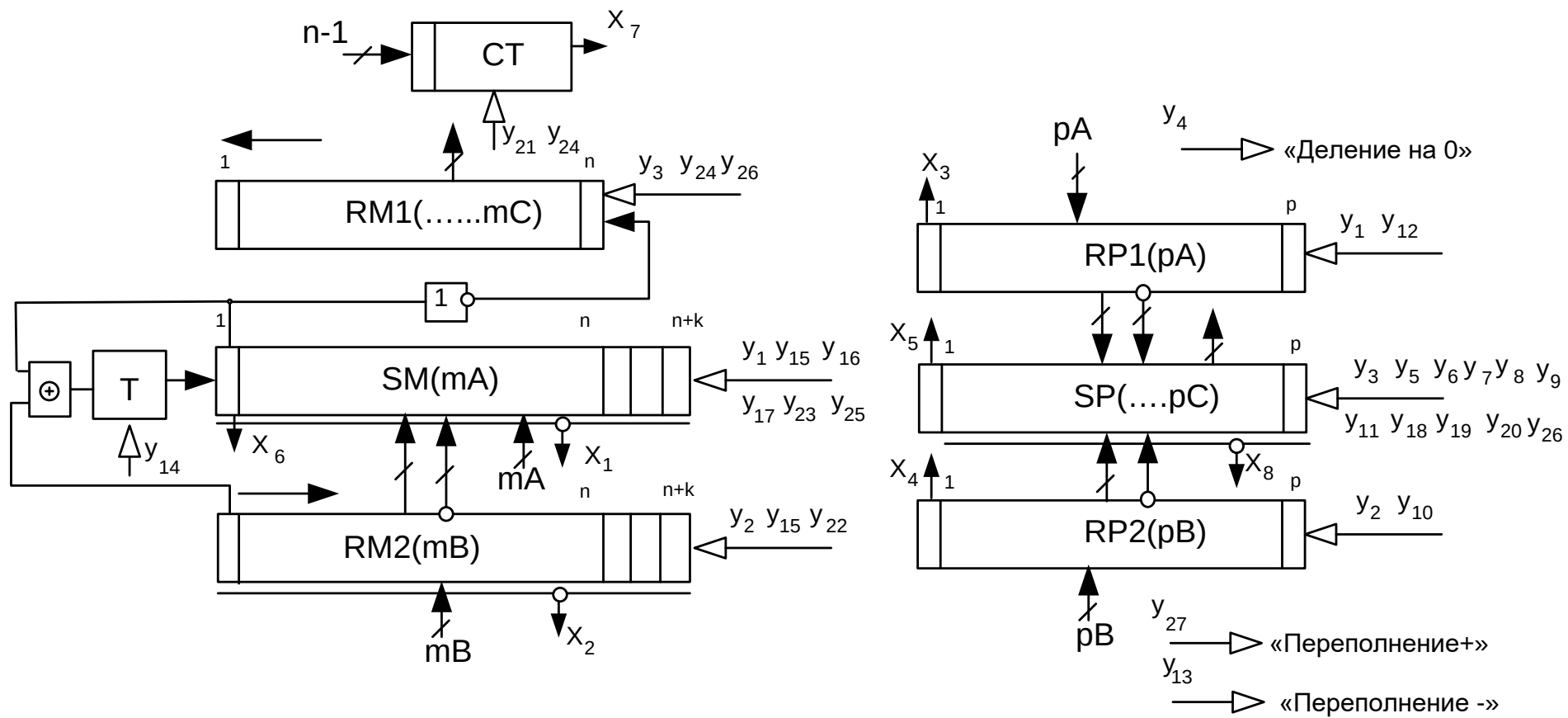


Рис. 18. Полная структурная схема устройства деления чисел с плавающей запятой

Прямая связь обеспечивает операцию $SM = SM + RM2$, инверсная связь – операцию $SM = SM - RM2$, которая реализуется через дополнительный код как $SM = SM + \overline{RM2} + 2^n$.

Счетчик СТ используется для подсчета необходимого числа циклов, на счетчик подаются два управляющих сигнала: сигнал y_{21} загружает необходимое число циклов, соответствующее числу разрядов числа, за вычетом знакового (то есть в счетчик загружается $n-1$); сигнал y_{24} производит декремент счетчика (уменьшение на 1). Когда счетчик обнуляется, активизируется логическое условие x_7 .

Часть, отвечающая за работу с порядками, содержит сумматор порядков SP, регистры порядков RP1 и RP2.

Сумматор порядков SP содержит индикатор нуля, настроенный на проверку всех разрядов за исключением знакового (за исключением первого), когда все эти разряды равны 0 активизируется логическое условие x_8 .

К сумматору SP от регистров RP1 и RP2 идут соответственно по две многоразрядные связи (прямая и инверсная), таким образом, обеспечивается реализация операций: $SP = SP + RP1$, $SP = SP - RP1$, $SP = SP + RP2$, $SP = SP - RP2$, а также операции загрузки $SP = RP1$, $SP = RP2$.

Кроме того, устройство вырабатывает сигналы исключительных ситуаций: «деление на 0», «Переполнение +», «Переполнение –».

4.2. РАЗРАБОТКА АЛГОРИТМА ФУНКЦИОНИРОВАНИЯ УСТРОЙСТВА

На рис. 19–24 показана схема алгоритма выполнения операции деления с неподвижным сумматором чисел с плавающей запятой в прямых кодах. Алгоритм содержит этапы: загрузка данных, проверка операндов на ноль, работа с порядками, проверка на возможность деления, собственно деление мантисс, вывод данных.

На рис. 19 приведена схема первого этапа работы алгоритма деления (ввод данных и начальная проверка).

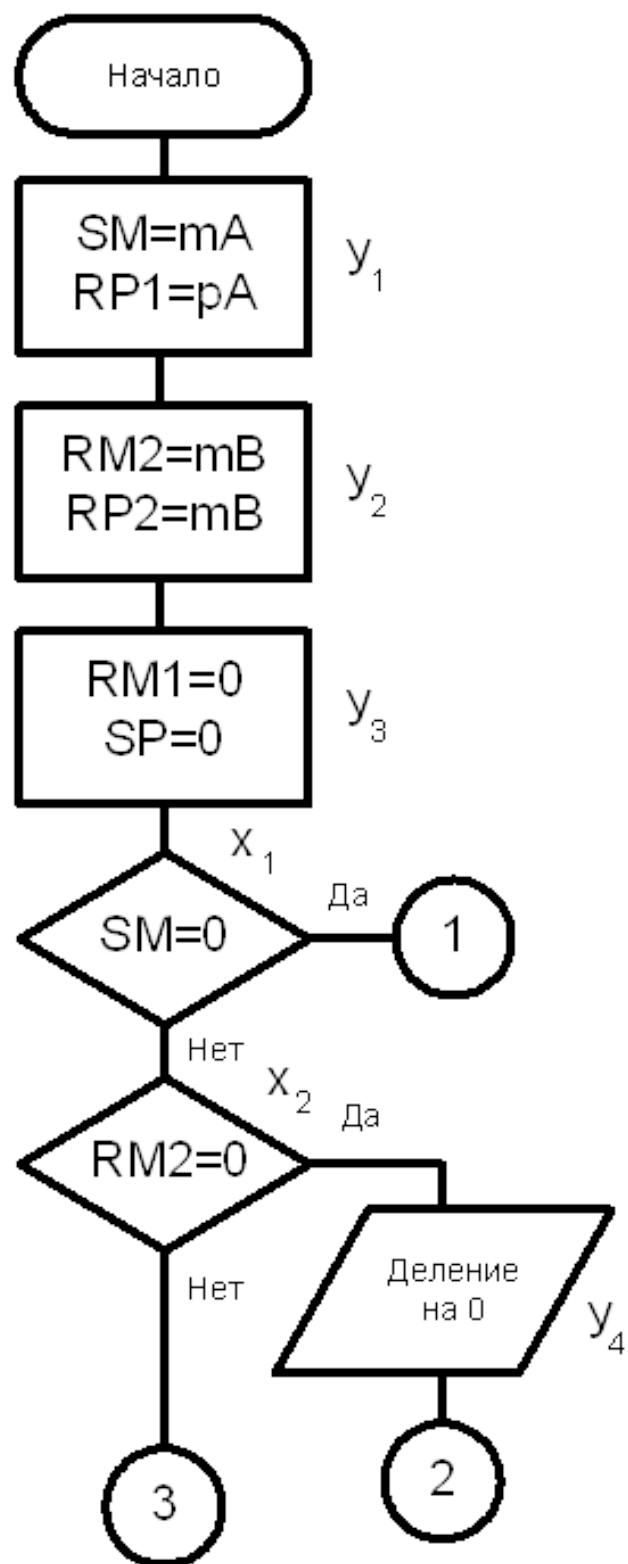


Рис. 19. Ввод данных и начальная проверка

Сначала производится загрузка данных, мантисса A загружается в сумматор мантисс SM , порядок в – $RP1$. Мантисса B загружается в регистр мантисс $RM2$, порядок – в $RP2$. Как правило, операнды загружаются

с одной шины устройства, поэтому загрузку операндов А и В нужно разделить во времени и управлять с помощью различных сигналов (y_1 и y_2). Порядок и мантиссу можно загружать в устройство одновременно, поскольку загрузка происходит в разные регистры по разным шинам. В соответствии со структурой устройства сигнал y_1 нужно подавать к сумматору мантисс SM и к регистру порядка RP1, сигнал y_2 – к регистрам RM2 и RP2.

Следующее действие – одновременно обнуляются регистр RM1, где будет формироваться мантисса результата, и сумматор порядков SP, где будет формироваться порядок результата, соответственно сигнал y_3 подается на RM1 и SP.

При реализации длинных операций рекомендуется выполнять проверку операндов на ноль, во-первых, это позволяет в некоторых случаях сократить время выполнения операции, в некоторых случаях избежать закликивания. Для реализации подобной проверки на ноль в структуре сумматора SM и регистра RM2 содержатся схемы – детекторы нуля, реализованные в простейшем случае на базе многовходового элемента ИЛИ.

Сначала выполняется проверка SM на ноль, если проверка подтверждается, осуществляется переход на вывод результата (в регистрах RM1 и SP находятся нулевые значения, они и выведутся.) Далее проверяется RM2 на ноль, если проверка подтверждается, выводится сообщение об ошибке «деление на ноль».

Далее на рис. 20 приводится схема алгоритма работы с порядками. При выполнении операции деления в формате с плавающей запятой порядок результата находится как $pC = pA - pB$, но поскольку порядки в нашем случае целые числа, представленные в прямом коде, это действие разворачивается в довольно большой алгоритм. В зависимости от знаков операндов (соответственно первые разряды регистров RP1 и RP2 – логические условия x_3 и x_4) с помощью оператора CASE рассматриваются все возможные варианты (их всего четыре).

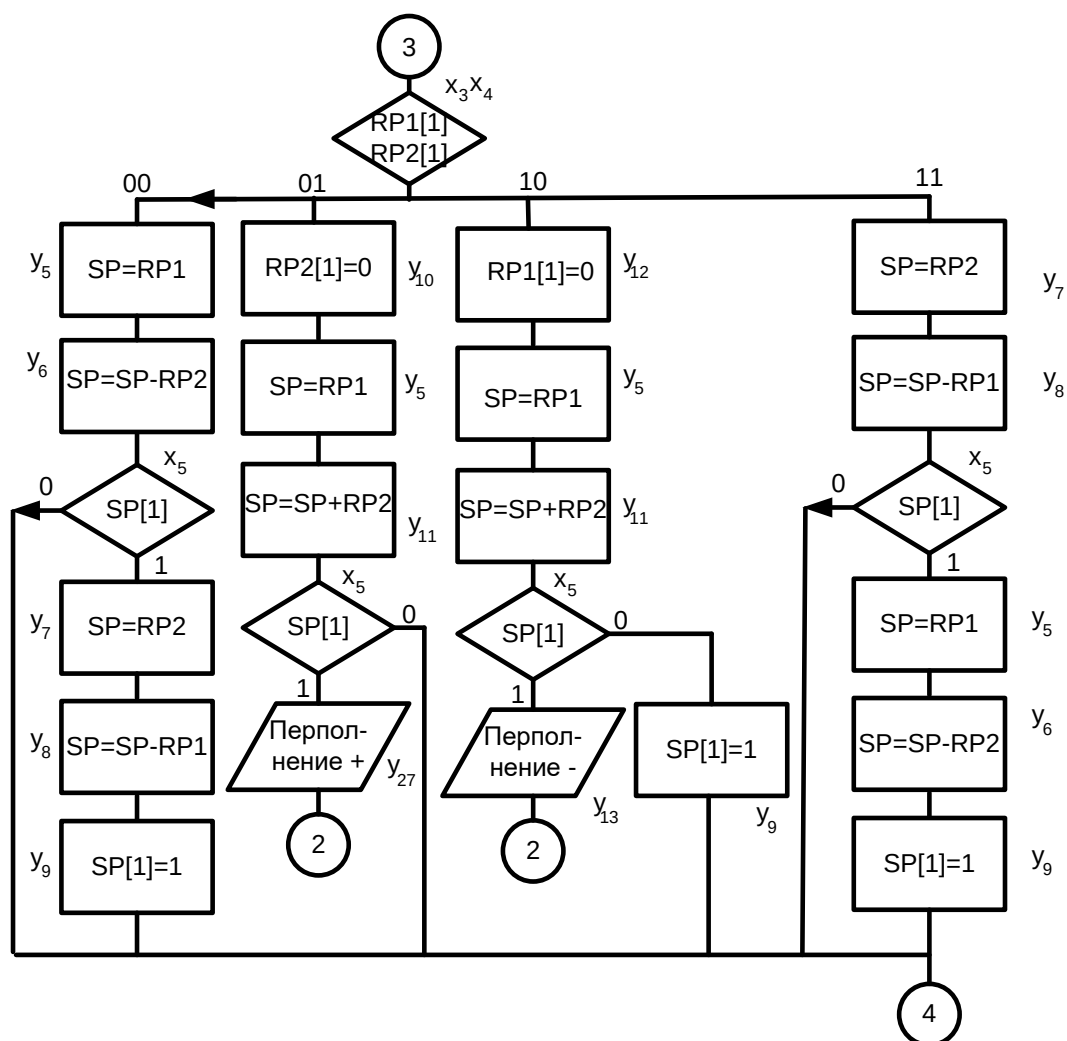


Рис. 20. Работа с порядками

Ветвь «00» – порядки положительны. Находим разность $RP1$ и $RP2$, причем в два этапа: на первом этапе содержимое $RP1$ загружается в сумматор порядков SP , на втором этапе производится вычитание SP и $RP2$ (т. е. выполняется $SP = SP + \overline{RP2} + 1$). Результат может получиться отрицательный, и если он отрицательный, то будет представлен в дополнительном коде, а поскольку в нашем случае используется прямой код, то легче заново пересчитать результат. Если знаковый разряд SP (условие x_5) равен 1 (результат отрицательный), пересчитываем порядок: загружаем в SP уже $RP2$, находим разность с $RP1$ и принудительно присваиваем знаковый разряд 1, таким образом, получаем отрицательное число в прямом коде.

Ветвь «01» – порядок в $RP1$ положительный, в $RP2$ – отрицательный, т. е. чтобы найти разность $RP1$ и $RP2$, к $RP1$ прибавляется $-RP2$, которое

в данном случае получается занулением знакового разряда RP2, то есть находится сумма двух положительных чисел. После сложения на сумматоре анализируется знак результата на предмет переполнения, если знак окажется 1, выдается сообщение о переполнении («переполнение +»), и работа алгоритма завершается, если переполнения нет, работа алгоритма продолжается.

Ветвь «10» – порядок в RP2 положительный, в RP1 – отрицательный. Здесь аналогично, чтобы найти разность RP2 и RP1, к RP2 прибавляется RP1, которое получается занулением знакового разряда RP1, находим сумму двух положительных чисел (при этом зная, что они на самом деле отрицательные). После сложения на сумматоре анализируется знак результата на предмет переполнения, если знак окажется 1, выдается сообщение о переполнении («переполнение –»), и работа алгоритма завершается. Рекомендуется различать положительное переполнение порядков (машинная бесконечность) и отрицательное переполнение порядков (машинный ноль), во втором случае результат операции можно просто считать нулевым (на самом деле он просто настолько мал, что не фиксируется заданной разрядной сеткой). В случае отсутствия переполнения принудительно присваивается отрицательный знак результату порядков.

Ветвь «11» – порядки отрицательны. Находим разность RP2 и RP1, причем в два этапа: на первом этапе содержимое RP2 загружается в сумматор порядков SP, на втором этапе производится вычитание SP и RP1 (т. е. выполняется $SP = SP + \overline{RP1} + 1$). Результат может получиться отрицательный, и если он отрицательный, то будет представлен в дополнительном коде, а поскольку в нашем случае используется прямой код, то легче заново пересчитать результат. Если знаковый разряд SP (условие x_5) равен 1 (результат отрицательный, знаковые разряды компенсируют друг друга), пересчитываем порядок: загружаем в SP уже RP1, находим разность с RP2 и принудительно присваиваем знаковый разряд 1, таким образом, получаем отрицательное число в прямом коде.

Далее (рис. 21) в триггер загружается знак мантиссы результата, вычисляемой на основе знаков операндов с помощью элемента, реализующего функцию «сумма по модулю два», после чего знаковые разряды операндов в SM и RM2 обнуляются.

На следующем этапе при необходимости производится денормализация делимого (рис. 21).

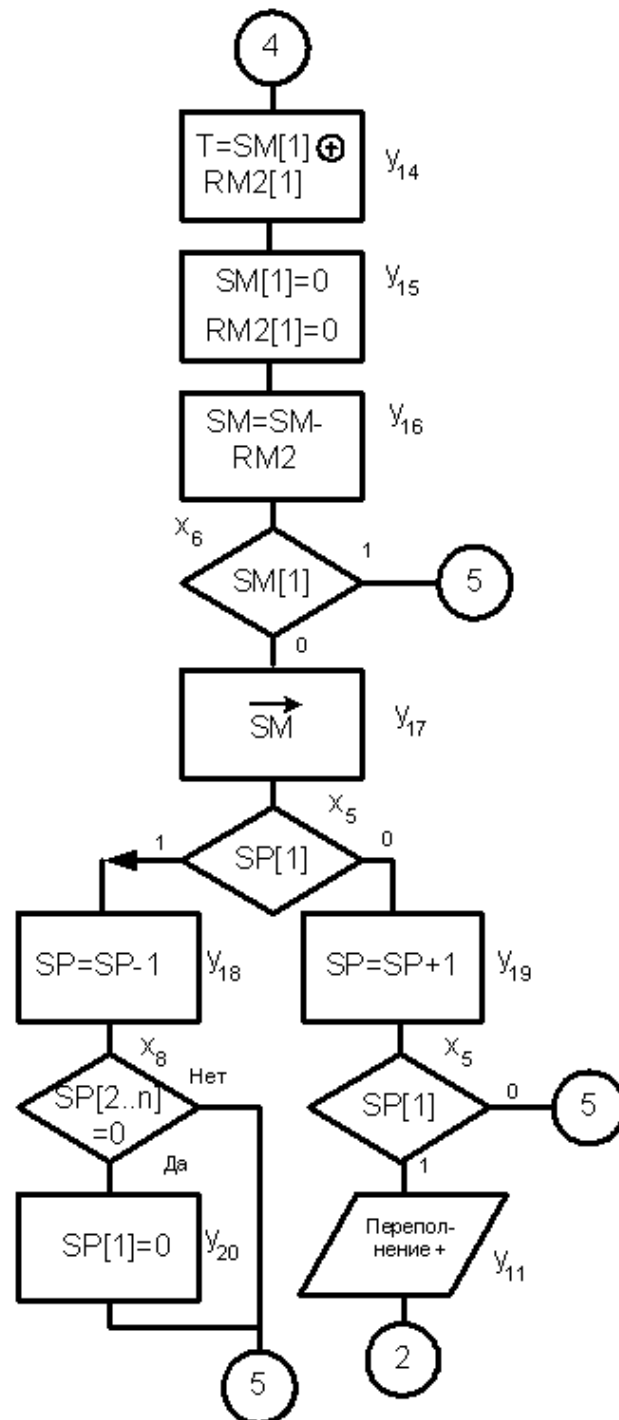


Рис. 21. Денормализация делимого

С целью обеспечения в результате нормализованного частного необходимо, чтобы соблюдалось условие $|mA| < |mB|$. Для проверки этого из сумматора мантисс вычитается RM2 и анализируется знак результата, если результат отрицательный (логическое условие x_6), условие $|mA| < |mB|$ выполняется, и осуществляется переход непосредственно к выполнению алгоритма деления над мантиссами, при этом восстанавливать сумматор нет необходимости, поскольку далее используется алгоритм деления без восстановления остатка. Если условие $|mA| < |mB|$ не выполняется ($x_6 = 0$), производится денормализация делимого на сумматоре SM, сумматор SM сдвигается в сторону младших разрядов, а порядок на SP увеличивается на единицу. При этом, если порядок положительный, он увеличивается на единицу, отслеживается возможное переполнение порядков; если порядок отрицательный, он уменьшается на единицу, отслеживается возможный случай перехода порядка из положительного в отрицательный, при таком переходе в прямых кодах необходимо принудительно установить знак порядка в 0.

Основная часть алгоритма деления мантисс показана на рис. 22.

Сначала в счетчик СТ заносится необходимое число циклов, соответствующих числу цифр частного, поскольку один разряд отводится под знак, получается $n-1$ циклов.

Далее выполняется собственно алгоритм деления мантисс (рис. 22).

Поскольку сумматор мантисс остается неподвижным, сдвигается регистр RM2, содержащий делимое.

При выполнении алгоритма без восстановления остатка частичный остаток на сумматоре может быть как положительный, так и отрицательный, поэтому сначала определяется знак сумматора SM. Если он положительный ($x_6 = 0$), производится вычитание сдвинутого на RM2 делителя из сумматора; если он отрицательный ($x_6 = 1$), производится сложение сдвинутого на RM2 делителя и сумматора. В любом случае инверсия знака на сумматоре (это и есть очередной разряд частного) задвигается в регистр RM1 со стороны младших разрядов.

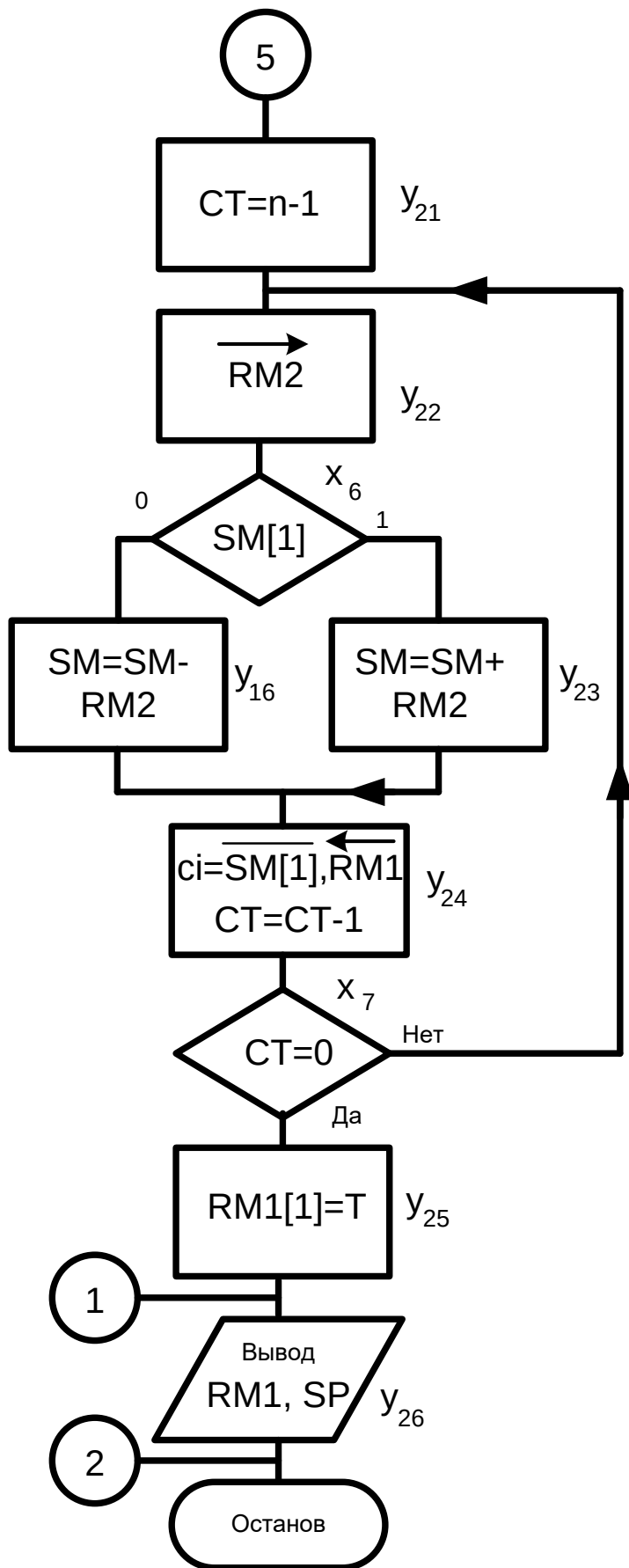


Рис. 22. Деление мантисс и вывод

Далее содержимое счетчика уменьшается на 1 и проверяется на ноль содержимое счетчика: если счетчик не равен нулю ($x_7 = 0$), начинается новый цикл; если счетчик равен нулю ($x_7 = 1$), то есть все разряды частного получены, присваивается знак результата, сохраненный в триггере, и выводится результат. Мантисса результата находится в регистре RM1, порядок на сумматоре порядков SP.

Алгоритм заканчивается.

4.3. ПОСТРОЕНИЕ ТАБЛИЦ УСЛОВИЙ И СИГНАЛОВ

Сведем в одну таблицу все логические условия, формируемые устройством (табл. 8).

Таблица 8

Логические условия

Обозначение	Сокращенная запись	Комментарии
x_1	SM = 0	Проверка сумматора мантисс (делимого) на ноль
x_2	RM2 = 0	Проверка регистра делителя на ноль
x_3	RP1[1]	Знак порядка делимого
x_4	RP2[1]	Знак порядка делителя
x_5	SP[1]	Знак сумматора порядка
x_6	SM[1]	Знак сумматора мантисс
x_7	CT=0	Проверка счетчика на ноль
x_8	SP[2...p]	Проверка сумматора порядков на ноль (без учета знакового разряда)

Сведем в одну таблицу все управляющие сигналы, формируемые устройством (табл. 9).

Управляющие сигналы

Обозначение	Сокращенная запись	Комментарии
y_1	$SM = mA, RP1 = pA$	Загрузка мантиссы и порядка A
y_2	$RM2 = mB, RP2 = pB$	Загрузка мантиссы и порядка B
y_3	$RM1 = 0, SP = 0$	Сброс RM1 и SP
y_4	Деление на 0	Вывод сигнала «Деление на 0»
y_5	$SP = RP1$	Загрузка сумматора SP из RP1
y_6	$SP = SP - RP2$	Вычитание SP и RP2, результат в SP (содержимое RP2 не меняется)
y_7	$SP = RP2$	Загрузка сумматора SP из RP2
y_8	$SP = SP - RP1$	Вычитание SP и RP1, результат в SP (содержимое RP1 не меняется)
y_9	$SP[1] = 1$	Установка знакового разряда SP
y_{10}	$RP2[1] = 0$	Сброс знакового разряда RP2
y_{11}	$SP = SP + RP2$	Сумма SP и RP2, результат в SP (содержимое RP2 не меняется)
y_{12}	$RP1[1] = 0$	Сброс знакового разряда RP1
y_{13}	Переполнение -	Вывод сигнала «Переполнение -»
y_{14}	$T = SM[1] \oplus RM2[1]$	Получение знака результата и сохранение его в триггере
y_{15}	$SM[1] = 0, RM2[1] = 0$	Сброс знаковых разрядов SM и RM2
y_{16}	$SM = SM - RM2$	Вычитание SM и RM2, результат в SM
y_{17}	$SM \rightarrow$	Сдвиг SM вправо
y_{18}	$SP = SP + 1$	Увеличение SP на 1
y_{19}	$SP = SP - 1$	Уменьшение SP на 1
y_{20}	$SP[1] = 0$	Сброс знакового разряда SP

Обозначение	Сокращенная запись	Комментарии
y_{21}	$CT = n-1$	Загрузка счетчика (n-1 циклов)
y_{22}	$RM2 \rightarrow$	Сдвиг $RM2$ вправо
y_{23}	$SM = SM + RM1$	Сложение SM и $RM2$, результат в SM
y_{24}	$c_i = \overline{SM[1]}, RM1 \leftarrow$	В $RM1$ задвигается очередной разряд частного c_i
y_{25}	$RM1[1] = T$	Присвоение знака результата
y_{26}	Вывод $RM1, SP$	Вывод мантиссы и порядка результата
y_{27}	Переполнение +	Вывод сигнала «Переполнение +»

Вопросы и задания

На основе описанного примера организации устройства разработайте структуру и алгоритм функционирования устройства деления указанным методом:

- с подвижным сумматором с восстановлением остатка;
- с подвижным сумматором без восстановления остатка;
- с неподвижным сумматором с восстановлением остатка;
- ускоренным методом с анализом одного разряда после запятой;
- ускоренным методом с анализом двух разрядов после запятой.

ЗАКЛЮЧЕНИЕ

Рассмотренные в учебном пособии базовые аспекты компьютерной арифметики и логики при глубоком их изучении студентами в первом и втором семестрах составляют фундамент их общепрофессиональной подготовки и в значительной степени влияют на качество подготовки специалистов.

Учебное пособие построено так, что каждый теоретический материал, читаемый на лекциях, подкреплён многочисленными примерами, используемыми при проведении практических и лабораторных занятий и выполнении расчетно-графических работ. При этом лабораторные работы выполняются в последующих курсах «Элементная база вычислительной техники», «Прикладная теория цифровых автоматов» в программных симуляторах на персональных компьютерах.

Учебное пособие составлено так, что позволяет проводить аттестацию студентов по отдельным разделам дисциплин по мере их изучения и тем самым повысить качество аттестации студентов.

Изучаемые первоначально фундаментальные понятия информатики, системы счисления, алгоритмы перевода чисел из любой системы счисления в любую другую, рассматриваемые алгоритмы выполнения основных арифметических операций над числами в двоичной и десятичной системе с фиксированной и плавающей запятой, а также многочисленные алгоритмы ускоренного выполнения операций компьютерной арифметики используются далее в курсах, связанных с программированием и использованием микропроцессорных систем. Полученные базовые знания по реализации машинных операций используются студентами для разработки соответствующих программ на ассемблерах микропроцессорных систем.

Следующей ступенью изучения материала учебного пособия являются проработка более сложных алгоритмов двоично-десятичной арифметики, основных понятий математической логики и переключательных (булевых) функций, а также изучение стадий логического проектирования комбинационных схем от простейших операционных устройств к сложным. Закрепление у студентов полученных на данном этапе теоретических знаний осуществляется с использованием систем схемотехнического моделирования персональных ЭВМ в рамках работ проектной деятельности и выполнения выпускной квалификационной работы.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Потапов, В. И. Компьютерная арифметика и алгоритмическое моделирование арифметических операций : учеб. пособие / В. И. Потапов, О. П. Шафеева ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2014. – 1 CD-ROM. (936 Кб). – Загл. с этикетки диска. – ISBN 5-8149-0208-6.

2. Червенчук, И. В. Основы компьютерной логики : учеб. пособие / И. В. Червенчук, О. П. Шафеева, А. С. Грицай ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2019. – 1 CD-ROM (0,86 Мб). – Загл. с этикетки диска. – ISBN 978-5-8149-2801-6.

3. Потапов, В. И. Компьютерная арифметика и алгоритмическое моделирование арифметических операций : учеб. пособие / В. И. Потапов, О. П. Шафеева ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2005. – 95 с. – ISBN 5-8149-0208-6.

4. Потапов, В. И. Основы компьютерной арифметики и логики : учеб. пособие / В. И. Потапов, О. П. Шафеева, И. В. Червенчук ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2004. – 172 с.

5. Разработка арифметических устройств : метод. указания / Ом. гос. техн. ун-т. ; сост. И. В. Червенчук. – Омск : Изд-во ОмГТУ, 2017. – 32 с.